

Cartographier avec Atlas SQL

Le guide complet : inventaire, cartographie, analyse du code T-SQL, dépendances, dictionnaire RGPD, performances, débogage, refactoring et interception d'un patrimoine SQL Server, expliqués pas à pas avec des exemples réels.

21 chapitres

64 exemples exécutés

26 captures d'écran

18 modules

3 à 10 agents parallèles

- Démarrer
- L'interface
- Les sources
- La file d'attente
- L'entrepôt DuckDB
- L'analyse du code
- Les constats
- Les dépendances
- La cartographie
- Le dictionnaire
- Les types métier
- Les flux de données
- Le SQL dynamique
- Les performances
- Les doublons
- Le patrimoine
- L'éditeur et le débogueur
- Le refactoring
- L'interception
- L'assistant
- Aller plus loin

Atlas SQL · un programme Lexis+
Comprenez, documentez et transformez votre patrimoine SQL Server

© 2026 Jean-Marc QUÉRÉ
sonaliwan.fr

Bienvenue dans Atlas SQL

Une base SQL Server qui vit depuis dix ans ressemble à une ville : des centaines de tables, de vues et de procédures, des jobs nocturnes, des serveurs liés, des synonymes, du SQL construit à la volée… et plus personne pour en dresser le plan. **Atlas SQL** est cet urbaniste. Il inventorie un ou plusieurs serveurs, lit **tout** le code T-SQL, reconstruit qui lit et qui écrit quoi, documente les données (RGPD compris), mesure la dette technique, et vous donne des outils pour comprendre, déboguer et transformer le patrimoine en toute sécurité.

Atlas SQL est un programme **Lexis+** (environ 10 000 lignes, livré dans `exemples/atlassql`). Il s'exécute avec l'atelier **wdlexis+** ou le runtime **wdlexisrt**. Toutes ses informations sont rangées **sur votre poste**, dans une base **DuckDB** par projet. Les traitements sont placés dans une **file d'attente** et exécutés en parallèle par **3 à 10 agents**, pour aller vite sans surcharger les bases analysées.

Comment lire ce guide

- Les **onglets de couleur** sur le bord droit des pages indiquent le chapitre ; leur icône reprend celle du chapitre.
- Chaque notion est illustrée par un **exemple réel** : un code T-SQL, une requête ou un programme Lexis+, suivi de ce que produit Atlas **pour de vrai**. Tous les résultats ont été obtenus en exécutant Atlas sur la démonstration fournie (3 serveurs simulés).
- Les **captures d'écran** sont celles de l'application, en thème sombre ; les pastilles jaunes numérotées renvoient au texte.
- Les noms de boutons et d'onglets sont écrits comme à l'écran : ▶ Analyser, onglet **Dépendances**, menu *Sources › Nouvelle source…*
- Les annexes rassemblent les références : tous les constats, le modèle de données, les raccourcis, un glossaire et un guide de dépannage.

Les encadrés

À retenir
L'essentiel d'une notion, à garder en tête.

Attention
Un piège fréquent, une limite à connaître.

Pour bien comprendre
Une explication en coulisses : comment Atlas calcule ce qu'il affiche.

À vous de jouer
Un petit exercice à faire sur la démonstration.

Astuce
Un raccourci, une manière plus rapide ou plus sûre de faire.

Sécurité
Ce qu'Atlas fait (et ne fait jamais) sur vos serveurs.

Pour démarrer tout de suite
Lancez `wdlexis+ exemples/atlassql/principal.prg`, choisissez *Sources › Ajouter la démonstration*, puis ▶ Analyser (F5). En deux secondes, vous avez trois serveurs cartographiés : c'est le terrain de jeu de tout ce guide (chapitre 1).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Les dix-huit modules d'Atlas SQL

Atlas SQL réalise les dix-huit modules du cahier des charges. Chaque carte indique le chapitre qui l'explique en détail. Les modules s'appuient tous sur le même socle : la **file d'attente** (chapitre 4) qui alimente l'**entrepôt DuckDB** (chapitre 5).

Inventaire et découverte
Serveurs, bases, schémas, tables, vues, procédures, fonctions, déclencheurs, jobs, synonymes, serveurs liés.
chapitre 3

Cartographie globale
Cinq niveaux : global, serveur, base, schéma, objet ; zoom, recentrage, historique, favoris.
chapitre 9

Graphe des dépendances
Qui lit, écrit, supprime, met à jour, appelle ; fréquence, volumes, complexité.
chapitre 8

Dictionnaire intelligent
Objet métier, famille, nature, sensibilité, RGPD, descriptions générées.
chapitre 10

Types métier
Colonnes équivalentes (CODE_CLIENT, CLI_CODE, CODCLI…) et types incohérents.
chapitre 11

Analyseur SQL et AST
Jetons, instructions, arbre, phases, références, variables, complexité.
chapitre 6

Duplications
Code identique ou voisin, jointures et règles répétées, mutualisation.
chapitre 15

SQL dynamique
Reconstitution du SQL construit, variantes, valeurs observées.
chapitre 13

Éditeur SQL avancé
Coloration, complétion, diagnostics soulignés, extraits, exécution en lecture.
chapitre 17

Performances
Statistiques, plans en cache, index manquants, script de corrections.
chapitre 14

Synoptique logique
Le code découpé en phases et synchronisé avec l'éditeur.
chapitre 17

Flux de données
Flux, Sankey, pipeline et vue temporelle des jobs.
chapitre 12

Débogueur
Pas à pas symbolique, points d'arrêt, pile d'appels, variables.
chapitre 17

Interception des appels
Capture XE, import, simulation, rejeu transactionnel, export.
chapitre 19

Refactoring assisté
Renommer, extraire, découper, fusionner, avec impact et aperçu.
chapitre 18

Analyse patrimoniale
Dette, couplage, cohésion, maintenabilité, classes de risque.
chapitre 16

Interface multi-fenêtres
Onglets, panneau détachable, espaces de travail, thèmes.
chapitre 2

Assistant IA
Questions en français ; règles intégrées ou modèle local / distant.
chapitre 20

Un parcours conseillé

Vous voulez…	Lisez
découvrir l'outil en vingt minutes	chapitres 1 et 2, puis 9 (cartographie)
brancher vos serveurs SQL Server	chapitres 3 et 4
savoir « qui touche à cette table ? »	chapitres 8 et 9
documenter et cartographier les données personnelles	chapitres 10 et 11
préparer une migration ou une refonte	chapitres 8, 15, 16 et 18
comprendre une procédure ou analyser une lenteur	chapitres 6, 13, 14, 17 et 19
exploiter les résultats dans vos propres outils	chapitre 5 et annexe B

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Sommaire

1. Démarrer 6

1.1 Atlas SQL, qu'est-ce que c'est ? 6
1.2 Installer et lancer 7
1.3 Un projet = un dossier 7
1.4 Votre première analyse : la démonstration 8
1.5 Ce que contient la démonstration 9
1.6 Lire le tableau de bord 10

2. L'interface 11

2.1 Vue d'ensemble 11
2.2 La barre d'outils 12
2.3 Le panneau de gauche 13
2.4 Masquer, redimensionner ou détacher le panneau 14
2.5 Les vues centrales 15
2.6 Le panneau du bas et la barre d'état 16
2.7 L'objet courant, l'historique et la recherche 16
2.8 Les menus 16
2.9 Réglages, thème et espaces de travail 17

3. Les sources 18

3.1 Quatre genres de source 18
3.2 La fiche « Nouvelle source » 18
3.3 L'authentification 19
3.4 Chiffrement et certificat (SQL Server 2022 et 2025) 20
3.5 Les mots de passe 20
3.6 Tester une connexion 21
3.7 Les droits nécessaires sur SQL Server 21
3.8 Ce qu'Atlas lit sur le serveur 22

4. La file d'attente 23

4.1 D'une campagne à des travaux 23
4.2 Les genres de travaux 24
4.3 Les états d'un travail 24
4.4 Les agents : de 3 à 10 25
4.5 Suivre et piloter la file 25

5. L'entrepôt DuckDB 27

5.1 Une base DuckDB par projet 27
5.2 Interroger l'entrepôt en SQL 28
5.3 Exploiter l'entrepôt avec Lexis+ 29
5.4 Exporter, sauvegarder, partager 31

6. L'analyse du code 32

6.1 La chaîne d'analyse 32
6.2 Les jetons 33
6.3 Des jetons aux instructions, puis à l'arbre 33
6.4 Les phases : le synoptique du code 34
6.5 Les références 35
6.6 Variables, tables temporaires, curseurs et CTE 36
6.7 La complexité cyclomatique 37
6.8 Jointures et règles de gestion 37
6.9 Ce que l'analyseur ne fait pas 38

7. Les constats 39

7.1 Trois niveaux de gravité 39
7.2 Un module qui accumule les défauts 39
7.3 Les règles de l'analyseur 40
7.4 Les constats des autres moteurs 42
7.5 L'onglet Constats 43

8. Les dépendances 44

8.1 Un graphe orienté 44
8.2 Résoudre les noms 45
8.3 Fréquence, volumes et complexité 46
8.4 Les références introuvables 47
8.5 L'onglet Dépendances 47
8.6 L'analyse d'impact 48

9. La cartographie 50

9.1 Cinq niveaux de lecture 50
9.2 Lire une carte 52
9.3 Naviguer et régler 52

10. Le dictionnaire 54

10.1 Des noms techniques aux mots métier 54
10.2 Objet métier, famille et nature 55
10.3 Sensibilité et RGPD 55
10.4 Votre glossaire 56
10.5 L'onglet Dictionnaire et l'export 57

11. Les types métier 58

11.1 Une donnée, plusieurs noms 58
11.2 Le catalogue des types métier 59
11.3 Incohérences de types et de nommage 59

12. Les flux de données 61

12.1 Des dépendances aux flux 61
12.2 Quatre façons de voir les flux 62
12.3 La chronologie des jobs 63

13. Le SQL dynamique 64

13.1 Pourquoi le SQL dynamique est difficile 64
13.2 La reconstitution 65
13.3 EXEC (@sql) et l'injection 65
13.4 OPENQUERY : le SQL envoyé à un autre serveur 66
13.5 Variantes et valeurs observées 66

14. Les performances 68

14.1 D'où viennent les mesures 68
14.2 Les mesures par objet 69
14.3 Les règles de performance 69
14.4 Des index manquants au script 70
14.5 L'onglet Perfs 71

15. Les doublons 72

15.1 Quatre genres de doublons 72
15.2 La similarité de Jaccard, pas à pas 72
15.3 Indices et onglet Doublons 73

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

16. Le patrimoine 75
16.1 Les métriques d'un objet 75
16.2 Les formules, sur un exemple 76
16.3 Les classes 76
16.4 L'onglet Patrimoine 77

17. L'éditeur et le débogueur 78
17.1 L'éditeur T-SQL 78
17.2 Le synoptique, l'arbre et l'analyse en direct 79
17.3 Exécuter une requête en lecture 80
17.4 Le débogueur symbolique 80

18. Le refactoring 82
18.1 Le principe : l'impact d'abord 82
18.2 Renommer un objet ou une colonne 83
18.3 Extraire, découper, fusionner 83

19. L'interception 85
19.1 Voir les appels sans toucher aux applications 85
19.2 Installer la capture 85
19.3 Importer et lire les appels 86
19.4 Simuler, rejouer, exporter 87

20. L'assistant 88
20.1 Poser une question en français 88
20.2 Brancher un modèle de langage 90

21. Aller plus loin 92
21.1 L'architecture 92
21.2 Performances et volumétrie 93
21.3 Limites connues 93
21.4 De l'outil au service 93

Annexe A · Référence des constats 94
A.1 Les vingt-sept règles 94

Annexe B · Modèle de données 96
B.1 Les tables et leurs colonnes 96

Annexe C · Aide-mémoire 98
C.1 Raccourcis clavier 98
C.2 Gestes de souris 98
C.3 Où est… ? 99

Annexe D · Glossaire 100
D.1 Glossaire 100

Annexe E · Dépannage 102
E.1 Problèmes et solutions 102

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 1 · DÉMARRER

Vos premiers pas, en trois clics

Ce qu'est Atlas SQL, comment le lancer, comment un projet est organisé, et votre première cartographie complète avec la démonstration fournie.

1.1 Atlas SQL, qu'est-ce que c'est ?
1.2 Installer et lancer
1.3 Un projet = un dossier
1.4 Votre première analyse : la démonstration
1.5 Ce que contient la démonstration
1.6 Lire le tableau de bord

Dans ce chapitre, vous apprendrez à…

- comprendre ce que fait Atlas SQL et ce qu'il ne fait pas
- lancer Atlas avec wdlexis+ ou le runtime
- savoir où sont rangées les informations d'un projet
- analyser la démonstration en une campagne
- connaître les défauts volontaires de la démonstration
- lire le tableau de bord

1.1 Atlas SQL, qu'est-ce que c'est ?

Atlas SQL répond aux questions que l'on se pose devant un patrimoine SQL Server que l'on n'a pas écrit : « *Qui alimente cette table ?* », « *Que casse-t-on si l'on renomme cette colonne ?* », « *Où sont les données personnelles ?* », « *Pourquoi cette procédure est-elle lente ?* », « *Quelles procédures sont mortes ?* ».

Pour y répondre, il **extrait** le catalogue de vos serveurs (objets, colonnes, code, statistiques, jobs), le **range** dans une base DuckDB locale, puis **analyse** tout le code T-SQL avec ses propres moteurs. Les vues de l'interface ne font ensuite que lire cette base : elles sont instantanées et ne sollicitent plus jamais les serveurs.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Atlas ne modifie jamais vos bases

Les sources SQL Server sont attachées en **lecture seule** (`READ_ONLY`). Atlas lit le catalogue ({c('sys.*')}), le code des modules, les vues de gestion dynamiques et {c('msdb')}. Les scripts qu'il propose (index, renommages, refactoring) sont **générés** pour que vous les relisiez et les exécutiez vous-même. Seule exception, opt-in et confirmée : le **rejeu** d'un appel capturé, exécuté dans une transaction annulée (chapitre 19).

1.2 Installer et lancer

Atlas SQL est un exemple livré avec Lexis+ : il n'y a rien à installer en dehors de wdlexis+. Le dossier `exemples/atlassql` contient quinze fichiers `.prg`, l'icône et deux documents (`LISEZMOI.md`, `ARCHITECTURE.md`).

COMMANDE **Lancer Atlas SQL** terminal

```
# depuis le dossier de nim.lexis
wdlexis+ exemples/atlassql/principal.prg                # projet par défaut : ~/AtlasSQL
wdlexis+ exemples/atlassql/principal.prg ~/Audit-ERP    # un projet par dossier
wdlexisrt exemples/atlassql/principal.prg ~/Audit-ERP   # runtime seul, sans l'atelier
```

Il faut...	Pour...	Remarque
wdlexis+ ou wdlexisrt	exécuter le programme	compilés avec `-d:sdlttf` (polices) ; DuckDB est intégré
l'extension DuckDB `mssql`	lire SQL Server en direct	téléchargée automatiquement à la première connexion (accès Internet)
l'extension `nanodbc` et un pilote ODBC	le genre « SQL Server par ODBC »	ODBC Driver 18 for SQL Server conseillé
rien de plus	la démonstration et les dossiers de scripts	aucun réseau nécessaire

Astuce

La fenêtre s'ouvre agrandie. Toutes les tailles (panneaux, séparateurs) se règlent à la souris et sont retrouvées au lancement suivant.

1.3 Un projet = un dossier

Un **projet** est un dossier. Atlas y crée la base `atlas.duckdb` qui contient **tout** : sources déclarées, catalogue extrait, résultats d'analyse, historique de la file d'attente, journal, favoris et espaces de travail. Changer de projet (*Fichier › Ouvrir un projet...*), c'est simplement ouvrir une autre base.

Fichier	Emplacement	Contenu
`atlas.duckdb`	dossier du projet	l'entrepôt complet (33 tables, chapitre 5)
`glossaire.json`	dossier du projet (facultatif)	votre vocabulaire métier et vos abréviations (chapitre 10)

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Fichier	Emplacement	Contenu
`~/.atlassql.json`	dossier personnel	réglages de l'interface : agents, thème, tailles des panneaux…
`~/.atlassql-secrets.json`	dossier personnel	mots de passe mémorisés, **chiffrés** avec une clé propre au poste

> **Une base, un seul programme à la fois**
>
> DuckDB verrouille la base pendant qu'Atlas l'utilise. Si vous lancez Atlas deux fois sur le même projet, le second affiche « *base du projet verrouillée* ». Pour interroger la base avec un autre outil (chapitre 5), fermez Atlas ou travaillez sur une copie.

1.4 Votre première analyse : la démonstration

Le plus simple pour découvrir Atlas est sa **démonstration** : trois serveurs SQL Server simulés, avec leur catalogue, leur code, leurs statistiques et leurs jobs. Elle passe par exactement les mêmes traitements qu'un vrai serveur.

1. Lancez Atlas sur un dossier vide : l'accueil est vide et l'explorateur indique « *Aucune source : + Source, puis ▶ Analyser* ».
2. Choisissez *Sources › Ajouter la démonstration*. (Si vous cliquez `▶ Analyser` sans source, Atlas vous la propose.)
3. Cliquez `▶ Analyser` ou appuyez sur `F5`. La campagne de 37 travaux part dans la file d'attente, en bas.
4. Les agents travaillent en parallèle ; la barre d'état annonce « *Campagne terminée en 1.8 s. Vues mises à jour.* »

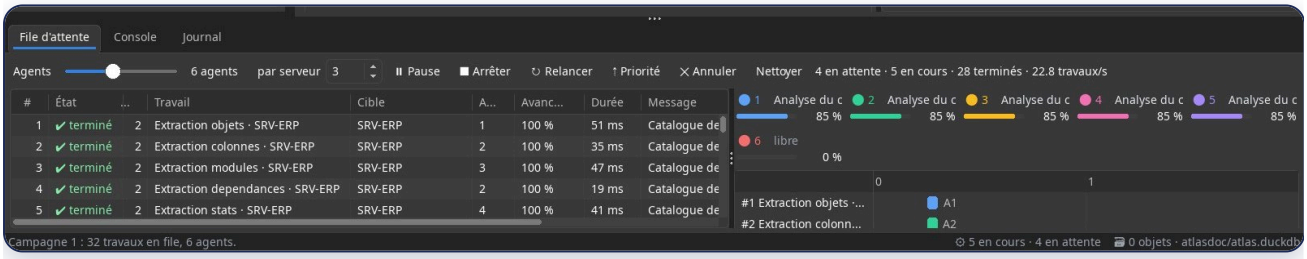


Figure 1 — La file d'attente pendant la campagne : cinq agents analysent le code en même temps (bandeau à droite), la grille liste les travaux et leur état.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

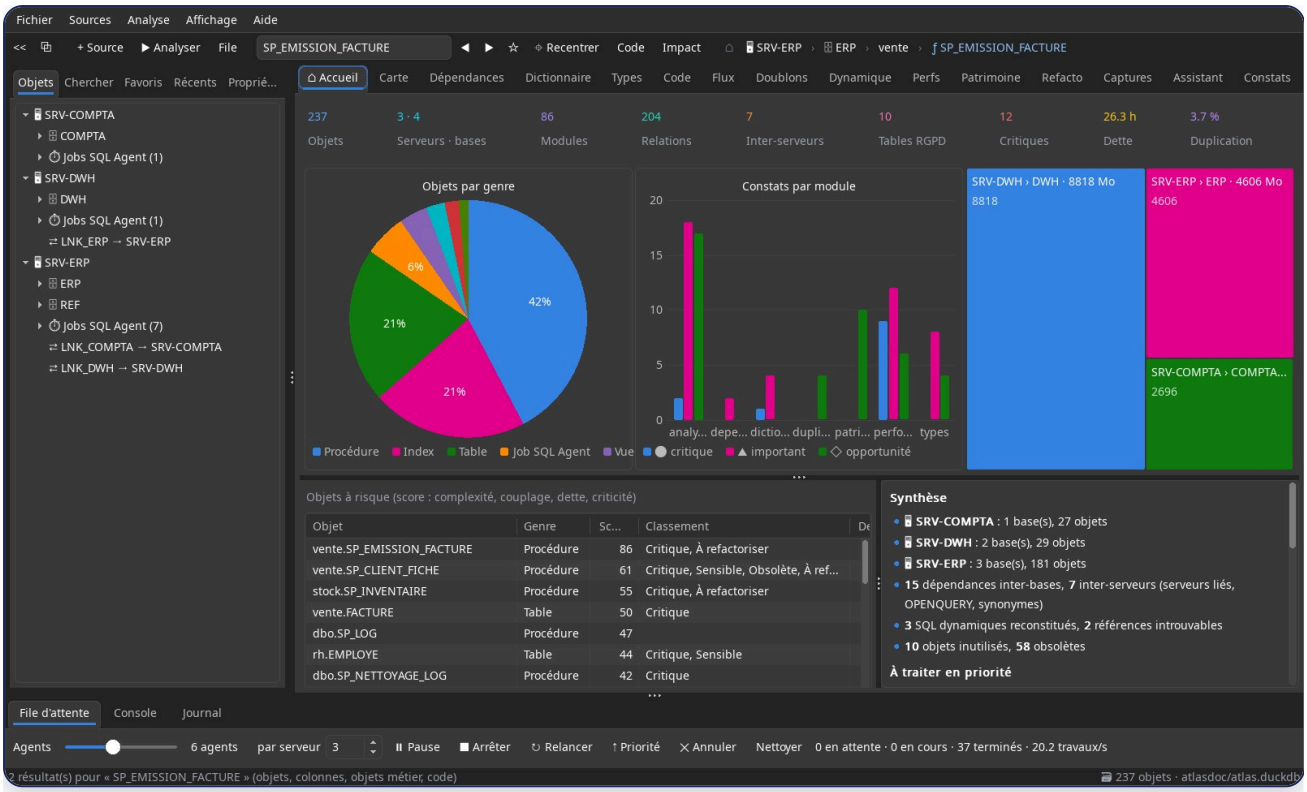


Figure 2 — Après la campagne : l'accueil résume le patrimoine (237 objets, 204 relations, 7 liens inter-serveurs, 26,3 h de dette).

1.5 Ce que contient la démonstration

La démonstration reproduit un petit système d'information d'entreprise. Ses défauts sont **volontaires** : ils servent d'exemples dans tout le guide.

Serveur	Bases	Contenu
SRV-ERP	ERP (schémas `vente`, `stock`, `rh`, `dbo`), REF	clients, commandes, factures, stock, salariés ; référentiels pays, TVA, devises ; 60 procédures, 7 jobs, 2 serveurs liés
SRV-COMPTA	COMPTA	écritures, comptes, tiers ; balance ; serveur lié vers l'ERP
SRV-DWH	DWH (schémas `stg`, `dw`)	entrepôt décisionnel : dimensions, faits, chargements nocturnes

Les défauts que vous allez découvrir

- `vente.SP_RAPPORT_DYNAMIQUE` construit son SQL par concaténation : risque d'**injection** (chapitre 13).
- `vente.SP_SYNCHRO_TIERS` fait un **UPDATE sans WHERE** et lit la comptabilité par `OPENQUERY`.
- `rh.SP_PAIE_MENSUELLE` écrit dans `COMPTA.dbo.ECRITURE` … sur le serveur de l'ERP, où cette base n'existe pas : **référence introuvable** (chapitre 8).
- `vente.SP_EMISSION_FACTURE` (64 lignes, curseur, `NOLOCK`, `YEAR()` sur une colonne filtrée) est l'objet le plus risqué du patrimoine (chapitre 16).
- Les colonnes `CODE_CLIENT`, `CLI_CODE`, `CODCLI` … désignent la même donnée avec des types différents (chapitre 11).
- Des tables `CLIENT_OLD`, `TMP_IMPORT_CLIENT` et des procédures CRUD jamais appelées : **obsolètes ou inutilisées**.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

À vous de jouer

Avant de lire la suite, cliquez sur la ligne **vente.SP_EMISSION_FACTURE** du tableau « Objets à risque » de l'accueil (double-clic) : la fiche de la procédure s'affiche dans la page **Propriétés**, à gauche. Repérez sa complexité, sa dette et son classement.

1.6 Lire le tableau de bord

L'onglet ⌂ **Accueil** est la synthèse du patrimoine. Il est recalculé à la fin de chaque campagne.

Indicateur	Signification
Objets	tables, vues, procédures, fonctions, déclencheurs, jobs, synonymes, index, contraintes… inventoriés
Serveurs · bases	nombre de serveurs et de bases analysés
Modules	objets qui portent du code T-SQL (procédures, fonctions, vues, déclencheurs, jobs)
Relations	arêtes du graphe des dépendances (chapitre 8)
Inter-serveurs	dépendances qui traversent un serveur lié, un synonyme ou un `OPENQUERY`
Tables RGPD	tables qui contiennent des données personnelles (chapitre 10)
Critiques	constats de niveau critique (chapitre 7)
Dette	dette technique estimée, en heures (chapitre 16)
Duplication	part des instructions dupliquées ou quasi identiques (chapitre 15)

Dessous : la répartition des objets par genre, les constats par module et par niveau, la volumétrie des bases (mosaïque), les **objets à risque** triés par score (double-clic pour ouvrir) et la **synthèse**, qui liste les points à traiter en priorité.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 2 · L'INTERFACE

Un atelier pensé pour lire un patrimoine

La barre d'outils, le fil d'Ariane, le panneau de navigation détachable, les quinze vues, la file d'attente et les réglages : tour complet de l'écran.



2.1 Vue d'ensemble
2.2 La barre d'outils
2.3 Le panneau de gauche
2.4 Masquer, redimensionner ou détacher le panneau
2.5 Les vues centrales
2.6 Le panneau du bas et la barre d'état
2.7 L'objet courant, l'historique et la recherche
2.8 Les menus
2.9 Réglages, thème et espaces de travail

Dans ce chapitre, vous apprendrez à...

- repérer chaque zone de l'écran
- naviguer avec l'objet courant, l'historique et les favoris
- utiliser les cinq pages du panneau de gauche
- masquer ou détacher le panneau sur un second écran
- chercher un objet, une colonne ou un texte de code
- retrouver vos réglages et espaces de travail

2.1 Vue d'ensemble

L'écran est organisé comme un atelier de développement : une barre d'outils unique, un panneau de navigation à gauche, les vues au centre et le suivi des traitements en bas. Tout est redimensionnable ; l'interface utilise la densité compacte de wdgui pour laisser le maximum de place aux données.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

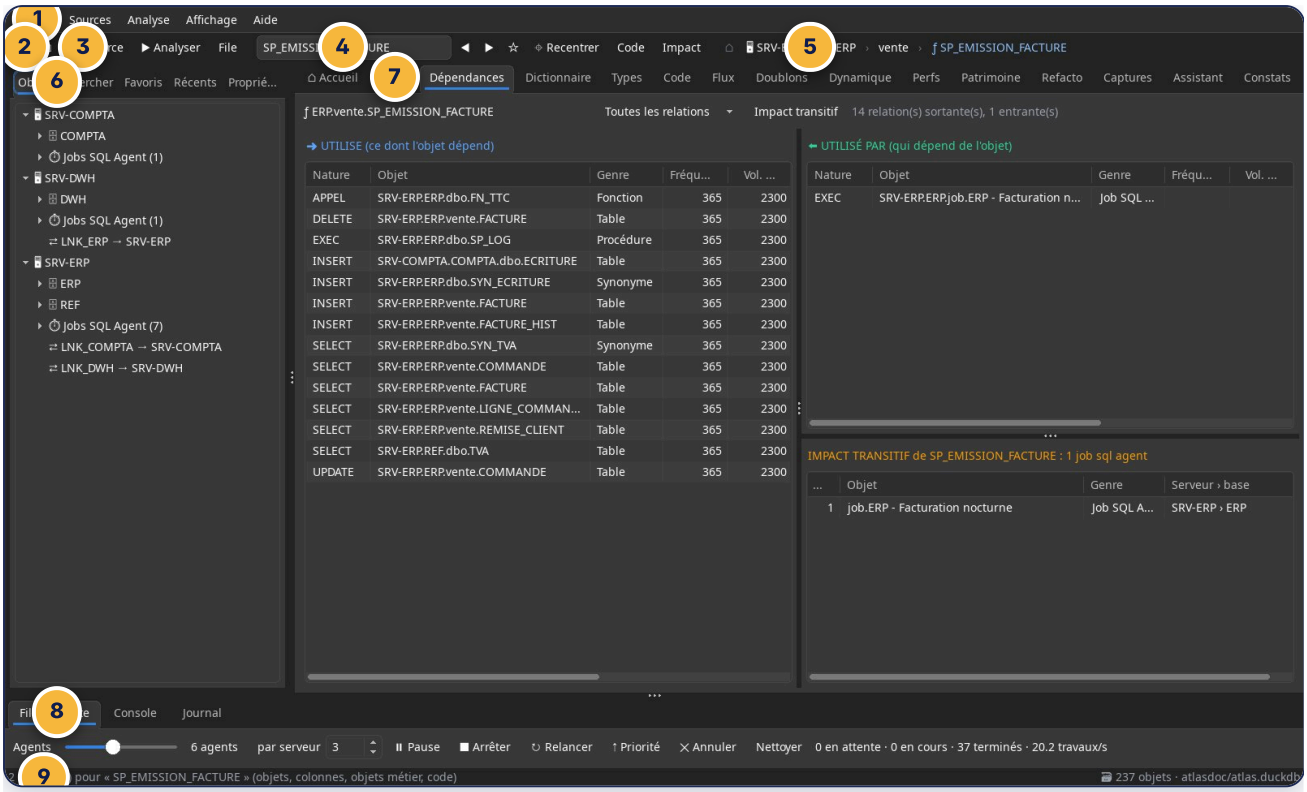


Figure 3 — L'écran principal. Les repères renvoient au tableau ci-dessous.

	Zone	Rôle
1	Menus	Fichier, Sources, Analyse, Affichage, Aide
2	`<<` / `>>`	masquer / réafficher le panneau de gauche
3	⧉	détacher le panneau de gauche dans sa propre fenêtre
4	Recherche	objets, colonnes, objets métier, texte du code (`Ctrl+F`)
5	Fil d'Ariane	serveur › base › schéma › objet courant ; chaque niveau ouvre sa cartographie
6	Panneau de gauche	Objets, Chercher, Favoris, Récents, Propriétés
7	Vues	quinze onglets d'analyse (section 2.6)
8	Panneau du bas	File d'attente, Console, Journal
9	Barre d'état	dernier message, état de la file, taille et emplacement de la base

2.2 La barre d'outils

Une seule ligne de boutons plats (le relief apparaît au survol), regroupés par usage :

Bouton	Action	Raccourci
`<<` ⧉	masquer ou détacher le panneau de gauche	
`+ Source`	déclarer un serveur, une connexion ODBC ou un dossier de scripts (chapitre 3)	
`▶ Analyser`	lancer la campagne complète sur toutes les sources actives (chapitre 4)	`F5`
`File`	afficher la file d'attente	
Rechercher...	recherche globale ; `Entrée` ouvre le premier résultat	`Ctrl+F`

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Bouton	Action	Raccourci
◀ ▶	objet précédent / suivant de l'historique	Alt+← Alt+→
☆	ajouter ou retirer l'objet courant des favoris	Ctrl+D
⌖ Recentrer	recentrer la cartographie sur l'objet courant	
Code	ouvrir le code de l'objet courant dans l'éditeur	
Impact	analyse d'impact de l'objet courant (chapitre 8)	

À droite, le **fil d'Ariane** rappelle où vous êtes ; en jaune à l'extrême droite, l'état de la file (*« ⚙ 4 en cours · 6 en attente »*).

2.3 Le panneau de gauche

Le panneau de gauche regroupe cinq pages :

Page	Contenu
Objets	l'explorateur : 🖥 serveurs › 🗄 bases › schémas › genres (Tables, Vues, Procédures…) › objets. Les jobs SQL Agent et les serveurs liés sont rangés sous leur serveur. Les dossiers se chargent à l'ouverture.
Chercher	les résultats de la recherche : objets ▦ ƒ, colonnes ▤, objets métier ◎, citations dans le code ⌕
Favoris	les objets marqués d'une étoile ☆
Récents	l'historique de navigation
Propriétés	la fiche de l'objet courant : genre, dates, taille, code, complexité, dette, classement, exécutions, dépendances, paramètres, constats ; ses liens ouvrent le code, les dépendances, l'impact et la carte

Dans l'explorateur, des marques signalent les objets à surveiller : ◆ critique, 🔒 données personnelles, ⌀ inutilisé ou obsolète. Une source déclarée mais pas encore analysée apparaît comme *« 🖹 Production ERP (sqlprod01,1433) — à analyser (F5) »* : un clic affiche sa fiche, un double-clic l'ouvre en modification.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

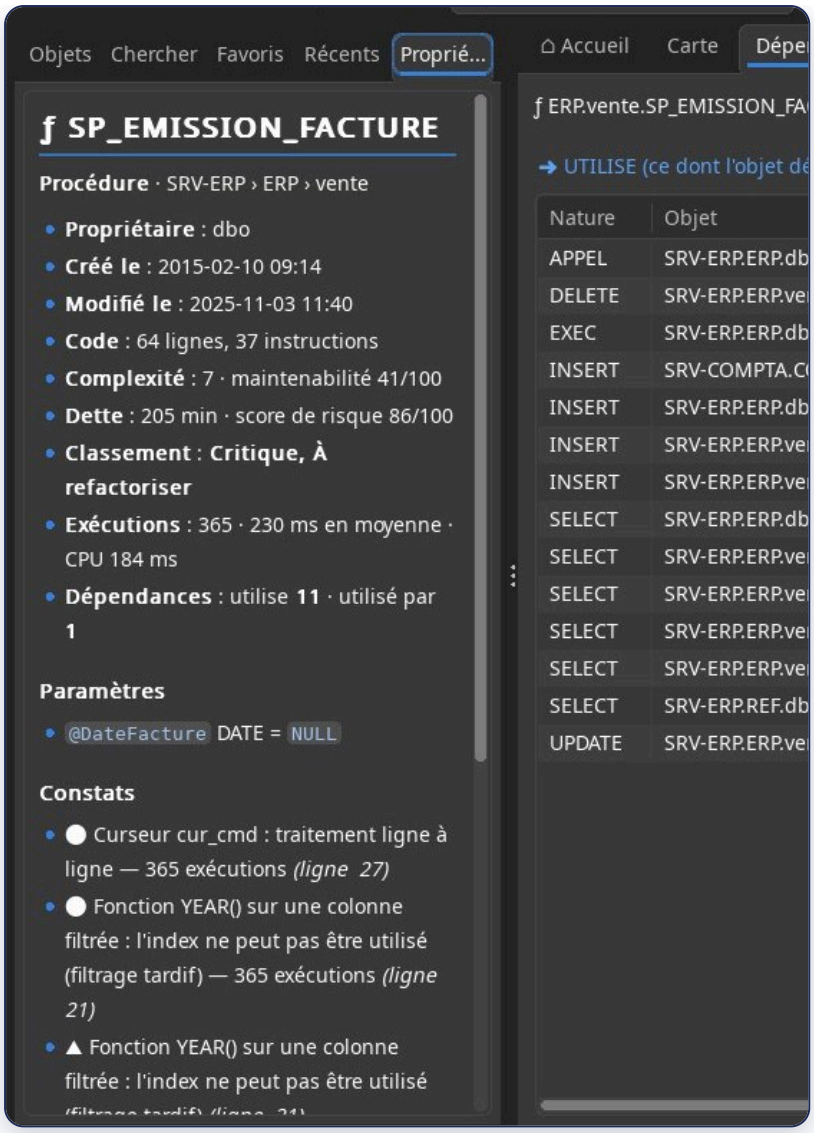


Figure 4 — La page Propriétés de vente.SP_EMISSION_FACTURE : la fiche reprend toutes les mesures d'Atlas et se termine par des liens.

> **Astuce**
> Double-clic dans l'explorateur : sur un objet, ouvre son **code** ; sur un serveur, une base ou un schéma, ouvre sa **cartographie**.

2.4 Masquer, redimensionner ou détacher le panneau

Trois façons de régler la place prise par la navigation :

- **Redimensionner** : faites glisser le séparateur vertical (les trois points) entre le panneau et les vues.
- **Masquer** : `<<` libère toute la largeur pour les vues ; le bouton devient `>>`, qui réaffiche le panneau **à la même largeur**.
- **Détacher** : ⧉ ouvre les cinq pages dans une fenêtre séparée (pratique sur un second écran) ; l'espace de gauche est libéré. Fermer cette fenêtre, ou cliquer ⧈ ou `>>`, remet les pages à gauche.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

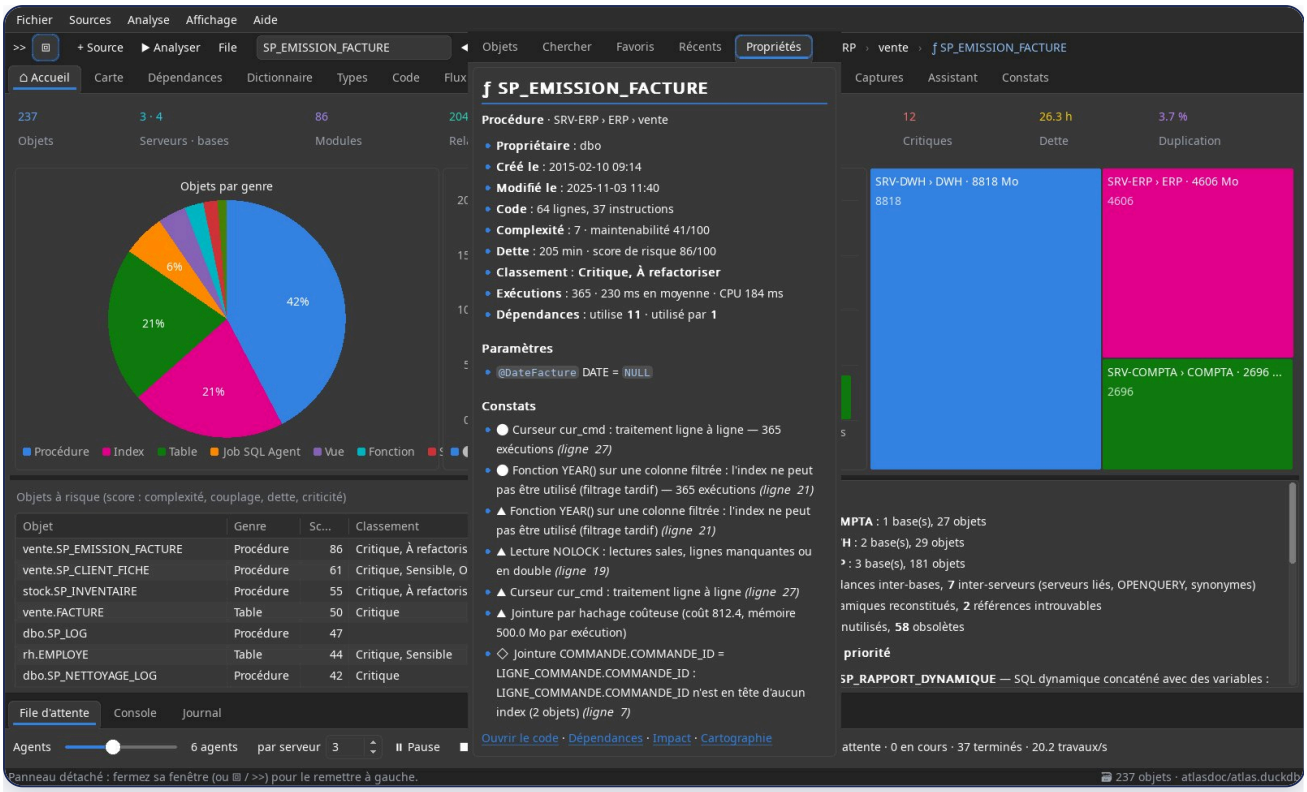


Figure 5 — Le panneau détaché dans sa propre fenêtre (ici sur la page Propriétés), au-dessus de la fenêtre principale qui occupe toute la largeur.

L'état masqué et la largeur du panneau sont retrouvés au lancement suivant.

2.5 Les vues centrales

Quinze onglets, chacun expliqué dans un chapitre. Les vues lourdes ne sont recalculées qu'à l'affichage de leur onglet et suivent l'objet courant.

Onglet	Ce qu'il montre	Chapitre
⌂ Accueil	indicateurs, graphiques, objets à risque, synthèse	1
Carte	cartographie à cinq niveaux	9
Dépendances	ce que l'objet utilise, qui l'utilise, impact transitif	8
Dictionnaire	objets métier, familles, sensibilité, RGPD, colonnes	10
Types	types métier et incohérences de types SQL	11
Code	éditeur T-SQL, synoptique, arbre, analyse, débogueur	17
Flux	flux de données, Sankey, pipeline, chronologie des jobs	12
Doublons	code dupliqué, jointures et règles répétées	15
Dynamique	SQL dynamique reconstitué et valeurs observées	13
Perfs	statistiques, opérateurs coûteux, index manquants	14
Patrimoine	dette, couplage, cohésion, maintenabilité, classes	16
Refacto	renommer, extraire, découper, fusionner	18
Captures	appels interceptés : simuler, rejouer, exporter	19
Assistant	questions en français	20
Constats	tous les diagnostics, filtrables	7

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

2.6 Le panneau du bas et la barre d'état

Onglet	Contenu
File d'attente	réglage des agents, commandes de la file, liste des travaux, cartes des agents, chronologie (chapitre 4)
Console	trace détaillée des traitements (requêtes envoyées, durées)
Journal	événements datés : campagnes, erreurs, connexions ; les erreurs apparaissent en rouge

La **barre d'état**, collée au bas de la fenêtre, affiche le dernier message (fin de campagne, nombre de résultats d'une recherche…), l'état de la file et la base du projet (*« 237 objets · atlasdoc/atlas.duckdb »*).

2.7 L'objet courant, l'historique et la recherche

Tout l'écran est synchronisé sur un **objet courant**. Le choisir — dans l'explorateur, la carte, une grille, la recherche ou un lien de fiche — met à jour le fil d'Ariane, la page Propriétés, l'historique et la vue affichée (sa carte, ses dépendances, son code…).

- **Historique** : ◀ ▶ (ou Alt+← Alt+→) parcourent les objets visités ; la page **Récents** les liste.
- **Favoris** : ☆ ou Ctrl+D ; la page **Favoris** les garde d'une session à l'autre (table `favoris`).
- **Recherche** : dès deux caractères, Atlas cherche dans les noms d'objets, les noms de colonnes, les objets métier du dictionnaire et le texte du code. Les résultats sont classés dans cet ordre ; Entrée ouvre le premier.

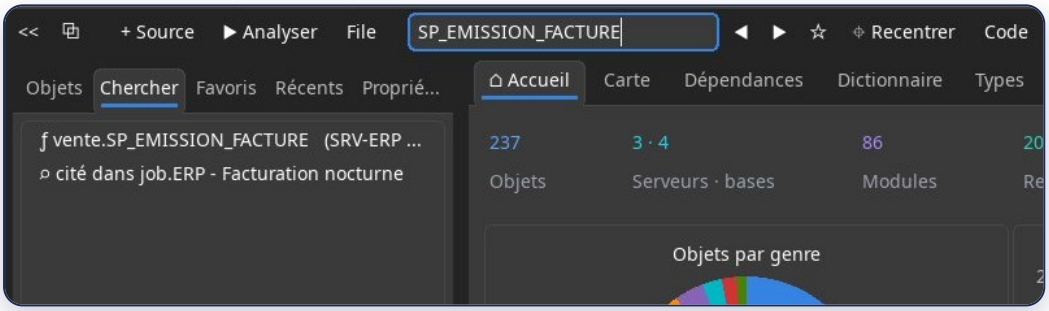


Figure 6 — Recherche de « SP_EMISSION_FACTURE » : la procédure elle-même, puis le job qui la cite dans son code (⌕).

2.8 Les menus

Menu	Entrées
Fichier	Nouveau projet…, Ouvrir un projet…, Exporter le dictionnaire (Markdown)…, Exporter l'inventaire (CSV)…, Exporter les dépendances (CSV)…, Quitter
Sources	Nouvelle source…, Modifier la source…, Ajouter la démonstration, Supprimer une source…
Analyse	Lancer l'analyse complète (F5), Relancer les travaux en erreur, Pause / reprise de la file, Tout arrêter, Importer les appels capturés
Affichage	les quinze vues, Masquer / afficher le panneau de gauche, Panneau de gauche dans une fenêtre, Cartographie dans une fenêtre, Thème sombre / clair, Enregistrer / Restaurer un espace de travail
Aide	Guide de démarrage, Raccourcis clavier, Architecture, À propos

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

2.9 Réglages, thème et espaces de travail

Les réglages de l'interface sont enregistrés automatiquement dans `~/.atlassql.json` :

FICHIER **Le fichier de réglages après une session** `~/.atlassql.json`

```
{"SOMBRE":true,"AGENTS":6,"PARSERVEUR":3,"PROJET":"","LOT":20,"PROFILAGE":false,
 "IA_MOTEUR":"regles","IA_MODELE":"","IA_URL":"","IA_NOM":"","IA_CLE_ENV":"",
 "SEP_GAUCHE":0.22,"PANNEAU_MASQUE":false,"SEP_BAS":0.7,"POLICE":13,"VUE":1,"ESPACE":"Standard"}
```

Clé	Rôle	Défaut
`AGENTS`	nombre d'agents parallèles (3 à 10)	4
`PARSERVEUR`	travaux simultanés au plus sur un même serveur	3
`LOT`	modules analysés par travail d'analyse	20
`SOMBRE`	thème sombre ou clair	vrai
`SEP_GAUCHE`, `SEP_BAS`	position des séparateurs (0 à 1)	0,22 · 0,70
`PANNEAU_MASQUE`	panneau de gauche masqué au lancement	faux
`IA_*`	moteur de l'assistant (chapitre 20)	règles

Un **espace de travail** (*Affichage › Enregistrer l'espace de travail…*) mémorise, sous un nom, la vue affichée, l'objet courant, les pages des panneaux et le réglage des agents. *Restaurer un espace de travail…* vous y ramène. Les espaces sont rangés dans la base du projet.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 3 · LES SOURCES

Brancher vos serveurs SQL Server

Déclarer un serveur, une connexion ODBC ou un dossier de scripts ; choisir l'authentification et le chiffrement ; tester ; donner à Atlas les seuls droits dont il a besoin.



3.1 Quatre genres de source
3.2 La fiche « Nouvelle source »
3.3 L'authentification
3.4 Chiffrement et certificat (SQL Server 2022 et 2025)
3.5 Les mots de passe
3.6 Tester une connexion
3.7 Les droits nécessaires sur SQL Server
3.8 Ce qu'Atlas lit sur le serveur

Dans ce chapitre, vous apprendrez à…

- ✓ choisir le bon genre de source
- ✓ remplir la fiche d'une source
- ✓ régler le chiffrement pour SQL Server 2022 et 2025
- ✓ comprendre où sont gardés les mots de passe
- ✓ interpréter le résultat de Tester
- ✓ créer un compte de lecture dédié

3.1 Quatre genres de source

Une **source** est un serveur (ou un ensemble de scripts) qu'Atlas inventorie. Un projet peut en compter autant que nécessaire : leurs dépendances croisées (serveurs liés, synonymes, OPENQUERY) sont justement ce qu'Atlas sait reconstituer.

Genre	Accès	Quand l'utiliser
SQL Server (extension mssql)	extension DuckDB `mssql` : `ATTACH '…' (TYPE mssql, READ_ONLY)`	le cas général : lecture directe du serveur, sans pilote à installer
SQL Server par ODBC	extension `nanodbc` : `odbc_query(connection=…, query=…)`	si un pilote ODBC est imposé, ou pour le chiffrement strict
Dossier de scripts .sql	lecture des fichiers `CREATE …` (lots séparés par `GO`, `USE` respecté)	un dépôt Git, un export SSMS, un serveur inaccessible
Démonstration	catalogue simulé	découvrir et se former

3.2 La fiche « Nouvelle source »

Sources › Nouvelle source… ou `+ Source` ouvre la fiche ci-dessous.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

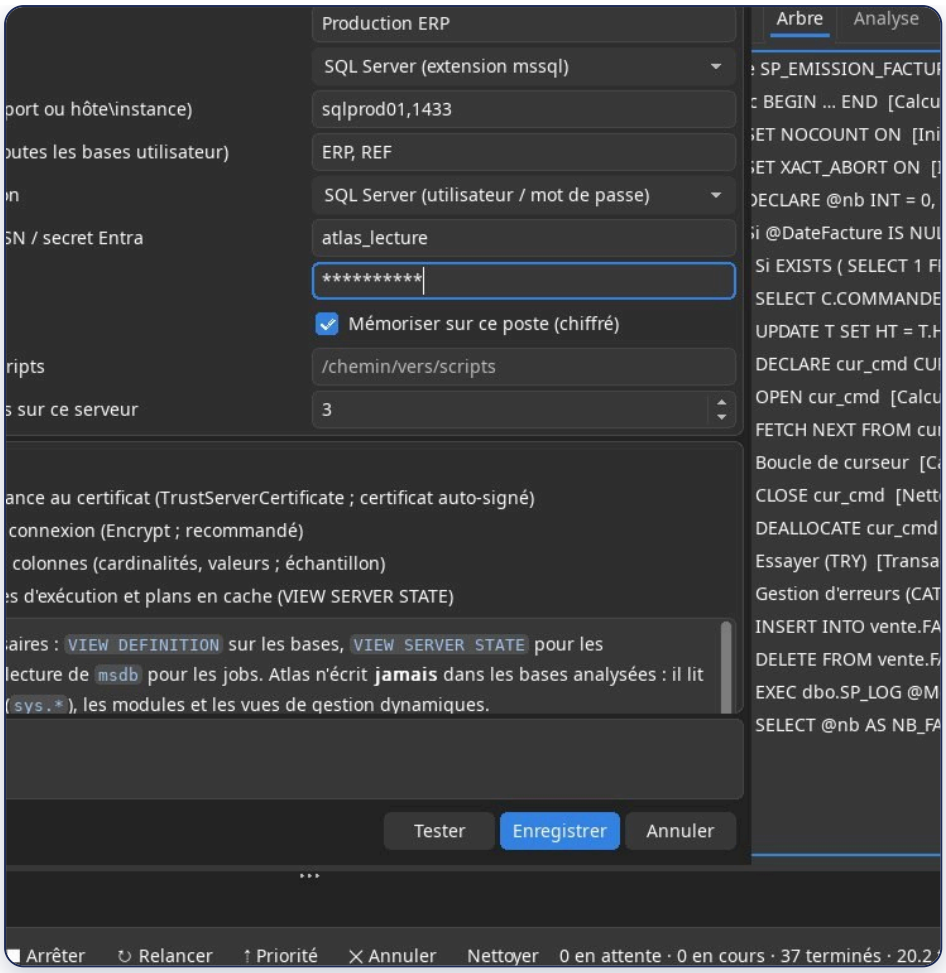


Figure 7 — La fiche d'une source SQL Server : identité, serveur, bases, authentification, mot de passe, limite d'agents et options.

Champ	Rôle	Exemple
Nom	libellé de la source dans Atlas	Production ERP
Genre	les quatre genres ci-dessus	SQL Server (extension mssql)
Serveur	`hôte`, `hôte,port` ou `hôte\instance`	`sqlprod01,1433`
Bases	liste séparée par des virgules ; vide = toutes les bases utilisateur	`ERP, REF`
Authentification	SQL Server, Windows, Entra ID, chaîne ODBC	
Utilisateur / DSN / secret Entra	selon l'authentification	`atlas_lecture`
Mot de passe	champ masqué, actif pour l'authentification SQL Server	••••••
Mémoriser sur ce poste	chiffre le mot de passe dans `~/.atlassql-secrets.json`	coché
Dossier des scripts	pour le genre « dossier »	`/depots/erp/sql`
Agents au plus sur ce serveur	limite propre à cette source	3

3.3 L'authentification

Mode	Ce qu'Atlas envoie	À savoir
SQL Server	`User Id=…;Password=…`	le mot de passe est saisi dans la fiche ; l'authentification SQL doit être activée sur le serveur

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Mode	Ce qu'Atlas envoie	À savoir
Windows (SSPI / Kerberos)	`Integrated Security=SSPI`	le compte de la session du poste ; Kerberos sous Linux/macOS
Entra ID	option `azure_secret` de l'ATTACH	indiquez dans « Utilisateur » le nom d'un secret DuckDB de type azure
Chaîne ODBC	la chaîne telle quelle	mettez la chaîne complète (`DSN=…` ou `Driver={…};Server=…`) dans « Utilisateur / DSN »

3.4 Chiffrement et certificat (SQL Server 2022 et 2025)

Deux options règlent la couche TLS de la connexion :

- **Chiffrer la connexion** ({c('Encrypt=yes')}, cochée par défaut) : la connexion est chiffrée. Décochée : {c('Encrypt=no')}, aucun TLS.
- **Faire confiance au certificat** ({c('TrustServerCertificate=yes')}, cochée par défaut) : le certificat du serveur est accepté sans vérification de sa chaîne.

SQL Server 2022 et 2025 s'installent avec un **certificat auto-signé** ; l'extension `mssql` le refuse si l'on ne fait pas confiance au certificat. C'est la cause la plus fréquente d'un message « *TLS negotiation failed* ».

CHAÎNE **La chaîne de connexion construite par Atlas** `ATTACH … (TYPE mssql, READ_ONLY)`

```
Server=192.168.1.24,1433;Database=master;User Id=atlas_lecture;Password=••••••;
Encrypt=yes;TrustServerCertificate=yes;Application Name=Atlas SQL
```

Tester essaie les autres réglages pour vous

Si la négociation TLS échoue, Tester réessaie automatiquement avec « chiffrée, certificat accepté », puis « non chiffrée ». Le premier réglage qui fonctionne est **coché dans la fiche** et signalé ; il ne reste qu'à cliquer Enregistrer.

Chiffrement strict (TDS 8.0)

Un serveur qui impose le chiffrement strict (option *Force Strict Encryption* de SQL Server 2022/2025) peut refuser l'extension. Désactivez cette option dans le Gestionnaire de configuration SQL Server, ou utilisez le genre **SQL Server par ODBC** avec ODBC Driver 18.

3.5 Les mots de passe

- Le mot de passe saisi est gardé **en mémoire** pour la session.
- Si *Mémoriser sur ce poste* est coché, il est aussi **chiffré** dans {c('~/.atlassql-secrets.json')}, avec une clé dérivée du poste, du projet et de la source.
- Il n'est **jamais** écrit dans la base du projet : vous pouvez partager {c('atlas.duckdb')} sans risque.
- En modification, le champ indique « •••••• *mémorisé* » : le laisser vide conserve le mot de passe ; le remplir le remplace.
- Sans mot de passe mémorisé, Atlas le demande au lancement de l'analyse (boîte de saisie masquée).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

FICHIER **Le fichier des secrets : une entrée chiffrée par source** ~/.atlassql-secrets.json

```
{"S2":"lx1:k1RRzgUUtvAb2+onqbBJCKRo5aMqfFLc+fD2If6DtT2nMik+2zV3hQJXXET0vffVE1108v3s9PRtMQ=="}
```

3.6 Tester une connexion

Tester essaie la connexion avec ce qui est saisi, **sans rien enregistrer** : Atlas attache la source sous un alias temporaire, lit `@@VERSION` et détache. Les réponses possibles :

Message	Signification et piste
Connexion réussie : Microsoft SQL Server 2025…	tout va bien ; cliquez Enregistrer
Saisissez le mot de passe de « … » pour tester.	authentification SQL sans mot de passe
… certificate …, TLS negotiation failed	cochez **Faire confiance au certificat** (Tester le fait pour vous, section 3.4)
Login failed, erreur 18456	utilisateur ou mot de passe refusés ; authentification SQL désactivée sur le serveur
timeout, *refused*, *resolve*	hôte, port (`hôte,1433`), pare-feu ; une instance nommée nécessite le service SQL Browser (UDP 1434)
extension DuckDB « mssql » non installée	le poste n'a pas pu télécharger l'extension : accès Internet ou proxy

3.7 Les droits nécessaires sur SQL Server

Atlas n'a besoin que de droits de **lecture des métadonnées**. Voici un compte dédié, à créer une fois par serveur :

EXEMPLE **Un compte de lecture pour Atlas** droits_atlas.sql

```
CREATE LOGIN atlas_lecture WITH PASSWORD = N'…', CHECK_POLICY = ON;
GRANT VIEW ANY DEFINITION TO atlas_lecture;    -- code des modules, toutes les bases
GRANT VIEW SERVER STATE TO atlas_lecture;      -- statistiques, plans en cache, index manquants
GRANT CONNECT ANY DATABASE TO atlas_lecture;   -- se connecter aux bases (SQL Server 2014 et +)
USE msdb;
CREATE USER atlas_lecture FOR LOGIN atlas_lecture;
GRANT SELECT ON dbo.sysjobs TO atlas_lecture;          -- jobs SQL Agent
GRANT SELECT ON dbo.sysjobsteps TO atlas_lecture;
GRANT SELECT ON dbo.sysjobschedules TO atlas_lecture;
GRANT SELECT ON dbo.sysschedules TO atlas_lecture;
GRANT SELECT ON dbo.sysjobhistory TO atlas_lecture;
```

À retenir

Sans {c('VIEW SERVER STATE')}, l'analyse fonctionne mais les chapitres Performances et Flux (volumes, fréquences) sont vides : décochez alors l'option *Statistiques d'exécution*. L'option *Profiler les colonnes* lit les données (nombre de valeurs distinctes, de nuls, sur un échantillon) : elle demande en plus {c('SELECT')} sur les tables (rôle {c('db_datareader')}).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

3.8 Ce qu'Atlas lit sur le serveur

Une campagne envoie, **pour chaque base**, une requête par genre d'information ; puis deux requêtes au niveau du serveur. Toutes sont des `SELECT` sur des vues système.

Genre	Vues système lues	Rangé dans
objets	`sys.objects`, `sys.schemas`, `sys.partitions`, `sys.allocation_units`, `sys.synonyms`, `sys.foreign_keys`, `sys.indexes`, `sys.stats`, `sys.types`, `sys.assemblies`	`ext_objets`
modules	`sys.sql_modules` (le code)	`ext_objets`
colonnes	`sys.columns`, `sys.types`, `sys.index_columns`, `sys.foreign_key_columns`, `sys.default_constraints`	`ext_colonnes`
dépendances	`sys.sql_expression_dependencies`	`ext_dependances`
stats	`sys.dm_exec_procedure_stats`, `sys.dm_exec_query_stats`, `sys.dm_exec_query_plan`	`ext_stats`
index manquants	`sys.dm_db_missing_index_*`	`ext_index_manquants`
serveurs liés	`sys.servers`	`ext_objets`
jobs	`msdb.dbo.sysjobs`, `sysjobsteps`, `sysjobschedules`, `sysjobhistory`	`ext_jobs`

Classements et mots réservés

Les requêtes d'extraction ajoutent {c('COLLATE DATABASE_DEFAULT')} à chaque texte concaténé : une base en {c('Latin1_General_BIN')} et les vues système en {c('Latin1_General_CI_AS_KS_WS')} ne provoquent donc plus l'erreur 451 (conflit de classement). Les noms de colonnes réservés ({c('[plan]')}) sont entre crochets.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 4 · LA FILE D'ATTENTE

Une file d'attente, 3 à 10 agents

Comment une campagne est découpée en travaux, comment les agents les exécutent en parallèle, et comment protéger vos serveurs en limitant la charge.

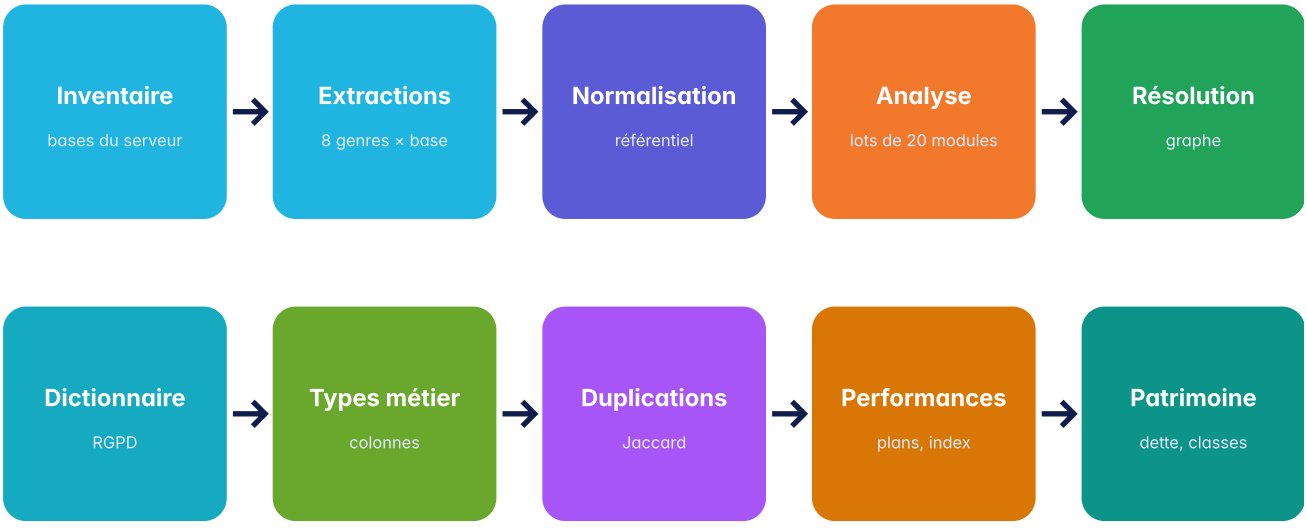
4.1 D'une campagne à des travaux
4.2 Les genres de travaux
4.3 Les états d'un travail
4.4 Les agents : de 3 à 10
4.5 Suivre et piloter la file

Dans ce chapitre, vous apprendrez à...

- comprendre le graphe des travaux d'une campagne
- connaître les genres et les états des travaux
- régler le nombre d'agents selon la machine
- limiter la charge par serveur
- suivre l'avancement dans la grille et le Gantt
- piloter la file : pause, arrêt, relance, priorité

4.1 D'une campagne à des travaux

Cliquer ▶ Analyser lance une **campagne** : Atlas la découpe en **travaux** indépendants, reliés par des dépendances, qu'il place dans une file d'attente. Pour la démonstration, la campagne compte 37 travaux.



Les travaux d'une même rangée peuvent tourner en même temps ; une flèche signifie « attend la fin de ». Le dictionnaire part dès la normalisation ; duplications et performances attendent la résolution ; le patrimoine at-

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

tend tout le reste.

4.2 Les genres de travaux

Genre	Ce qu'il fait	Interroge le serveur ?
inventaire	liste les bases à analyser et crée un travail d'extraction par base et par genre	oui
extraction	une requête catalogue (section 3.8) ; résultat inséré en masse dans `ext_*`	oui
normalisation	construit `objets`, `colonnes`, statistiques, jobs ; découpe le code en lots	non
analyse	analyse T-SQL d'un lot de modules (chapitre 6)	non
resolution	transforme les références en graphe (chapitre 8)	non
dictionnaire	dictionnaire intelligent et RGPD (chapitre 10)	non
types	types métier (chapitre 11)	non
duplications	code dupliqué (chapitre 15)	non
performances	diagnostic des performances (chapitre 14)	non
patrimoine	métriques, dette, classes (chapitre 16)	non
profilage	cardinalités et nuls d'une table (option)	oui
captures	import des appels interceptés (chapitre 19)	oui
rejeu	rejoue un appel dans une transaction annulée (chapitre 19)	oui

À retenir

Seuls les genres **inventaire, extraction, profilage, captures et rejeu** sollicitent les serveurs. Tout le reste travaille sur la base locale : relancer une analyse après avoir enrichi le glossaire, par exemple, ne coûte rien aux serveurs.

4.3 Les états d'un travail

État	Signification
en attente	prêt ou attend la fin de travaux dont il dépend
▶ **en cours**	un agent l'exécute ; la colonne Avancement progresse
✔ **terminé**	fini sans erreur ; la colonne Message résume le résultat
✖ **erreur**	l'exécution a échoué ; le message donne l'erreur du serveur ou de DuckDB
⛔ **bloqué**	une dépendance **stricte** a échoué ou a été annulée : le travail ne peut pas partir
○ annulé	annulé par vous, ou par ■ Arrêter

Dépendances strictes et souples

Un travail ne démarre que lorsque ses dépendances sont terminées. Si l'une échoue, il est **bloqué**… sauf si la dépendance est **souple**. Ainsi l'analyse patrimoniale attend les duplications, les performances, le dictionnaire et les types, mais s'exécute quand même si l'un d'eux échoue : elle travaille alors avec ce qui existe. Seul l'échec de la résolution la bloque.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

4.4 Les agents : de 3 à 10

Un **agent** est un fil d'exécution qui prend le travail prêt le plus prioritaire, l'exécute, puis recommence. Chaque agent a sa propre connexion à la base DuckDB ; les requêtes lourdes s'exécutent donc réellement en parallèle.

- Le curseur **Agents** règle leur nombre, de **3 à 10**, à chaud : un agent supplémentaire démarre aussitôt, un agent retiré finit son travail en cours.
- Choisissez-le selon la machine (cœurs, mémoire) et la charge acceptable pour les serveurs : 4 à 6 est un bon départ.
- La limite de connexions de l'extension {c('mssql')} suit ce réglage (agents + 2).

Ménager les serveurs : la limite par serveur

Le compteur **par serveur** (et le champ *Agents au plus sur ce serveur* de chaque source) fixe le nombre maximal de travaux qui interrogent **en même temps** un même serveur. Avec 8 agents et une limite de 2, Atlas peut analyser du code en local avec 6 agents pendant que 2 seulement lisent le serveur de production.

> **Astuce**
> Sur un serveur de production en journée : limite à **1**. La campagne reste rapide, car seules les extractions touchent le serveur ; tout le reste se fait en local.

4.5 Suivre et piloter la file

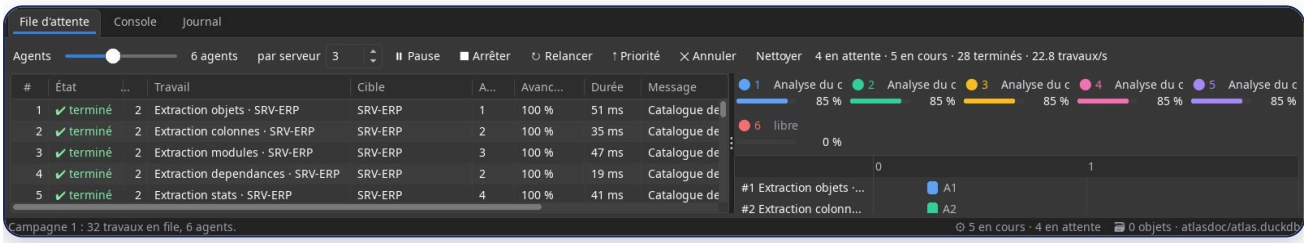


Figure 8 — Le panneau File d'attente : réglages et commandes en haut, liste des travaux à gauche, cartes des agents et chronologie à droite.

Élément	Lecture
Compteurs	`4 en attente · 5 en cours · 28 terminés · 22.8 travaux/s` : l'état de la campagne et son débit
Grille	numéro, état, priorité, travail, cible, agent, avancement, durée, message ; couleurs par état ; double-clic : détail du travail dans Propriétés
Cartes des agents	une par agent : numéro, travail en cours, barre de progression ; infobulle avec la durée
Chronologie	un diagramme de Gantt : chaque travail est une barre sur la ligne de son agent

Commande	Effet
⏸ Pause	les travaux en cours se terminent, aucun autre ne démarre ; second clic : reprise
■ Arrêter	interrompt les requêtes en cours et annule la file
↻ Relancer	remet en file les travaux en erreur, annulés ou bloqués (et ceux qui en dépendent)
↑ Priorité	augmente la priorité du travail sélectionné
✕ Annuler	annule le travail sélectionné (interrompt sa requête s'il est en cours)

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Commande	Effet
Nettoyer	retire de la liste les travaux terminés et annulés

À vous de jouer

Relancez l'analyse avec 3 agents, puis avec 10, et comparez le débit affiché (travaux/s) et la durée annoncée en fin de campagne.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 5 · L'ENTREPÔT DUCKDB

Toutes les informations, en local

L'entrepôt DuckDB du projet : ses zones, ses tables principales, et comment l'interroger en SQL ou l'exploiter dans vos propres programmes Lexis+.



5.1 Une base DuckDB par projet
5.2 Interroger l'entrepôt en SQL
5.3 Exploiter l'entrepôt avec Lexis+
5.4 Exporter, sauvegarder, partager

Dans ce chapitre, vous apprendrez à...

- savoir ce que contient atlas.duckdb
- distinguer extraction, référentiel et atelier
- interroger l'entrepôt avec DuckDB
- écrire un rapport Lexis+ sur l'entrepôt
- exporter en CSV, Markdown ou Parquet
- sauvegarder et partager un projet

5.1 Une base DuckDB par projet

Tout ce qu'Atlas sait est rangé dans `atlas.duckdb`, une base **DuckDB** : un seul fichier, des requêtes SQL analytiques très rapides, lisible par de nombreux outils (DuckDB CLI, Python, R, DBeaver, DuckDB Studio livré avec Lexis+). Les vues d'Atlas ne sont que des requêtes sur cette base.

Trois zones

Zone	Tables	Rôle
Extraction	`ext_objets`, `ext_colonnes`, `ext_dependances`, `ext_stats`, `ext_index_manquants`, `ext_jobs`, `ext_profils`	le brut, tel que lu sur chaque source
Référentiel et analyses	`objets`, `colonnes`, `dependances`, `instructions`, `ast`, `diagnostics`, `metriques`, `dico_*` ...	ce que calculent les moteurs
Atelier	`sources`, `travaux`, `journal`, `favoris`, `espaces`, `meta`	fonctionnement de l'application

L'annexe B décrit les 33 tables colonne par colonne. Les plus utiles :

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Table	Une ligne =	Colonnes clés
`objets`	un objet (table, procédure, job…)	`id, serveur, base, sch, nom, genre, definition, lignes, modifie`
`dependances`	une arête du graphe	`de_id → vers_id, nature, interbase, interserveur, frequence`
`diagnostics`	un constat	`objet_id, module, regle, niveau, message, suggestion, correctif, ligne`
`metriques`	les mesures d'un objet	`complexite, entrant, sortant, dette_min, maintenabilite, classe, score`
`dico_tables`, `dico_colonnes`	le dictionnaire	`objet_metier, famille, sensibilite, rgpd, description`

5.2 Interroger l'entrepôt en SQL

Fermez Atlas (ou copiez le fichier), puis ouvrez la base : `duckdb ~/AtlasSQL/atlas.duckdb`. Voici quatre requêtes types, exécutées sur la démonstration.

EXEMPLE Combien d'objets de chaque genre ? duckdb

```
1  select genre, count(*) as n from objets group by genre order by n desc limit 8
```

▶ CE QUE PRODUIT ATLAS

genre	n
PROCEDURE	66
CONTRAINTE	41
INDEX	33
TABLE	33
STATISTIQUE	27
JOB	9
VUE	6
PARTITION	6

EXEMPLE Les objets les plus risqués duckdb

```
1  select o.sch || '.' || o.nom as objet, m.complexite, m.entrant, m.sortant,
2         round(m.maintenabilite) as mi, round(m.dette_min) as dette, round(m.score) as score, m.classe
3  from metriques m join objets o on o.id = m.objet_id
4  order by m.score desc limit 6
```

▶ CE QUE PRODUIT ATLAS

objet	complexite	entrant	sortant	mi	dette	score	classe
vente.SP_EMISSION_FACTURE	7	1	11	41	205	86	Critique, À refactoriser
vente.SP_CLIENT_FICHE	1	0	4	68	195	61	Critique, Sensible, Obsolète, À refactoriser
stock.SP_INVENTAIRE	3	1	3	57	140	55	Critique, À refactoriser
vente.FACTURE	1	11	1		60	50	Critique
dbo.SP_LOG	1	29	1	75	0	47	
rh.EMPLOYE	1	8	0		60	44	Critique, Sensible

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE Toutes les dépendances inter-serveurs — duckdb

```duckdb
select a.serveur as de_serveur, a.nom as de_objet, d.nature, d.vers_nom
from dependances d join objets a on a.id = d.de_id
where d.interserveur order by 1, 2
```

▶ CE QUE PRODUIT ATLAS

de_serveur	de_objet	nature	vers_nom
SRV-DWH	SP_CHARGER_DIM_CLIENT	SELECT	SRV-ERP.ERP.vente.CLIENT
SRV-ERP	SP_EMISSION_FACTURE	INSERT	SRV-COMPTA.COMPTA.dbo.ECRITURE
SRV-ERP	SP_EXPORT_DWH	INSERT	SRV-DWH.DWH.stg.STG_FACTURE
SRV-ERP	SP_EXPORT_DWH	EXEC	SRV-DWH.DWH.dw.SP_CHARGER_FAIT_VENTE
SRV-ERP	SP_SYNCHRO_TIERS	SELECT	SRV-COMPTA.COMPTA.dbo.ECRITURE
SRV-ERP	SP_SYNCHRO_TIERS	OPENQUERY	LNK_COMPTA
SRV-ERP	SYN_ECRITURE	SYNONYME	SRV-COMPTA.COMPTA.dbo.ECRITURE

EXEMPLE Impact d'une modification de la table CLIENT (CTE récursive) — duckdb

```duckdb
with recursive imp(id, niveau) as (
  select id, 0 from objets where nom = 'CLIENT' and genre = 'TABLE'
  union
  select d.de_id, i.niveau + 1 from dependances d join imp i on d.vers_id = i.id where i.niveau < 3)
select o.sch || '.' || o.nom as objet, o.genre, min(niveau) as niveau
from imp join objets o on o.id = imp.id group by all order by niveau, objet limit 12
```

▶ CE QUE PRODUIT ATLAS

objet	genre	niveau
vente.CLIENT	TABLE	0
dbo.V_CLIENT_COMPLET	VUE	1
dw.SP_CHARGER_DIM_CLIENT	PROCEDURE	1
vente.COMMANDE	TABLE	1
vente.FN_CA_CLIENT	FONCTION	1
vente.SP_CA_CLIENT	PROCEDURE	1
vente.SP_CA_MENSUEL	PROCEDURE	1
vente.SP_CLIENT_AJOUTER	PROCEDURE	1
vente.SP_CLIENT_FICHE	PROCEDURE	1
vente.SP_CLIENT_LIRE	PROCEDURE	1
vente.SP_CLIENT_LISTE	PROCEDURE	1
vente.SP_CLIENT_MODIFIER	PROCEDURE	1

Pour bien comprendre

La dernière requête est celle qu'utilise Atlas pour l'**impact transitif** (chapitre 8) : on remonte les arêtes « qui dépend de moi ? » niveau par niveau. Le niveau 1 est ce qui lit ou écrit directement CLIENT ; le niveau 2, ce qui dépend de ceux-là.

5.3 Exploiter l'entrepôt avec Lexis+

Un programme Lexis+ peut ouvrir l'entrepôt et en tirer des rapports, des tableaux de bord ou des contrôles automatiques (par exemple dans une chaîne d'intégration). Les trois programmes ci-dessous ont été exécutés

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

avec `wdlexisrt`.

EXEMPLE **Les cinq objets les plus risqués** risques.prg

```
&& Les 5 objets les plus risqués du patrimoine
ouvrirbase("~/AtlasSQL/atlas.duckdb")
pour chaque r dans requete("select o.sch || '.' || o.nom as objet, round(m.score) as score, m.classe " + ;
                           "from metriques m join objets o on o.id = m.objet_id order by m.score desc limit 5")
  ? r.objet, "·", r.score, "·", r.classe
suivant
```

▶ CE QU'AFFICHE LE PROGRAMME

```
vente.SP_EMISSION_FACTURE · 86 · Critique, À refactoriser
vente.SP_CLIENT_FICHE · 61 · Critique, Sensible, Obsolète, À refactoriser
stock.SP_INVENTAIRE · 55 · Critique, À refactoriser
vente.FACTURE · 50 · Critique
dbo.SP_LOG · 47 ·
```

EXEMPLE **Qui écrit dans une table ?** ecrivains.prg

```
ouvrirbase("~/AtlasSQL/atlas.duckdb")
locale table_ = "FACTURE"
locale l = requete("select distinct a.sch || '.' || a.nom as objet, d.nature, d.frequence " + ;
                   "from dependances d join objets a on a.id = d.de_id join objets t on t.id = d.vers_id " + ;
                   "where t.nom = ? and d.nature in ('INSERT', 'UPDATE', 'DELETE', 'MERGE') order by 1, 2", table_)
? #l, "écriture(s) dans", table_
pour chaque r dans l
  ? " -", r.nature, r.objet, "(" + r.frequence + " exécutions)"
suivant
```

▶ CE QU'AFFICHE LE PROGRAMME

```
2 écriture(s) dans FACTURE
 - DELETE vente.SP_EMISSION_FACTURE (365 exécutions)
 - INSERT vente.SP_EMISSION_FACTURE (365 exécutions)
```

EXEMPLE **Exporter les constats critiques pour un tableur** export.prg

```
ouvrirbase("~/AtlasSQL/atlas.duckdb")
locale n = valeursql("select count(*) from diagnostics where niveau = 'critique'")
requete("copy (select d.module, d.regle, o.sch || '.' || o.nom as objet, d.ligne, d.message, d.suggestion " + ;
        "from diagnostics d left join objets o on o.id = d.objet_id where d.niveau = 'critique' order by 1, 3) " + ;
        "to 'critiques.csv' (header, delimiter ';')")
? n, "constats critiques exportés"
```

▶ CE QU'AFFICHE LE PROGRAMME

```
12 constats critiques exportés
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Astuce

Le paramètre {c('?')} de {c('requete')} évite de construire le SQL par concaténation : c'est exactement la bonne pratique qu'Atlas recommande pour le T-SQL (chapitre 13).

5.4 Exporter, sauvegarder, partager

- *Fichier › Exporter l'inventaire (CSV)…* et *Exporter les dépendances (CSV)…* produisent des fichiers séparés par des points-virgules, prêts pour un tableur.
- *Fichier › Exporter le dictionnaire (Markdown)…* produit une documentation lisible des tables et colonnes (chapitre 10).
- Pour sauvegarder un projet, copiez son dossier, Atlas fermé. Le fichier ne contient aucun mot de passe.
- Pour comparer deux campagnes, copiez la base avant de relancer l'analyse : chaque nouvelle extraction remplace celle de la source.

EXEMPLE **Exporter tout le graphe au format Parquet (DuckDB)** duckdb

```duckdb
copy (select * from dependances) to 'dependances.parquet' (format parquet);
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 6 · L'ANALYSE DU CODE

Lire le T-SQL comme un expert

Comment Atlas découpe le code en jetons et en instructions, construit l'arbre, attribue les phases, relève les références, les variables, la complexité, les jointures et les règles.



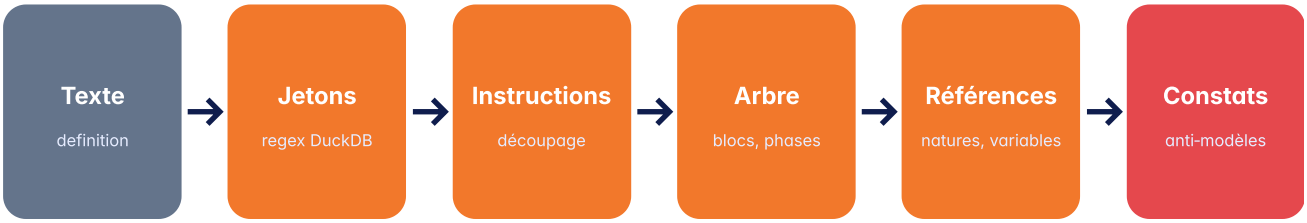
6.1 La chaîne d'analyse
6.2 Les jetons
6.3 Des jetons aux instructions, puis à l'arbre
6.4 Les phases : le synoptique du code
6.5 Les références
6.6 Variables, tables temporaires, curseurs et CTE
6.7 La complexité cyclomatique
6.8 Jointures et règles de gestion
6.9 Ce que l'analyseur ne fait pas

Dans ce chapitre, vous apprendrez à...

- suivre la chaîne d'analyse d'un module
- lire la liste des jetons et leurs genres
- comprendre le découpage en instructions
- interpréter les phases et le synoptique
- reconnaître les natures de références
- calculer une complexité cyclomatique

6.1 La chaîne d'analyse

Chaque module (procédure, fonction, vue, déclencheur, étape de job) passe par la même chaîne. Elle sert aussi à l'éditeur, qui analyse le code **pendant la frappe** (chapitre 17).



Pourquoi DuckDB découpe le texte

Parcourir un long texte caractère par caractère serait lent dans un langage interprété. Atlas confie donc le découpage en jetons à une seule expression régulière exécutée par DuckDB ({c('regexp_extract_all')}) : environ 40 000 jetons en 0,3 s, pour des dizaines de modules à la fois.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

6.2 Les jetons

Un **jeton** est le plus petit morceau qui a un sens : un mot, une chaîne, un nombre, une variable, un opérateur. Les commentaires disparaissent ; chaque jeton garde son numéro de ligne.

EXEMPLE **Découper deux lignes de T-SQL** jetons

```
1  SELECT @n = COUNT(*) -- total
2  FROM [vente].COMMANDE WHERE STATUT = N'V' AND QTE > 10
```

▶ CE QUE PRODUIT ATLAS

ligne	jeton	genre
1	`SELECT`	W
1	`@n`	V
1	`=`	P
1	`COUNT`	W
1	`(`	P
1	`*`	P
1	`)`	P
2	`FROM`	W
2	`[vente]`	I
2	`.`	P
2	`COMMANDE`	W
2	`WHERE`	W
2	`STATUT`	W
2	`=`	P
2	`N'V'`	S
2	`AND`	W
2	`QTE`	W
2	`>`	P
2	`10`	N

Genre	Sens	Exemples
W	mot (mot-clé ou nom)	`SELECT` `COMMANDE`
V	variable	`@n` `@@ROWCOUNT`
T	table temporaire	`#cmd`
S	chaîne	`N'V'` `'x'`
N	nombre	`10`
I	identifiant entre crochets	`[vente]`
P	ponctuation, opérateur	`=` `(` `.`

6.3 Des jetons aux instructions, puis à l'arbre

Les jetons sont regroupés en **instructions**. Une instruction se termine sur un `;` ou quand un mot-clé en commence une nouvelle — avec des exceptions : `INSERT … SELECT`, `WITH … SELECT`, `UNION`, `MERGE … ;`,

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

`UPDATE … SET`, `DECLARE c CURSOR FOR SELECT` restent une seule instruction. L'en-tête `CREATE PROCEDURE … AS` fournit les paramètres.

Les instructions forment ensuite un **arbre** : un `BEGIN … END` contient ses instructions ; un `IF` contient son corps et son `ELSE` ; `BEGIN TRY` et `BEGIN CATCH` sont des blocs. L'onglet **Arbre** de l'éditeur affiche cet arbre (chapitre 17).

EXEMPLE **Une procédure de clôture journalière** `vente.SP_CLOTURE_JOUR`

```
1  CREATE PROCEDURE vente.SP_CLOTURE_JOUR @Jour DATE
2  AS
3  BEGIN
4    SET NOCOUNT ON;
5    DECLARE @nb INT = 0;
6    IF @Jour IS NULL
7      THROW 50001, 'Jour obligatoire', 1;
8    SELECT @nb = COUNT(*) FROM vente.COMMANDE WHERE DATE_COMMANDE = @Jour;
9    UPDATE vente.COMMANDE SET STATUT = 'C' WHERE DATE_COMMANDE = @Jour AND STATUT = 'V';
10   INSERT INTO dbo.LOG_TRAITEMENT (DATE_LOG, PROCEDURE_NOM, MESSAGE)
11   VALUES (SYSDATETIME(), 'SP_CLOTURE_JOUR', CONCAT(@nb, ' commandes'));
12   RETURN @nb;
13 END
```

▶ **CE QUE PRODUIT ATLAS**

n°	ligne	genre	phase
2	3	BLOC	Initialisation
3	4	SET	Initialisation
4	5	DECLARE	Déclarations
5	6	IF	Contrôles
6	7	THROW	Contrôles
7	8	SELECT	Chargement
8	9	UPDATE	Écriture
9	10	INSERT	Traces
10	12	RETURN	Restitution

6.4 Les phases : le synoptique du code

Chaque instruction reçoit une **phase** qui dit à quoi elle sert. La suite des phases est le **synoptique** du module : pour la procédure ci-dessus, Atlas obtient **Initialisation → Déclarations → Contrôles → Chargement → Écriture → Traces → Restitution**.

Phase	Instructions concernées
Initialisation	`SET NOCOUNT ON`, `SET XACT_ABORT ON`, affectation d'une constante
Déclarations	`DECLARE` (variables, variables tables, curseurs)
Contrôles	`IF` dont le corps lève une erreur ou sort (`THROW`, `RAISERROR`, `RETURN`)
Chargement	lecture vers une variable ou une table temporaire (`SELECT @x = …`, `INSERT INTO #t SELECT`)
Calcul	travail sur des tables temporaires, `SET @x = @y * 2`, curseurs (`OPEN`, `FETCH`)
Écriture	`INSERT`, `UPDATE`, `MERGE` sur une table permanente
Purge	`DELETE` ou `TRUNCATE` d'une table permanente
Archivage	écriture dans une table `…HIST…` ou `…ARCH…`

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Phase	Instructions concernées
Traces	`PRINT`, écriture dans une table `LOG`, `AUDIT`, `TRACE`, `JOURNAL`
SQL dynamique	construction de la requête dans une variable, `EXEC (@sql)`, `sp_executesql`
Transaction	`BEGIN TRAN`, `COMMIT`, `ROLLBACK`
Gestion d'erreurs	bloc `BEGIN CATCH`
Nettoyage	`CLOSE`, `DEALLOCATE`, `DROP TABLE #t`
Traitements	appel d'une autre procédure (`EXEC`), autres instructions
Restitution	`SELECT` qui renvoie un résultat, `RETURN`

La phase d'un IF dépend de son corps

Ligne 6 du code ci-dessus, {c('IF @Jour IS NULL THROW ...')} est un **contrôle** parce que son corps lève une erreur. Un {c('IF')} dont le corps fait surtout des écritures prend la phase **Écriture** : Atlas remonte la phase dominante des instructions du corps.

6.5 Les références

Pour chaque instruction, Atlas relève les objets cités et la **nature** de l'accès. Un nom peut avoir de une à quatre parties (`objet`, `schéma.objet`, `base.schéma.objet`, `serveur.base.schéma.objet`) ; les alias de tables sont résolus.

EXEMPLE Une procédure qui parle à trois serveurs — vente.SP_EXPORT_SOLDE

```
1  CREATE PROCEDURE vente.SP_EXPORT_SOLDE
2  AS
3  BEGIN
4    INSERT INTO LNK_COMPTA.COMPTA.dbo.ECRITURE (JOURNAL_CODE, DATE_ECRITURE, COMPTE, DEBIT)
5    SELECT 'VTE', DATE_FACTURE, '411000', MONTANT_TTC FROM vente.FACTURE WHERE STATUT = 'E';
6    SELECT * FROM OPENQUERY(LNK_DWH, 'SELECT CLIENT_SK, CODE_CLIENT FROM DWH.dw.DIM_CLIENT');
7    EXEC dbo.SP_LOG 'export';
8    SELECT dbo.FN_TTC(100, 'N');
9    MERGE dbo.SYN_ECRITURE AS cible USING vente.REGLEMENT AS src ON cible.PIECE = CAST(src.REGLEMENT_ID
   AS VARCHAR(20))
10   WHEN NOT MATCHED THEN INSERT (PIECE) VALUES (CAST(src.REGLEMENT_ID AS VARCHAR(20)));
11 END
```

▶ CE QUE PRODUIT ATLAS

nature	référence	ligne	serveur distant
INSERT	LNK_COMPTA.COMPTA.dbo.ECRITURE	4	LNK_COMPTA
SELECT	vente.FACTURE	5	
OPENQUERY	LNK_DWH	6	LNK_DWH
EXEC	dbo.SP_LOG	7	
APPEL	dbo.FN_TTC	8	
MERGE	dbo.SYN_ECRITURE	9	
SELECT	vente.REGLEMENT	9	

Nature	Détectée sur
SELECT	tables lues : `FROM`, `JOIN`, sous-requêtes, source d'un `MERGE`

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Nature	Détectée sur
INSERT · UPDATE · DELETE · MERGE	table cible de l'instruction
EXEC	procédure appelée
APPEL	fonction utilisateur (`dbo.FN_TTC(…)`)
OPENQUERY	serveur lié interrogé par `OPENQUERY` / `OPENROWSET` ; la requête distante est conservée

À retenir

Les tables temporaires ({c('#t')}), les variables tables ({c('@t')}), les CTE et les pseudo-tables {c('INSERTED')} / {c('DELETED')} des déclencheurs ne sont pas des dépendances : Atlas les reconnaît et les écarte. Une table citée **dans un SQL dynamique reconstitué** est, elle, bien relevée (chapitre 13).

6.6 Variables, tables temporaires, curseurs et CTE

Atlas recense tout ce que le module déclare ; l'éditeur s'en sert pour la complétion et le débogueur pour suivre les valeurs.

EXEMPLE Réapprovisionnement avec un curseur — `stock.SP_REAPPRO`

```
1  CREATE PROCEDURE stock.SP_REAPPRO @Seuil DECIMAL(10,3) = 5, @Entrepot INT
2  AS
3  BEGIN
4    DECLARE @n INT = 0;
5    DECLARE @a_commander TABLE (CODE_ARTICLE VARCHAR(15), QTE DECIMAL(10,3));
6    CREATE TABLE #manquants (CODE_ARTICLE VARCHAR(15));
7    WITH bas AS (SELECT CODE_ARTICLE, QTE_DISPO FROM stock.STOCK WHERE ENTREPOT_ID = @Entrepot AND
   QTE_DISPO < @Seuil)
8    INSERT INTO @a_commander SELECT CODE_ARTICLE, @Seuil * 2 - QTE_DISPO FROM bas;
9    DECLARE c_art CURSOR FOR SELECT CODE_ARTICLE FROM @a_commander;
10   OPEN c_art;
11   FETCH NEXT FROM c_art INTO @code;
12   WHILE @@FETCH_STATUS = 0
13   BEGIN
14     SET @n += 1;
15     FETCH NEXT FROM c_art INTO @code;
16   END
17   CLOSE c_art; DEALLOCATE c_art;
18   DROP TABLE #manquants;
19 END
```

▶ CE QUE PRODUIT ATLAS

nom	genre	type	ligne	défaut
@Seuil	parametre	DECIMAL(10,3)	1	5
@Entrepot	parametre	INT	1	
@n	variable	INT	4	0
@a_commander	variable_table	TABLE	5	
#manquants	table_temp	CREATE TABLE	6	
bas	cte		7	
c_art	curseur	CURSOR	9	

Le même code produit un constat : « *Curseur c_art : traitement ligne à ligne* », avec la suggestion de le ré-écrire en traitement ensembliste (chapitre 7).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

6.7 La complexité cyclomatique

La **complexité cyclomatique** compte les chemins indépendants à travers le code : c'est le nombre minimal de cas de test pour passer partout. Atlas part de 1 et ajoute 1 pour chaque `IF`, chaque `WHILE`, chaque `AND` / `OR` de leurs conditions, chaque `WHEN` d'un `CASE` et chaque bloc `CATCH`.

Module	Calcul	Complexité
`vente.SP_CLOTURE_JOUR` (section 6.3)	1 + 1 IF	**2**
`vente.V_CA_CLIENT` (section 6.8)	1 + 2 WHEN	**3**
`vente.SP_EMISSION_FACTURE` (démonstration)	1 + 3 IF + 1 AND (condition d'un IF) + 1 WHILE + 1 CATCH	**7**

À retenir

Au-delà de 12, Atlas classe le module « À refactoriser » ; au-delà de 20, le constat devient important (chapitre 16).

6.8 Jointures et règles de gestion

Deux autres informations sont extraites pour la détection des doublons (chapitre 15) et l'analyse des performances (chapitre 14) :

- les **jointures** : quelles colonnes de quelles tables sont comparées ;
- les **règles** : chaque {c('CASE')} et chaque filtre {c('WHERE')}, sous une forme normalisée (alias, espaces et casse uniformisés).

EXEMPLE Une vue de chiffre d'affaires — `vente.V_CA_CLIENT`

```
1  CREATE VIEW vente.V_CA_CLIENT AS
2  SELECT C.CODE_CLIENT, C.NOM,
3         SUM(F.MONTANT_TTC) AS CA,
4         CASE WHEN SUM(F.MONTANT_TTC) > 10000 THEN 'GRAND' WHEN SUM(F.MONTANT_TTC) > 1000 THEN 'MOYEN'
   ELSE 'PETIT' END AS SEGMENT
5  FROM vente.CLIENT C
6  JOIN vente.FACTURE F ON F.CODE_CLIENT = C.CODE_CLIENT
7  LEFT JOIN REF.dbo.PAYS P ON P.CODE = C.PAYS_CODE
8  WHERE F.STATUT <> 'A'
9  GROUP BY C.CODE_CLIENT, C.NOM
```

▶ CE QUE PRODUIT ATLAS

ligne	colonne	= colonne
6	CLIENT.CODE_CLIENT	FACTURE.CODE_CLIENT
7	CLIENT.PAYS_CODE	PAYS.CODE

genre	règle normalisée
CASE	CASE WHEN SUM (F . MONTANT_TTC) > 10000 THEN 'GRAND' WHEN SUM (F . MONTANT_TTC) > 1000 THEN 'MOYEN' ELSE 'PETIT' END
FILTRE	F . STATUT <> 'A'

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

6.9 Ce que l'analyseur ne fait pas

L'analyseur est **tolérant** : il ne s'arrête jamais sur une erreur de syntaxe, et ne prétend pas être un compilateur T-SQL complet.

- Les constructions rares (XML DML, CLR, certaines options de requête) sont gardées comme texte.
- Les noms construits dynamiquement ({c('FROM vente.{@Table}')}) ne peuvent pas être résolus : ils apparaissent comme références introuvables avec le marqueur {c('{...}')}.
- Les éléments ajoutés aux tout derniers T-SQL (2025) sont lus comme des mots ordinaires ; ils ne gênent pas le découpage.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 7 · LES CONSTATS

Les constats, du critique à l'opportunité

Les diagnostics produits par Atlas : niveaux de gravité, les quatorze règles de l'analyseur avec des corrections commentées, et les constats des autres moteurs.



7.1 Trois niveaux de gravité
7.2 Un module qui accumule les défauts
7.3 Les règles de l'analyseur
7.4 Les constats des autres moteurs
7.5 L'onglet Constats

Dans ce chapitre, vous apprendrez à...

- distinguer critique, important et opportunité
- reconnaître les quatorze anti-modèles détectés
- corriger NOT IN, NOLOCK, @@IDENTITY, transactions
- savoir quel moteur produit quel constat
- filtrer les constats et ouvrir le code à la ligne
- relier constats et dette technique

7.1 Trois niveaux de gravité

Un **constat** (ou diagnostic) est une remarque d'Atlas sur un objet : un risque, une lenteur probable, une incohérence, une opportunité. Chacun a un **module** (le moteur qui l'a produit), une **règle**, un **niveau**, un message, une suggestion et, quand c'est possible, un **correctif** T-SQL prêt à relire.

Niveau	Signe	Sens	Poids dans la dette
critique	●	risque réel de données fausses, de faille ou d'incident ; à traiter en priorité	60 min
important	▲	défaut qui coûte (performances, maintenance) ; à planifier	20 min
opportunité	◇	amélioration souhaitable, sans urgence	5 min

7.2 Un module qui accumule les défauts

Cette procédure (volontairement mauvaise) déclenche dix règles différentes de l'analyseur. Analysez-la vous-même : collez-la dans l'onglet **Code** (Nouveau), les constats apparaissent soulignés dans la marge et listés dans l'onglet **Analyse**.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE **Dix mauvaises pratiques en seize lignes** dbo.SP_MAUVAISES_PRATIQUES

```sql
CREATE PROCEDURE dbo.SP_MAUVAISES_PRATIQUES @Nom NVARCHAR(80)
AS
BEGIN
  DECLARE @sql NVARCHAR(MAX), @id INT;
  SELECT * FROM vente.CLIENT WITH (NOLOCK) WHERE UPPER(NOM) = UPPER(@Nom);
  SELECT CODE_CLIENT FROM vente.CLIENT WHERE NOM LIKE '%' + @Nom;
  SELECT NUM_FACTURE FROM vente.FACTURE WHERE CODE_CLIENT NOT IN (SELECT CODE_CLIENT FROM
vente.CLIENT_OLD);
  DELETE FROM dbo.TMP_IMPORT_CLIENT;
  BEGIN TRANSACTION;
  INSERT INTO vente.CONTACT (NOM) VALUES (@Nom);
  SET @id = @@IDENTITY;
  COMMIT;
  SET @sql = N'SELECT * FROM vente.CLIENT WHERE NOM = ''' + @Nom + N'''';
  EXEC (@sql);
  WAITFOR DELAY '00:00:05';
END
```

▶ CE QUE PRODUIT ATLAS

règle	niveau	ligne	message
SELECT_ETOILE	opportunite	5	SELECT * : toutes les colonnes sont lues
FONCTION_FILTRE	important	5	Fonction UPPER() sur une colonne filtrée : l'index ne peut pas être utilisé (filtrage tardif)
NOLOCK	important	5	Lecture NOLOCK : lectures sales, lignes manquantes ou en double
LIKE_DEBUT	important	6	LIKE '%…' : recherche sans index possible (balayage)
NOT_IN	important	7	NOT IN (SELECT …) : piège des NULL et plan coûteux
SANS_WHERE	critique	8	DELETE sans WHERE sur TMP_IMPORT_CLIENT
TRANSACTION_SANS_TRY	important	9	Transaction ouverte hors d'un bloc TRY…CATCH
IDENTITY	important	11	@@IDENTITY peut renvoyer l'identité créée par un déclencheur
INJECTION	critique	14	SQL dynamique concaténé avec des variables : risque d'injection SQL
WAITFOR	opportunite	15	WAITFOR : attente active dans le code

7.3 Les règles de l'analyseur

Règle	Niveau	Pourquoi c'est un problème	Ce qu'Atlas propose
SELECT_ETOILE	opportunité	SELECT * lit toutes les colonnes : plus d'entrées/sorties, pas de plan « couvrant » possible, et le code casse si la table change.	Lister les colonnes utiles.
SANS_WHERE	critique	Un UPDATE ou un DELETE sans WHERE modifie toute la table. Parfois voulu (table de travail), souvent un oubli.	Vérifier l'intention ; TRUNCATE si autorisé, sinon un filtre explicite.
FONCTION_FILTRE	important	UPPER(NOM) = ..., YEAR(DATE) = ... : la fonction appliquée à la colonne empêche l'usage de son index (filtrage « tardif », ligne à ligne).	Réécrire en intervalle, ou ajouter une colonne calculée indexée.
LIKE_DEBUT	important	LIKE '%texte' ne peut pas utiliser d'index : toute la table est parcourue.	Recherche plein texte, colonne inversée indexée, ou préfixe connu.
NOT_IN	important	NOT IN (SELECT ...) renvoie zéro ligne dès que la sous-requête contient un NULL, et donne souvent un plan coûteux.	NOT EXISTS (corrélé).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Règle	Niveau	Pourquoi c'est un problème	Ce qu'Atlas propose
NOLOCK	important	WITH (NOLOCK) lit des données non validées : lignes manquantes, en double, ou annulées ensuite.	Isolement par instantané (READ_COMMITTED_SNAPSHOT) ou verrous adaptés.
IDENTITY	important	@@IDENTITY renvoie la dernière identité de la session, y compris celle créée par un déclencheur.	SCOPE_IDENTITY() ou OUTPUT INSERTED.id.
GOTO	opportunité	GOTO rend le flot de contrôle difficile à suivre.	Blocs IF / TRY.
CURSEUR	important	Un curseur traite les lignes une par une : souvent 10 à 100 fois plus lent qu'une instruction ensembliste.	UPDATE ... FROM, INSERT ... SELECT, MERGE ; à défaut, LOCAL FAST_FORWARD.
INJECTION	critique	EXEC (@sql) sur un texte construit avec des paramètres : un utilisateur peut injecter son propre SQL.	sp_executesql avec paramètres typés ; QUOTENAME() pour les noms d'objets.
TRANSACTION_SANS_TRY	important	Une transaction ouverte hors d'un TRY... CATCH reste ouverte en cas d'erreur (verrous, incohérences).	BEGIN TRY ... COMMIT ... END TRY BEGIN CATCH ROLLBACK ; THROW END CATCH, avec SET XACT_ABORT ON.
WAITFOR	opportunité	Une attente active dans le code occupe une connexion pour rien.	Planifier par SQL Agent ou Service Broker.
ROWCOUNT	opportunité	SET ROWCOUNT est obsolète pour limiter les écritures.	TOP (n).
BOUCLE	opportunité	Une boucle WHILE cache souvent un traitement qui pourrait être ensembliste.	Vérifier qu'une seule instruction ne suffit pas.

Quatre corrections commentées

AVANT NOT IN et les NULL — avant

```
1  SELECT NUM_FACTURE FROM vente.FACTURE
2  WHERE CODE_CLIENT NOT IN
3    (SELECT CODE_CLIENT FROM
   vente.CLIENT_OLD);
4  -- un seul NULL dans CLIENT_OLD
5  -- => aucune ligne !
```

APRÈS NOT IN et les NULL — après

```
1  SELECT F.NUM_FACTURE FROM vente.FACTURE F
2  WHERE NOT EXISTS
3    (SELECT 1 FROM vente.CLIENT_OLD O
4     WHERE O.CODE_CLIENT = F.CODE_CLIENT);
```

AVANT Fonction sur une colonne filtrée — avant

```
1  SELECT * FROM vente.COMMANDE
2  WHERE YEAR(DATE_COMMANDE) = 2026;
3  -- l'index sur DATE_COMMANDE est ignoré
```

APRÈS Fonction sur une colonne filtrée — après

```
1  SELECT * FROM vente.COMMANDE
2  WHERE DATE_COMMANDE >= '20260101'
3    AND DATE_COMMANDE <  '20270101';
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

AVANT Identité créée — avant

```
INSERT INTO vente.CONTACT (NOM) VALUES
(@Nom);
SET @id = @@IDENTITY;
-- un déclencheur d'audit qui insère
-- ailleurs fausse @id
```

APRÈS Identité créée — après

```
INSERT INTO vente.CONTACT (NOM)
OUTPUT INSERTED.CONTACT_ID INTO @ids
VALUES (@Nom);
-- ou : SET @id = SCOPE_IDENTITY();
```

AVANT Transaction protégée — avant

```
BEGIN TRANSACTION;
INSERT INTO vente.CONTACT (NOM) VALUES
(@Nom);
UPDATE vente.CLIENT SET ACTIF = 1
WHERE CODE_CLIENT = @Code;
COMMIT;
```

APRÈS Transaction protégée — après

```
SET XACT_ABORT ON;
BEGIN TRY
  BEGIN TRANSACTION;
  INSERT INTO vente.CONTACT (NOM) VALUES
(@Nom);
  UPDATE vente.CLIENT SET ACTIF = 1 WHERE
CODE_CLIENT = @Code;
  COMMIT;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK;
  THROW;
END CATCH;
```

7.4 Les constats des autres moteurs

L'analyseur n'est pas seul à produire des constats. Chaque moteur ajoute les siens (détail dans l'annexe A) :

Module	Règles	Chapitre
dependances	`REF_INTROUVABLE`	8
dictionnaire	`RGPD_SENSIBLE`	10
types	`TYPE_INCOHERENT`, `NOMMAGE_INCOHERENT`	11
duplications	`DUPLICATION`	15
performances	`TABLE_SCAN`, `JOINTURE_COUTEUSE`, `INDEX_MANQUANT`, `JOINTURE_SANS_INDEX`, `CURSEUR_INUTILE`, `FILTRAGE_TARDIF`	14
patrimoine	`COMPLEXITE`, `INUTILISE`	16

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE **Répartition des constats de la démonstration** duckdb

```duckdb
select module, niveau, count(*) as n from diagnostics group by all order by module, niveau
```

▶ CE QUE PRODUIT ATLAS

module	niveau	n
analyse	critique	2
analyse	important	18
analyse	opportunite	17
dependances	important	2
dictionnaire	critique	1
dictionnaire	important	4
duplications	opportunite	4
patrimoine	opportunite	10
performances	critique	9
performances	important	12
performances	opportunite	6
types	important	8
types	opportunite	4

7.5 L'onglet Constats



Figure 9 — L'onglet Constats : filtres par module et par niveau, puis la liste triée par gravité. Double-clic : ouvre le code à la bonne ligne.

- Les listes déroulantes filtrent par **module** et par **niveau** ; le compteur indique le nombre de constats affichés.
- Un **double-clic** rend l'objet courant et ouvre son code à la ligne du constat.
- Les constats d'un objet sont aussi listés dans sa page **Propriétés** et soulignés dans l'éditeur.

À vous de jouer

Filtrez sur le niveau **critique** : combien de constats la démonstration compte-t-elle ? Lequel concerne `rh.EMPLOYE` , et pourquoi ?

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 8 · LES DÉPENDANCES

Qui lit, qui écrit, qui appelle

Le graphe orienté des dépendances : natures, résolution des noms (serveurs liés, synonymes, OPENQUERY), fréquences et volumes, références introuvables et analyse d'impact.



8.1 Un graphe orienté
8.2 Résoudre les noms
8.3 Fréquence, volumes et complexité
8.4 Les références introuvables
8.5 L'onglet Dépendances
8.6 L'analyse d'impact

Dans ce chapitre, vous apprendrez à...

- lire une arête du graphe et sa nature
- comprendre la résolution des noms à 1 à 4 parties
- repérer les dépendances inter-bases et inter-serveurs
- exploiter fréquences et volumes
- diagnostiquer une référence introuvable
- mesurer l'impact d'une modification

8.1 Un graphe orienté

Le graphe des dépendances est le cœur d'Atlas. Chaque arête relie un objet **qui utilise** (`de_id`) à un objet **utilisé** (`vers_id`), avec la **nature** de l'usage. « `SP_EMISSION_FACTURE` —INSERT→ `FACTURE` » se lit : *la procédure écrit dans la table*.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE **Les natures du graphe de la démonstration** duckdb

```duckdb
select nature, count(*) as n from dependances group by 1 order by 2 desc
```

▶ CE QUE PRODUIT ATLAS

nature	n
SELECT	83
EXEC	42
INSERT	23
DELETE	16
UPDATE	16
FK	14
SYNONYME	3
MERGE	2
APPEL	2
DECLENCHEUR	2
OPENQUERY	1

Nature	Origine	Exemple
SELECT, INSERT, UPDATE, DELETE, MERGE	analyse du code	une procédure lit ou écrit une table
EXEC	analyse du code	une procédure en appelle une autre ; un job appelle une procédure
APPEL	analyse du code	utilisation d'une fonction utilisateur
OPENQUERY	analyse du code	requête envoyée à un serveur lié
FK	catalogue	clé étrangère : la table enfant dépend de la table parente
DECLENCHEUR	catalogue	le déclencheur dépend de sa table
SYNONYME	catalogue	un synonyme pointe vers sa cible (éventuellement distante)
CATALOGUE	SQL Server	dépendance déclarée par `sys.sql_expression_dependencies` et non retrouvée dans le code

8.2 Résoudre les noms

Dans le code, un nom est souvent incomplet : `FACTURE`, `vente.FACTURE`, `ERP.vente.FACTURE`, `LNK_COMPTA.COMPTA.dbo.ECRITURE`… La **résolution** retrouve l'objet réel, en SQL, pour toutes les références d'un coup :

1. Les parties manquantes sont complétées par le contexte du module : son serveur, sa base, puis son schéma (ou `dbo`).
2. Un nom à quatre parties désigne un **serveur lié** : Atlas le remplace par le serveur qu'il cible (`LNK_COMPTA` → SRV-COMPTA).
3. Un **synonyme** est suivi jusqu'à sa cible : la procédure dépend du synonyme **et**, « via synonyme », de l'objet réel.
4. Pour `OPENQUERY`, la requête envoyée au serveur lié est elle-même analysée : ses tables deviennent des dépendances distantes.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

5. La comparaison ignore la casse et les crochets ; les objets système et les tables temporaires sont écartés.
6. Ce qui reste sans correspondance devient une **référence introuvable** (section 8.4).

EXEMPLE **Les dépendances inter-serveurs de la démonstration** duckdb

```duckdb
select a.serveur as de_serveur, a.nom as de_objet, d.nature, d.vers_nom
from dependances d join objets a on a.id = d.de_id
where d.interserveur order by 1, 2
```

▶ CE QUE PRODUIT ATLAS

de_serveur	de_objet	nature	vers_nom
SRV-DWH	SP_CHARGER_DIM_CLIENT	SELECT	SRV-ERP.ERP.vente.CLIENT
SRV-ERP	SP_EMISSION_FACTURE	INSERT	SRV-COMPTA.COMPTA.dbo.ECRITURE
SRV-ERP	SP_EXPORT_DWH	INSERT	SRV-DWH.DWH.stg.STG_FACTURE
SRV-ERP	SP_EXPORT_DWH	EXEC	SRV-DWH.DWH.dw.SP_CHARGER_FAIT_VENTE
SRV-ERP	SP_SYNCHRO_TIERS	SELECT	SRV-COMPTA.COMPTA.dbo.ECRITURE
SRV-ERP	SP_SYNCHRO_TIERS	OPENQUERY	LNK_COMPTA
SRV-ERP	SYN_ECRITURE	SYNONYME	SRV-COMPTA.COMPTA.dbo.ECRITURE

Inter-base et inter-serveur

Chaque arête porte deux indicateurs : `interbase` (la cible est dans une autre base du même serveur) et `interserveur` (elle est sur un autre serveur). Ce sont ces liens qui rendent une migration risquée : la carte les dessine en pointillés (chapitre 9).

8.3 Fréquence, volumes et complexité

Quand les statistiques d'exécution sont disponibles, chaque arête est enrichie : **fréquence** (nombre d'exécutions de l'objet appelant), **volume moyen** et **volume maximal** (lignes traitées par exécution), et **complexité** de l'appelant. C'est ce qui permet de distinguer une dépendance théorique d'une dépendance qui fait passer des millions de lignes chaque nuit.

EXEMPLE **Qui écrit dans FACTURE, et à quelle fréquence ?** duckdb

```duckdb
select o.sch || '.' || o.nom as objet, d.nature, d.frequence
from dependances d join objets o on o.id = d.de_id join objets t on t.id = d.vers_id
where t.nom = 'FACTURE' and d.nature in ('INSERT', 'UPDATE', 'DELETE', 'MERGE') order by 1, 2
```

▶ CE QUE PRODUIT ATLAS

objet	nature	frequence
vente.SP_EMISSION_FACTURE	DELETE	365
vente.SP_EMISSION_FACTURE	INSERT	365

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

8.4 Les références introuvables

EXEMPLE Ce que la résolution n'a pas trouvé — duckdb

```
1  select o.nom as objet, d.vers_nom, d.nature
2  from dependances d join objets o on o.id = d.de_id where d.vers_id is null
```

▶ CE QUE PRODUIT ATLAS

objet	vers_nom	nature
SP_PAIE_MENSUELLE	SRV-ERP.COMPTA.dbo.EXERCICE	SELECT
SP_PAIE_MENSUELLE	SRV-ERP.COMPTA.dbo.ECRITURE	INSERT

Le cas de la démonstration est instructif : `rh.SP_PAIE_MENSUELLE` est sur SRV-ERP et écrit dans `COMPTA.dbo.ECRITURE`. Mais la base COMPTA est sur SRV-COMPTA : sur SRV-ERP, ce nom à trois parties ne désigne rien. Soit la procédure ne fonctionne plus, soit elle est exécutée ailleurs. Atlas lève un constat `REF_INTROUVABLE`.

Cause fréquente	Que faire
objet supprimé ou renommé	code mort à corriger ou à supprimer
base ou serveur non analysé	ajouter la source, ou la base dans le champ « Bases »
nom construit dynamiquement (`{@Table}`)	voir les valeurs observées (chapitre 13)
mauvais serveur (comme ci-dessus)	utiliser un serveur lié ou un synonyme

8.5 L'onglet Dépendances

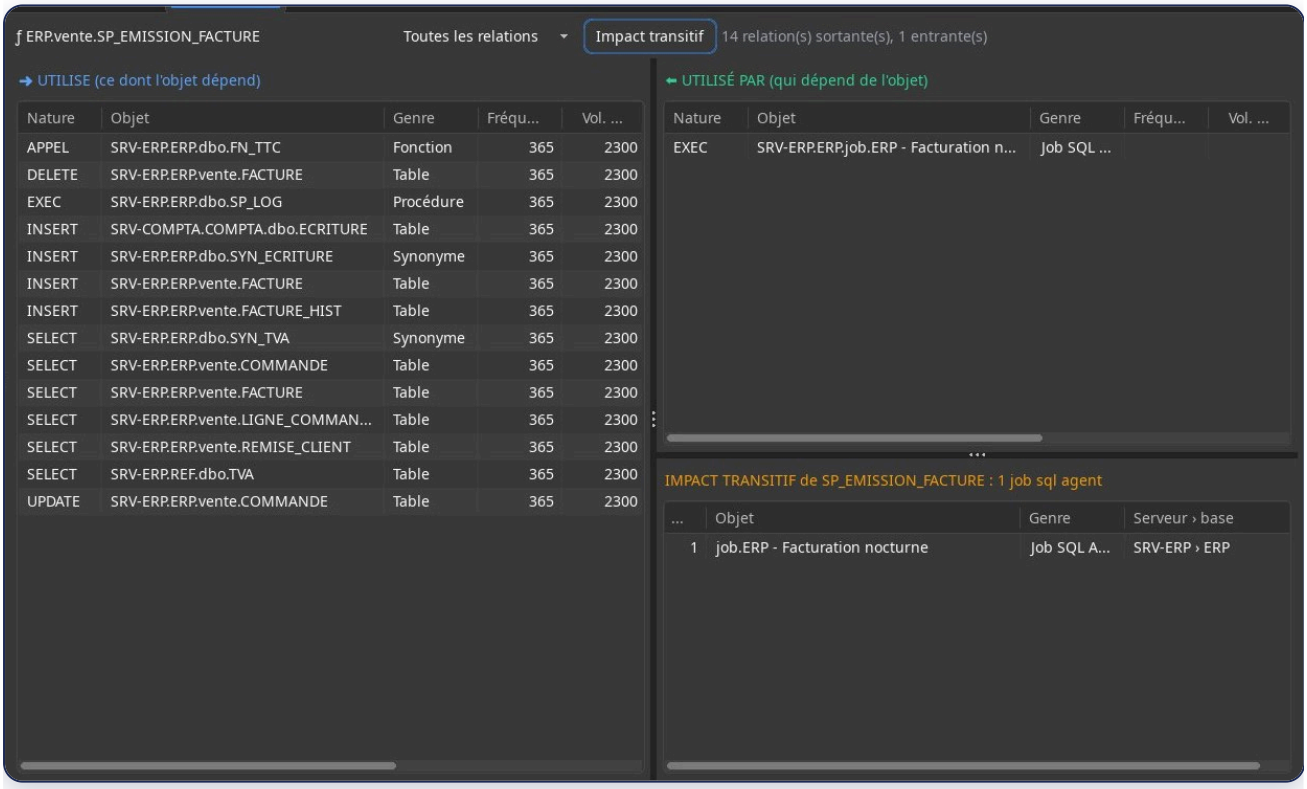


Figure 10 — L'onglet Dépendances de SP_EMISSION_FACTURE : à gauche ce qu'elle utilise (14 relations), à droite qui l'utilise (le job de facturation nocturne), en bas l'impact transitif.

La liste déroulante pose les questions classiques ; chacune filtre les natures :

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Question	Natures retenues
Qui lit ?	SELECT
Qui écrit ?	INSERT, UPDATE, DELETE, MERGE
Qui supprime ?	DELETE
Qui met à jour ?	UPDATE, MERGE
Qui insère ?	INSERT, MERGE
Qui appelle ?	EXEC, APPEL, OPENQUERY

Les grilles donnent pour chaque relation : nature, objet, genre, fréquence, volumes, complexité, portée (locale, bases, serveurs), origine et ligne de code. Un double-clic rend l'objet courant.

8.6 L'analyse d'impact

Impact (barre d'outils ou lien de la fiche) répond à : *« si je modifie cet objet, qu'est-ce qui peut casser ? »*. Atlas remonte le graphe « qui dépend de moi » jusqu'à six niveaux, sans boucler, et liste chaque objet touché avec son niveau.

EXEMPLE **Impact d'une modification de la table CLIENT** duckdb

```duckdb
with recursive imp(id, niveau) as (
  select id, 0 from objets where nom = 'CLIENT' and genre = 'TABLE'
  union select d.de_id, i.niveau + 1 from dependances d join imp i on d.vers_id = i.id where i.niveau
< 3)
select o.sch || '.' || o.nom as objet, o.genre, min(niveau) as niveau
from imp join objets o on o.id = imp.id group by all order by niveau, objet limit 12
```

▶ CE QUE PRODUIT ATLAS

objet	genre	niveau
vente.CLIENT	TABLE	0
dbo.V_CLIENT_COMPLET	VUE	1
dw.SP_CHARGER_DIM_CLIENT	PROCEDURE	1
vente.COMMANDE	TABLE	1
vente.FN_CA_CLIENT	FONCTION	1
vente.SP_CA_CLIENT	PROCEDURE	1
vente.SP_CA_MENSUEL	PROCEDURE	1
vente.SP_CLIENT_AJOUTER	PROCEDURE	1
vente.SP_CLIENT_FICHE	PROCEDURE	1
vente.SP_CLIENT_LIRE	PROCEDURE	1
vente.SP_CLIENT_LISTE	PROCEDURE	1
vente.SP_CLIENT_MODIFIER	PROCEDURE	1

L'impact d'une colonne

Pour une **colonne**, Atlas cherche aussi son nom dans le texte de tous les modules. L'assistant le fait en une question :

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

QUESTION **Demander l'impact d'une colonne à l'assistant** Assistant

```
1  impact de CLIENT_CODE
```

▶ RÉPONSE DE L'ASSISTANT

```
**Impact d'une modification de la colonne CLIENT_CODE**

Présente dans **1** table(s) ou vue(s) :
- SRV-ERP › ERP.vente.COMMANDE — `varchar(10)`

Citée dans le code de **6** objet(s) :
- 🔒 vente.FK_COMMANDE_CLIENT_CODE (1 occurrence(s))
- ƒ vente.SP_CALCUL_REMISE (1 occurrence(s))
- ƒ vente.SP_COMMANDE_AJOUTER (1 occurrence(s))
- ƒ vente.SP_COMMANDE_LIRE (1 occurrence(s))
- ƒ vente.SP_COMMANDE_MODIFIER (1 occurrence(s))
- ƒ vente.SP_EMISSION_FACTURE (4 occurrence(s))

Aperçu ligne par ligne et script de renommage : onglet *Refacto*, opération « Renommer ».
```

À retenir

L'impact transitif se fonde sur ce qu'Atlas a vu : le code des bases analysées. Les applications clientes, les rapports et les ETL externes ne sont pas dans le graphe — les appels capturés (chapitre 19) permettent d'en découvrir une partie.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 9 · LA CARTOGRAPHIE

Du serveur à la colonne, sur une seule carte

La cartographie à cinq niveaux : vue globale, serveur, base, schéma et objet ; légende des couleurs et des liens, navigation, options et fenêtre détachée.



9.1 Cinq niveaux de lecture · 9.2 Lire une carte · 9.3 Naviguer et régler

Dans ce chapitre, vous apprendrez à...

- ✓ choisir le bon niveau de carte
- ✓ lire couleurs, pointillés et épaisseurs
- ✓ naviguer à la souris et avec les boutons
- ✓ recentrer la carte sur un objet
- ✓ filtrer tables, vues et code
- ✓ ouvrir une carte sur un second écran

9.1 Cinq niveaux de lecture

L'onglet **Carte** dessine le patrimoine à cinq niveaux de zoom « métier ». La liste déroulante en haut à gauche choisit le niveau ; un double-clic sur un nœud descend d'un niveau, le fil d'Ariane remonte.

Vue	Nœuds	Liens
globale	les serveurs, avec leurs bases et leurs jobs	dépendances inter-serveurs (orange) et serveurs liés (pointillés)
serveur	les bases d'un serveur	dépendances inter-bases, avec leur nombre et leurs natures
base	les schémas d'une base, et les bases/serveurs qu'elle touche	dépendances entre schémas
schéma	les objets d'un schéma (tables avec leurs colonnes)	toutes les dépendances entre ces objets
objet	un objet et son voisinage (1 ou 2 sauts)	colorés par nature, épaisseur selon la fréquence

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

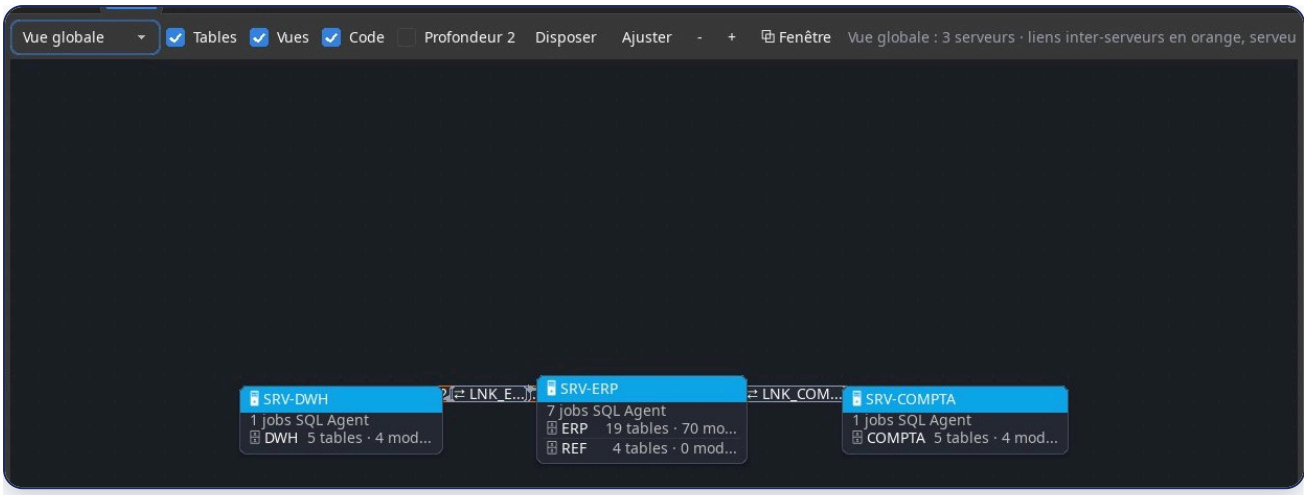


Figure 11 — Vue globale : les trois serveurs, leurs bases et leurs jobs ; les serveurs liés relient SRV-DWH, SRV-ERP et SRV-COMPTA.

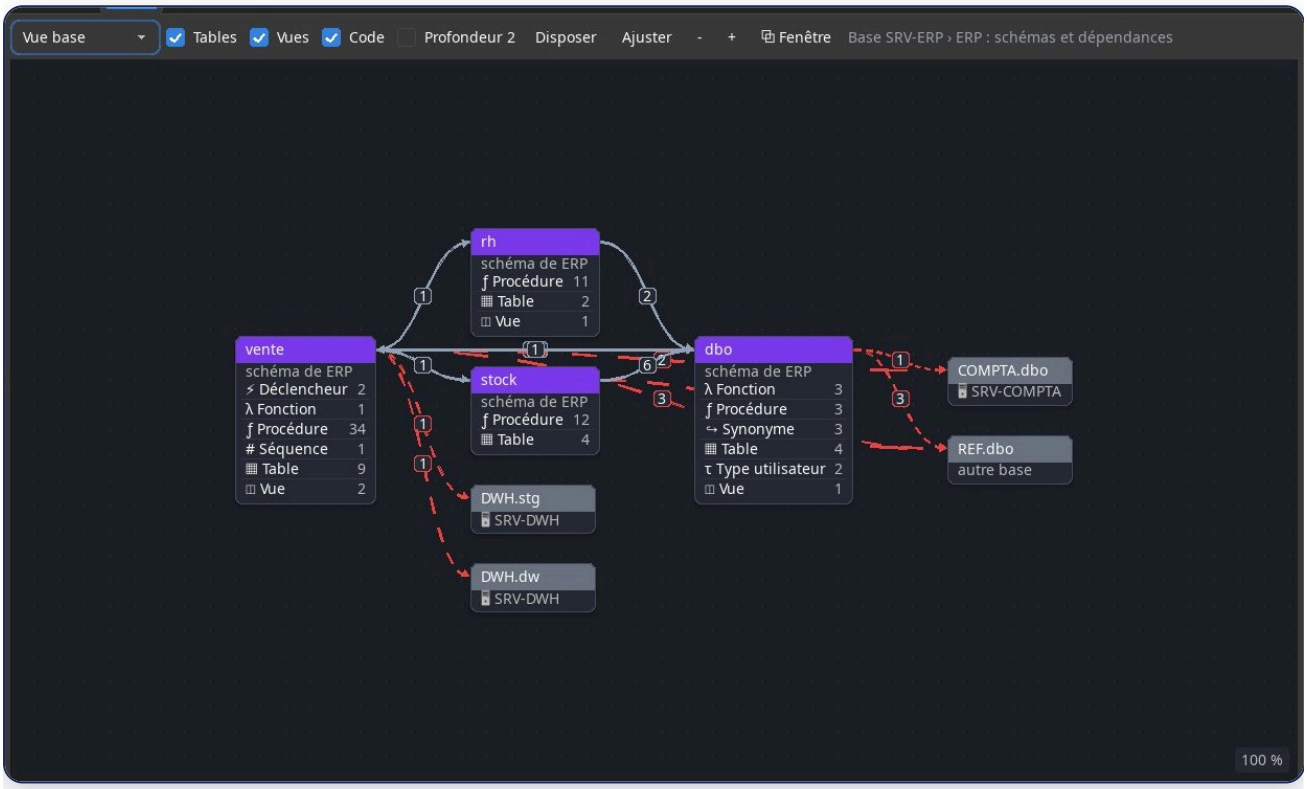


Figure 12 — Vue base de ERP : les schémas vente, stock, rh et dbo, et les liens vers les bases DWH, COMPTA et REF (pointillés : autre base ou autre serveur).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

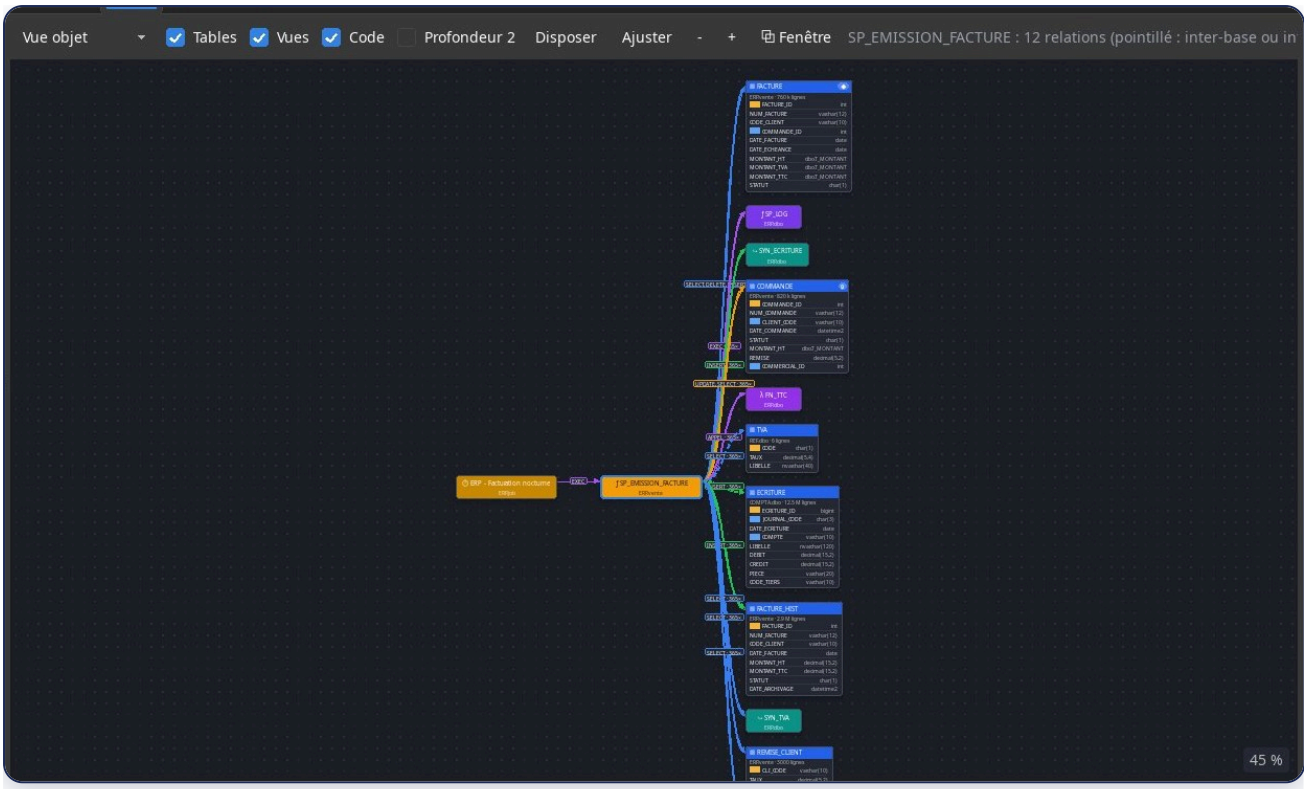


Figure 13 — Vue objet de SP_EMISSION_FACTURE : la procédure au centre, le job qui l'appelle à gauche, les tables qu'elle lit et écrit à droite.

9.2 Lire une carte

Couleur du nœud	Genre
bleu	table
cyan	vue
violet	procédure
pourpre	fonction
rose	déclencheur
vert d'eau	synonyme
ocre	job SQL Agent
orange	serveur lié

Couleur du lien	Nature
bleu	lecture (SELECT)
vert	insertion, MERGE
orange	mise à jour
rouge	suppression
violet	appel (EXEC, fonction)
gris	clé étrangère
pointillé	autre base ou autre serveur
épaisseur	fréquence d'exécution

Le libellé d'un lien de synthèse indique le nombre de dépendances et leurs natures (« 6 · SELECT, INSERT »). Un résumé en haut de la carte rappelle la légende du niveau affiché.

9.3 Naviguer et régler

Geste ou bouton	Effet
molette	zoom (le pourcentage s'affiche en bas à droite)
glisser le fond	déplacer la carte
clic sur un nœud	l'objet devient l'objet courant (fiche, fil d'Ariane) sans recentrer la carte

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Geste ou bouton	Effet
double-clic	descendre d'un niveau (serveur → base → schéma → objet) ; sur un objet : sa vue objet
⌖ Recentrer	recentrer sur l'objet courant
Disposer	disposition automatique en couches
Ajuster	tout le graphe dans la vue
− +	zoom arrière / avant
⧉ Fenêtre	ouvre la carte affichée dans une nouvelle fenêtre (second écran)

Option	Effet
Tables, Vues, Code	affiche ou masque les tables, les vues et les modules de code (procédures, fonctions…)
Profondeur 2	vue objet : voisinage à deux sauts au lieu d'un

Astuce

Sur une base de plusieurs milliers d'objets, commencez par la vue **base** puis descendez par double-clic : chaque niveau reste lisible. Décochez **Code** pour ne voir que le modèle de données et ses clés étrangères.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 10 · LE DICTIONNAIRE

Un dictionnaire qui s'écrit tout seul

Comment Atlas décrit tables et colonnes à partir de leurs noms, détecte les données personnelles, et comment l'enrichir avec votre propre glossaire.



10.1 Des noms techniques aux mots métier
10.2 Objet métier, famille et nature
10.3 Sensibilité et RGPD
10.4 Votre glossaire
10.5 L'onglet Dictionnaire et l'export

Dans ce chapitre, vous apprendrez à...

- ✓ comprendre le découpage des noms en mots
- ✓ lire objet métier, famille et nature d'une table
- ✓ repérer les données personnelles et leur sensibilité
- ✓ écrire un glossaire.json pour votre vocabulaire
- ✓ filtrer le dictionnaire
- ✓ exporter une documentation Markdown

10.1 Des noms techniques aux mots métier

Le **dictionnaire intelligent** documente chaque table et chaque colonne **sans que vous ayez rien à saisir**. Il s'appuie sur les noms, les types, les clés, le profilage éventuel et un **glossaire** de mots et d'abréviations. Première étape : découper un nom en mots.

Nom	Mots reconnus	Description générée	Type métier
`CODE_CLIENT`	CODE · CLIENT	Code du client	`TYPE_CODE_CLIENT`
`CLI_CODE`	CLIENT · CODE	Code du client	`TYPE_CODE_CLIENT`
`CODCLI`	CODE · CLIENT	Code du client	`TYPE_CODE_CLIENT`
`NUM_FACTURE`	NUMERO · FACTURE	Numéro de la facture	`TYPE_NUMERO_FACTURE`
`DATE_ECHEANCE`	DATE · ECHEANCE	Date d'échéance	`TYPE_DATE_ECHEANCE`
`MT_TTC`	MONTANT · TTC	Montant toutes taxes comprises	`TYPE_MONTANT_TTC`
`PRSP_CODE`	PRSP · CODE	Prsp code	`TYPE_CODE_PRSP`
`DT_RELANCE`	DATE · RELANCE	Date relance	`TYPE_DATE_RELANCE`

Le découpage gère les séparateurs (`_` , `-`), la casse mixte (`CodeClient`) et les **abréviations** (`COD` → CODE, `CLI` → CLIENT, `MT` → MONTANT, `DT` → DATE, `NUM` → NUMERO...). Les deux derniers noms (`PRSP_CODE` , `DT_RELANCE`) sont moins bien décrits : `PRSP` et `RELANCE` ne sont pas dans le glossaire. La section 10.4 montre comment l'enrichir.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

10.2 Objet métier, famille et nature

Pour chaque table, Atlas déduit :

- l'**objet métier** : le premier mot du nom reconnu par le glossaire (`LIGNE_COMMANDE` → Commande ; `CLIENT_OLD` → Client) ;
- la **famille** : Commercial, Finance, Logistique, Ressources humaines, Comptabilité, Référentiel, Décisionnel, Technique... (base REF : Référentiel ; schémas dw / stg : Décisionnel) ;
- la **nature**, d'après le nom, le volume et l'usage :

Nature	Reconnue à
Historique	`…HIST…` , `…ARCH…` , `…_OLD`
Journal technique	`…LOG…` , `…AUDIT…` , `…TRACE…`
Transit / technique	`TMP…` , `STG…` , `…IMPORT…` , schéma `stg`
Dimension / Fait (décisionnel)	`DIM_…` , `FAIT_…` , `FACT_…`
Référentiel	famille Référentiel, petite table sans date, `…PARAM…` , jamais écrite
Vue	les vues
Transactionnel	tous les autres cas

Chaque colonne reçoit une description (`FACTURE_ID` → *Identifiant de la facture (identifiant, clé primaire), numéroté automatiquement*), son type métier, sa sensibilité et, si le profilage est actif, sa **cardinalité** (constante, faible, élevée, unique).

10.3 Sensibilité et RGPD

Une colonne est une **donnée personnelle** quand son nom contient l'un des motifs ci-dessous (ou quand ses valeurs observées ressemblent à une adresse électronique ou à un IBAN français). La sensibilité d'une table est la plus forte de ses colonnes.

Motif	Catégorie RGPD	Sensibilité
`NIR` , `SECU`	identifiant national	**très élevée**
`CARTE`	données bancaires	**très élevée**
`IBAN` , `BIC`	données bancaires	élevée
`SALAIRE`	données financières personnelles	élevée
`MOTIF`	donnée potentiellement de santé	élevée
`NAISSANCE` , `PRENOM`	identité	moyenne
`EMAIL` , `MAIL` , `TEL` , `TELEPHONE` , `ADRESSE`	coordonnées	moyenne
`NOM` , `MATRICULE`	identité, identifiant interne	faible
`CODE_POSTAL` , `VILLE`	localisation	faible

Le NOM d'un article n'est pas une donnée personnelle

Un motif seul ne suffit pas : la colonne `NOM` d'une table de la famille Référentiel, Logistique, Technique ou Comptabilité (un article, un entrepôt, un pays) n'est pas retenue. Seuls EMAIL et TEL y restent sensibles.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

QUESTION Les tables à données personnelles de la démonstration — Assistant

```
quelles tables contiennent des données personnelles ?
```

▶ RÉPONSE DE L'ASSISTANT

```
**Données personnelles** (dictionnaire) :

- **SRV-ERP › ERP.rh.EMPLOYE** — très élevée : MATRICULE (Identifiant interne de personne), NOM (Identité),
PRENOM (Identité), NIR (Identifiant national (n° de sécurité sociale)), DATE_NAISSANCE (Identité), EMAIL
(Coordonnées), TELEPHONE (Coordonnées), SALAIRE_BRUT (Données financières personnelles)
- **SRV-COMPTA › COMPTA.dbo.TIERS** — élevée : IBAN (Données bancaires), EMAIL (Coordonnées)
- **SRV-ERP › ERP.rh.CONGE** — élevée : MATRICULE (Identifiant interne de personne), MOTIF (Donnée
potentiellement de santé)
- **SRV-ERP › ERP.vente.CLIENT** — élevée : NOM (Identité), PRENOM (Identité), EMAIL (Coordonnées),
TELEPHONE (Coordonnées), DATE_NAISSANCE (Identité), IBAN (Données bancaires), ADRESSE (Coordonnées),
CODE_POSTAL (Localisation), VILLE (Localisation)
- **SRV-ERP › ERP.vente.REGLEMENT** — élevée : IBAN_EMETTEUR (Données bancaires)
- **SRV-DWH › DWH.dw.DIM_CLIENT** — faible : NOM (Identité), VILLE (Localisation)
- **SRV-ERP › ERP.dbo.CLIENT_OLD** — moyenne : MAIL (Coordonnées), TELEPHONE (Coordonnées)
- **SRV-ERP › ERP.dbo.TMP_IMPORT_CLIENT** — moyenne : NOM (Identité), ADRESSE (Coordonnées), TEL
(Coordonnées)
- **SRV-ERP › ERP.vente.CONTACT** — moyenne : NOM (Identité), EMAIL (Coordonnées), TEL (Coordonnées)
- **SRV-ERP › ERP.vente.STG_CLIENT** — moyenne : NOM (Identité), PRENOM (Identité), EMAIL (Coordonnées),
VILLE (Localisation)

Copies hors production (DWH, exports) : vérifier la pseudonymisation ; registre des traitements à jour.
```

Les tables de sensibilité élevée ou très élevée reçoivent un constat `RGPD_SENSIBLE` (critique pour « très élevée ») qui rappelle les obligations : registre des traitements, durée de conservation, droits d'accès, pseudonymisation des copies (entrepôt décisionnel, recette).

10.4 Votre glossaire

Le glossaire intégré connaît le vocabulaire courant (client, commande, facture, article, stock, employé, écriture, compte…). Pour le vôtre, déposez un fichier `glossaire.json` dans le dossier du projet, puis relancez l'analyse : seul le travail *dictionnaire* (et ce qui en dépend) apporte du nouveau.

EXEMPLE Ajouter deux mots et une abréviation — glossaire.json

```
{"mots": {"PROSPECT": ["Prospect", "Commercial"],
          "RELANCE":  ["Relance",  "Commercial"]},
 "abreviations": {"PRSP": "PROSPECT"}}
```

▶ CE QUE PRODUIT ATLAS AVEC CE GLOSSAIRE

Nom	Mots reconnus	Description générée	Type métier
`PRSP_CODE`	PROSPECT · CODE	**Code du prospect**	`TYPE_CODE_PROSPECT`
`DT_RELANCE`	DATE · RELANCE	**Date de la relance**	`TYPE_DATE_RELANCE`

Chaque mot associe un **libellé** et une **famille** ; chaque abréviation, le mot qu'elle remplace. Comparez avec la section 10.1 : `PRSP_CODE` devient « Code du prospect » et rejoint le type `TYPE_CODE_PROSPECT` ; `DT_RELANCE` devient « Date de la relance ».

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

> **Astuce**
> Les articles sont accordés : « du client », « de la facture », « de l'écriture ». Le rôle passe devant le nom métier : `FACTURE_ID` → « Identifiant de la facture ».

10.5 L'onglet Dictionnaire et l'export

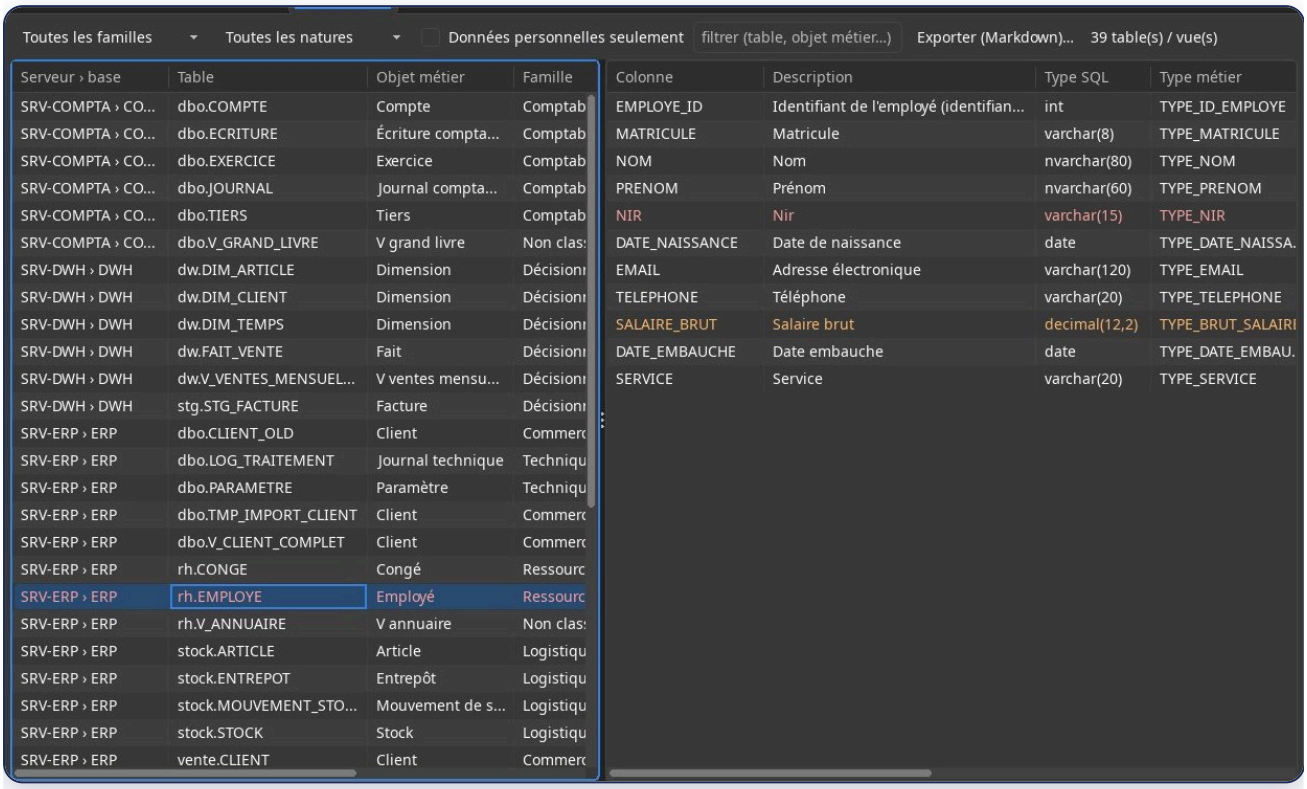


Figure 14 — L'onglet Dictionnaire : à gauche les tables (rh.EMPLOYE, sensible, en rouge), à droite ses colonnes avec description, type SQL, type métier et sensibilité.

- Filtres : famille, nature, **Données personnelles seulement**, texte libre (table, objet métier...).
- Les lignes sensibles sont colorées (rouge pour très élevée, orange pour élevée).
- `Exporter (Markdown)...` écrit un document : une section par table, avec un tableau des colonnes. Idéal pour un wiki ou un dossier RGPD.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 11 · LES TYPES MÉTIER

Une donnée, un seul type

Regrouper les colonnes qui désignent la même donnée sous des noms différents, et repérer les types SQL incohérents qui ralentissent les jointures.

11.1 Une donnée, plusieurs noms

11.2 Le catalogue des types métier

11.3 Incohérences de types et de nommage

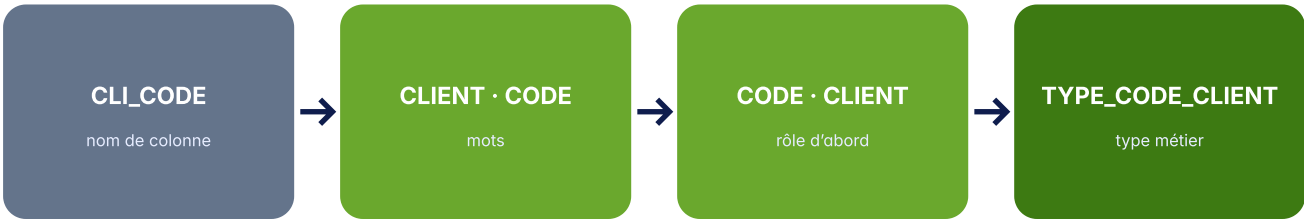


Dans ce chapitre, vous apprendrez à...

- ✓ comprendre la clé canonique d'un type métier
- ✓ lire le catalogue des types
- ✓ repérer une incohérence de type ou de nommage
- ✓ mesurer le risque des conversions implicites
- ✓ générer le script des types utilisateur
- ✓ préparer l'alignement des colonnes

11.1 Une donnée, plusieurs noms

Dans un patrimoine ancien, la même donnée porte des noms différents selon les tables et les époques : `CODE_CLIENT`, `CLI_CODE`, `CODCLI`, `CODE_CLI`, `CLIENT_CODE`... Atlas les regroupe en un **type métier** en construisant une clé canonique : les mots du nom (abréviations développées), **le rôle d'abord** (CODE, ID, DATE, MONTANT, LIBELLE...), puis les autres mots par ordre alphabétique.



© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

11.2 Le catalogue des types métier

EXEMPLE Les types métier les plus répandus — duckdb

```
select code, libelle, type_sql, occurrences, colonnes, variantes from types_metier order by occurrences desc limit 6
```

▶ CE QUE PRODUIT ATLAS

code	libelle	type_sql	occurrences	colonnes	variantes
TYPE_CODE_CLIENT	Code du client	varchar(10)	11	CLIENT_CODE, CLI_CODE, CODCLI, CODE_CLI, CODE_CLIENT	dbo.T_CODE_CLIENT, varchar(10)
TYPE_LIBELLE	Libellé	nvarchar(120)	9	LIBELLE	nvarchar(120), nvarchar(40), nvarchar(60), nvarchar(80)
TYPE_NOM	Nom	nvarchar(80)	6	NOM	nvarchar(80)
TYPE_EMAIL	Adresse électronique	varchar(120)	6	EMAIL, MAIL	varchar(120)
TYPE_MONTANT_HT	Montant hors taxes	decimal(15,2)	5	MONTANT_HT	dbo.T_MONTANT, decimal(15,2)
TYPE_CODE_ARTICLE	Code de l'article	varchar(15)	5	ARTICLE_CODE, CODE_ARTICLE	varchar(15)

`occurrences` compte les colonnes du type ; `colonnes` liste leurs noms ; `variantes` liste les types SQL rencontrés. Une virgule dans `variantes` signale une **incohérence**.

11.3 Incohérences de types et de nommage

Deux constats en découlent :

- `TYPE_INCOHERENT` (important) : la même donnée a plusieurs types SQL (`dbo.T_CODE_CLIENT` et `varchar(10)`). Les jointures entre types différents provoquent des **conversions implicites** qui empêchent l'usage des index.
- `NOMMAGE_INCOHERENT` (opportunité) : plusieurs noms pour la même donnée ; ils masquent les dépendances et compliquent la maintenance.

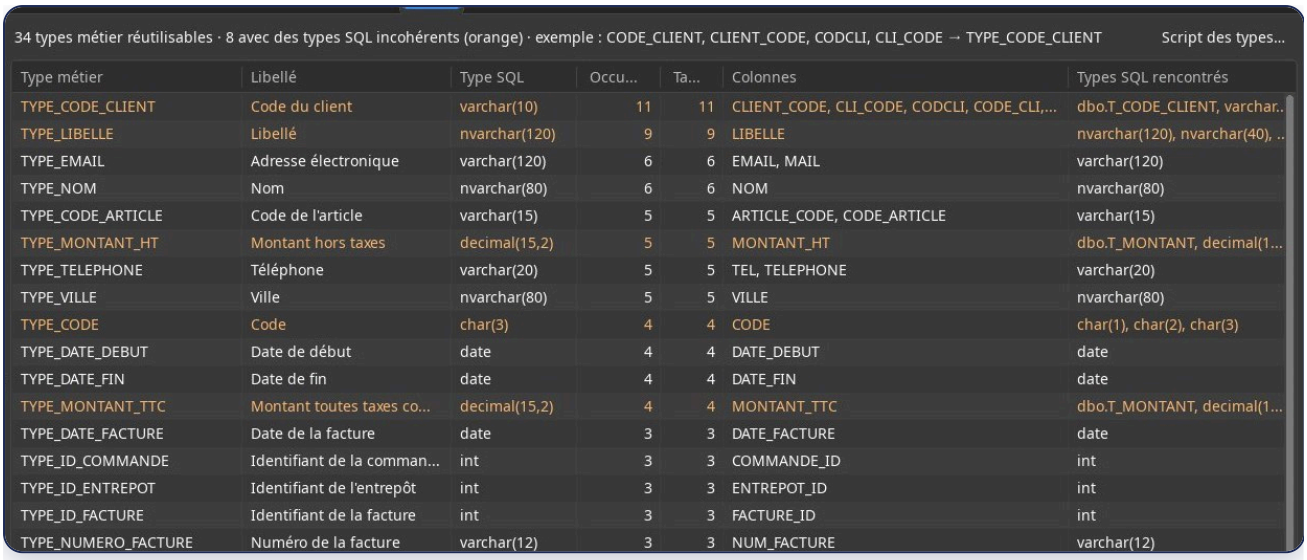
SCRIPT La correction suggérée par Atlas pour TYPE_CODE_CLIENT — Script des types

```
-- Créer un type utilisateur et aligner les colonnes
CREATE TYPE TYPE_CODE_CLIENT FROM varchar(10);
-- puis, table par table :
ALTER TABLE vente.COMMANDE ALTER COLUMN CLIENT_CODE TYPE_CODE_CLIENT NOT NULL;
```

Attention

Modifier le type d'une colonne indexée ou référencée par une clé étrangère demande de supprimer puis recréer ces index et contraintes. Utilisez l'onglet **Refacto** et l'analyse d'impact (chapitres 8 et 18) avant tout script.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



Type métier	Libellé	Type SQL	Occu...	Ta...	Colonnes	Types SQL rencontrés
TYPE_CODE_CLIENT	Code du client	varchar(10)	11	11	CLIENT_CODE, CLI_CODE, CODCLI, CODE_CLI,...	dbo.T_CODE_CLIENT, varchar..
TYPE_LIBELLE	Libellé	nvarchar(120)	9	9	LIBELLE	nvarchar(120), nvarchar(40), ..
TYPE_EMAIL	Adresse électronique	varchar(120)	6	6	EMAIL, MAIL	varchar(120)
TYPE_NOM	Nom	nvarchar(80)	6	6	NOM	nvarchar(80)
TYPE_CODE_ARTICLE	Code de l'article	varchar(15)	5	5	ARTICLE_CODE, CODE_ARTICLE	varchar(15)
TYPE_MONTANT_HT	Montant hors taxes	decimal(15,2)	5	5	MONTANT_HT	dbo.T_MONTANT, decimal(1...
TYPE_TELEPHONE	Téléphone	varchar(20)	5	5	TEL, TELEPHONE	varchar(20)
TYPE_VILLE	Ville	nvarchar(80)	5	5	VILLE	nvarchar(80)
TYPE_CODE	Code	char(3)	4	4	CODE	char(1), char(2), char(3)
TYPE_DATE_DEBUT	Date de début	date	4	4	DATE_DEBUT	date
TYPE_DATE_FIN	Date de fin	date	4	4	DATE_FIN	date
TYPE_MONTANT_TTC	Montant toutes taxes co...	decimal(15,2)	4	4	MONTANT_TTC	dbo.T_MONTANT, decimal(1...
TYPE_DATE_FACTURE	Date de la facture	date	3	3	DATE_FACTURE	date
TYPE_ID_COMMANDE	Identifiant de la comman...	int	3	3	COMMANDE_ID	int
TYPE_ID_ENTREPOT	Identifiant de l'entrepôt	int	3	3	ENTREPOT_ID	int
TYPE_ID_FACTURE	Identifiant de la facture	int	3	3	FACTURE_ID	int
TYPE_NUMERO_FACTURE	Numéro de la facture	varchar(12)	3	3	NUM_FACTURE	varchar(12)

Figure 15 — L'onglet Types : les types métier, en orange ceux dont les types SQL divergent ; le bouton Script des types génère les CREATE TYPE.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 12 · LES FLUX DE DONNÉES

Suivre la donnée de table en table

Les flux table lue → traitement → table écrite, en vue flux, Sankey, pipeline et chronologie des jobs nocturnes.



12.1 Des dépendances aux flux

12.2 Quatre façons de voir les flux

12.3 La chronologie des jobs

Dans ce chapitre, vous apprendrez à...

- comprendre comment un flux est déduit du graphe
- lire un flux pondéré par les volumes
- choisir entre flux, Sankey et pipeline
- filtrer les flux sur une base
- placer les jobs SQL Agent sur une ligne de temps
- repérer chevauchements et créneaux libres

12.1 Des dépendances aux flux

Un **flux de données** est un chemin *table lue → traitement → table écrite*. Atlas le déduit du graphe : quand un même objet lit une table (SELECT) et en écrit une autre (INSERT, UPDATE, MERGE), il en fait un flux, pondéré par le **volume** (lignes par exécution × fréquence).

MOTEUR **Les plus gros flux de la démonstration** FLUX

```
1  -- fonction FLUX() de dependances.prg : lecture + écriture par le même objet
```

▶ CE QUE PRODUIT ATLAS

source	traitement	destination	nature	volume
ERP.stock.ARTICLE	stock.SP_INVENTAIRE	ERP.stock.STOCK	UPDATE	1898000000
ERP.stock.MOUVEMENT_STOCK	stock.SP_INVENTAIRE	ERP.stock.STOCK	UPDATE	1898000000
DWH.dw.DIM_TEMPS	dw.SP_CHARGER_FAIT_VENTE	DWH.dw.FAIT_VENTE	INSERT	3577000
DWH.stg.STG_FACTURE	dw.SP_CHARGER_FAIT_VENTE	DWH.dw.FAIT_VENTE	INSERT	3577000
DWH.dw.DIM_CLIENT	dw.SP_CHARGER_FAIT_VENTE	DWH.dw.FAIT_VENTE	INSERT	3577000
COMPTA.dbo.ECRITURE	vente.SP_SYNCHRO_TIERS	ERP.vente.CLIENT	UPDATE	2710198
ERP.dbo.PARAMETRE	vente.SP_EXPORT_DWH	DWH.stg.STG_FACTURE	INSERT	1585050
ERP.vente.FACTURE	vente.SP_EXPORT_DWH	DWH.stg.STG_FACTURE	INSERT	1585050

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

12.2 Quatre façons de voir les flux

Mode	Ce qu'il montre
Vue flux	tables et traitements en couches, de gauche (sources) à droite (cibles)
Vue Sankey (volumes)	même graphe, l'épaisseur des liens proportionnelle au volume
Vue pipeline	les traitements enchaînés comme une chaîne de production (staging → dimensions → faits)
Vue temporelle (jobs)	un Gantt de la nuit : chaque job SQL Agent à son heure de début, sur sa durée moyenne

La seconde liste filtre sur une base. Comme dans la carte, un clic sur un nœud en fait l'objet courant.

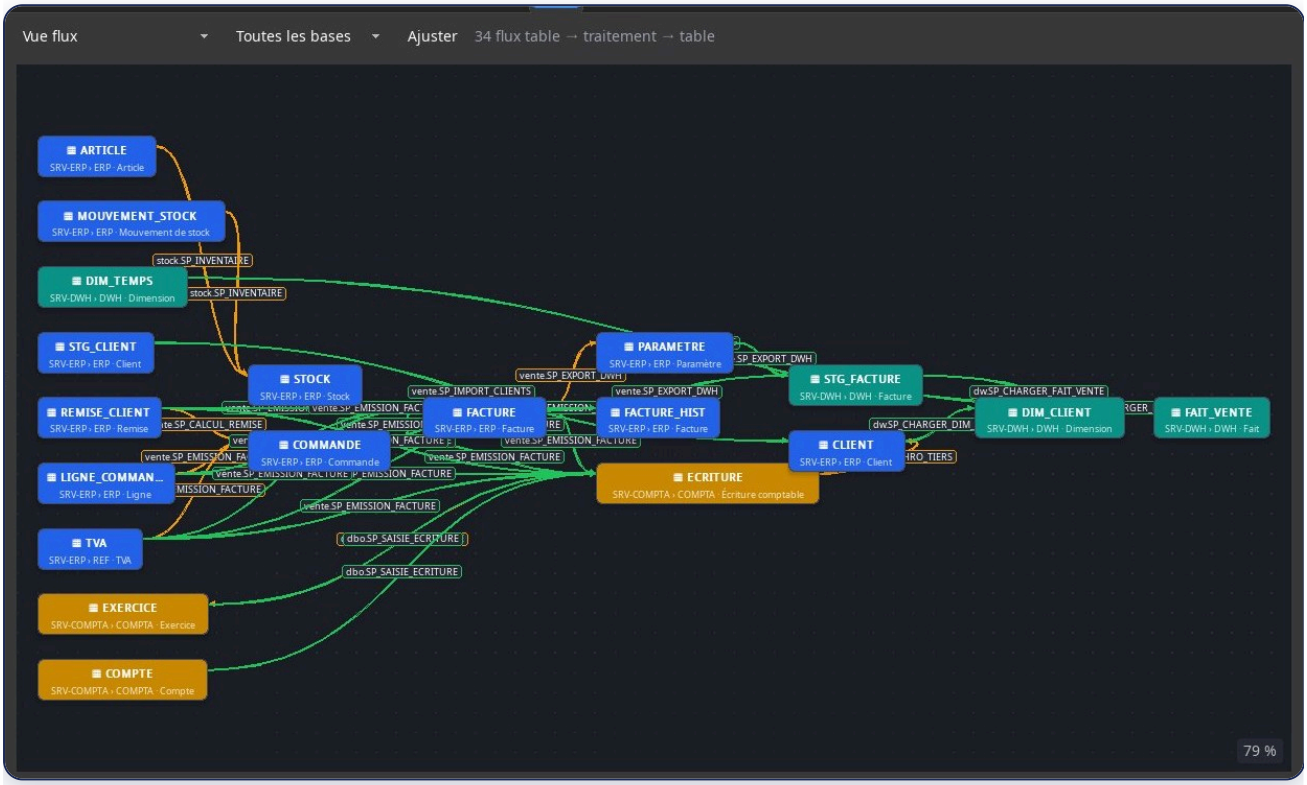


Figure 16 — La vue flux de la démonstration : ARTICLE et MOUVEMENT_STOCK alimentent STOCK ; les staging du DWH alimentent dimensions et faits.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

12.3 La chronologie des jobs

EXEMPLE Les jobs SQL Agent, leur fréquence et leur créneau — duckdb

```
1  select serveur, nom, cible, detail from objets where genre = 'JOB' order by 1, 2
```

▶ CE QUE PRODUIT ATLAS

serveur	job	frequence	detail
SRV-COMPTA	COMPTA - Balance	quotidien	début 06:00 · 420.0 s
SRV-DWH	DWH - Chargement	quotidien	début 03:20 · 2040.0 s
SRV-ERP	ERP - Export DWH	quotidien	début 03:00 · 900.0 s
SRV-ERP	ERP - Facturation nocturne	quotidien	début 01:30 · 1260.0 s
SRV-ERP	ERP - Import clients	quotidien	début 00:45 · 300.0 s
SRV-ERP	ERP - Inventaire	quotidien	début 02:15 · 1800.0 s
SRV-ERP	ERP - Nettoyage des journaux	hebdomadaire	début 05:00 · 600.0 s
SRV-ERP	ERP - Purge mensuelle	mensuel	début 04:00 · 2400.0 s
SRV-ERP	ERP - Relances	quotidien	début 08:00 · 240.0 s

La vue temporelle place ces jobs sur une ligne de temps : on voit immédiatement les chevauchements (ici, l'export DWH de 03:00 et le chargement du DWH de 03:20 dépendent l'un de l'autre) et les fenêtres libres pour de nouveaux traitements.

À vous de jouer

Dans la vue temporelle, trouvez le job le plus long. Puis ouvrez sa vue objet dans la carte : quelles procédures appelle-t-il ?

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 13 · LE SQL DYNAMIQUE

Reconstituer le SQL invisible

Suivre les variables qui construisent une requête, reconstituer le SQL exécuté, détecter l'injection, analyser OPENQUERY et confronter le tout aux valeurs observées.



13.1 Pourquoi le SQL dynamique est difficile
13.2 La reconstitution
13.3 EXEC (@sql) et l'injection
13.4 OPENQUERY : le SQL envoyé à un autre serveur
13.5 Variantes et valeurs observées

Dans ce chapitre, vous apprendrez à...

- comprendre pourquoi le SQL dynamique échappe aux outils
- lire un SQL reconstitué et ses accolades
- distinguer EXEC (@sql) et sp_executesql
- corriger un risque d'injection
- suivre une requête OPENQUERY
- exploiter les variantes et valeurs observées

13.1 Pourquoi le SQL dynamique est difficile

Une procédure peut construire sa requête dans une variable puis l'exécuter. Pour un outil d'analyse ordinaire, cette requête n'existe pas : c'est une chaîne. Ses tables échappent au graphe, ses risques (injection) passent inaperçus. Atlas, lui, **suit les affectations** de la variable et **reconstitue** le SQL final.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

13.2 La reconstitution

EXEMPLE Une recherche avec filtre et tri optionnels — vente.SP_RECHERCHE

```
1  CREATE PROCEDURE vente.SP_RECHERCHE @Ville NVARCHAR(80) = NULL, @Tri SYSNAME = N'NOM'
2  AS
3  BEGIN
4    DECLARE @sql NVARCHAR(MAX) = N'SELECT CODE_CLIENT, NOM, VILLE FROM vente.CLIENT WHERE ACTIF = 1';
5    IF @Ville IS NOT NULL
6      SET @sql = @sql + N' AND VILLE = @v';
7    SET @sql = @sql + N' ORDER BY ' + QUOTENAME(@Tri);
8    EXEC sp_executesql @sql, N'@v NVARCHAR(80)', @v = @Ville;
9  END
```

▶ CE QUE PRODUIT ATLAS

```
ligne 8 · sp_executesql
SQL        : SELECT CODE_CLIENT, NOM, VILLE FROM vente.CLIENT WHERE ACTIF = 1 AND VILLE = @v ORDER BY
{QUOTENAME(@Tri)}
paramètres : @v NVARCHAR(80) ← @Ville

référence ajoutée au graphe : SELECT vente.CLIENT (ligne 8)
```

Atlas a suivi trois affectations de `@sql` (déclaration, `IF`, ajout du tri) et assemblé le texte final. Les morceaux qu'il ne peut pas connaître sont gardés entre accolades : `{QUOTENAME(@Tri)}` est une expression évaluée à l'exécution. Les paramètres de `sp_executesql` sont listés avec leur type et la variable qui les alimente (`@v NVARCHAR(80) ← @Ville`).

Les deux branches d'un IF

Le texte reconstitué contient le filtre `AND VILLE = @v` : Atlas assemble **toutes** les branches pour obtenir le SQL le plus complet. Les valeurs observées (section 13.5) montrent ensuite les variantes réellement exécutées.

13.3 EXEC (@sql) et l'injection

EXEMPLE Le cas dangereux (extrait de la section 7.2) — dbo.SP_MAUVAISES_PRATIQUES

```
13 SET @sql = N'SELECT * FROM vente.CLIENT WHERE NOM = ''' + @Nom + N'''';
14 EXEC (@sql);
```

▶ CE QUE PRODUIT ATLAS

```
SQL reconstitué : SELECT * FROM vente.CLIENT WHERE NOM = '{@Nom}'
constat INJECTION (critique) : SQL dynamique concaténé avec des variables : risque d'injection SQL
```

Si `@Nom` vaut `x' OR 1=1 --`, la requête renvoie toute la table ; s'il vaut `x'; DROP TABLE …`, pire encore. La version sûre passe la valeur en **paramètre** et protège les **noms d'objets** avec `QUOTENAME` :

APRÈS La version paramétrée — correction

```
1  DECLARE @sql NVARCHAR(MAX) = N'SELECT CODE_CLIENT, NOM FROM vente.CLIENT WHERE NOM = @n';
2  EXEC sp_executesql @sql, N'@n NVARCHAR(80)', @n = @Nom;
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

13.4 OPENQUERY : le SQL envoyé à un autre serveur

La requête passée à `OPENQUERY` est exécutée par le serveur lié. Atlas la conserve comme un SQL dynamique et l'analyse à son tour : ses tables deviennent des dépendances **distantes** (chapitre 8).

EXEMPLE Une lecture distante — `vente.SP_EXPORT_SOLDE`

```
6  SELECT * FROM OPENQUERY(LNK_DWH, 'SELECT CLIENT_SK, CODE_CLIENT FROM DWH.dw.DIM_CLIENT');
```

▶ CE QUE PRODUIT ATLAS

```
mode OPENQUERY · serveur lié LNK_DWH
SQL distant : SELECT CLIENT_SK, CODE_CLIENT FROM DWH.dw.DIM_CLIENT
```

13.5 Variantes et valeurs observées

Le SQL reconstitué est une forme générale. Les **appels capturés** (chapitre 19) donnent les valeurs réellement passées : Atlas les présente à côté du SQL, et la page Captures les substitue pour montrer la requête exécutée. L'onglet **Dynamique** rassemble le tout.

EXEMPLE Les SQL dynamiques de la démonstration — `duckdb`

```
1  select o.nom, v.sql, v.occurrences from dyn_variantes v join objets o on o.id = v.objet_id
```

▶ CE QUE PRODUIT ATLAS

objet	sql	n
SP_RECHERCHE_CLIENT	SELECT CLIENT_ID, CODE_CLIENT, NOM, PRENOM, VILLE FROM vente.CLIENT WHERE ACTIF = 1 AND NOM LIKE @pNom AND VILLE = @pVille ORDER BY {QUOTENAME(@Tri)}	1
SP_RAPPORT_DYNAMIQUE	SELECT CODE_CLIENT, SUM(MONTANT_TTC) AS TOTAL FROM vente.{@Table} WHERE YEAR(DATE_FACTURE) = {CAST(@Annee AS VARCHAR(4))} GROUP BY CODE_CLIENT	1
SP_SYNCHRO_TIERS	SELECT CODE_TIERS, SUM(DEBIT - CREDIT) AS SOLDE FROM COMPTA.dbo.ECRITURE GROUP BY CODE_TIERS	1

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

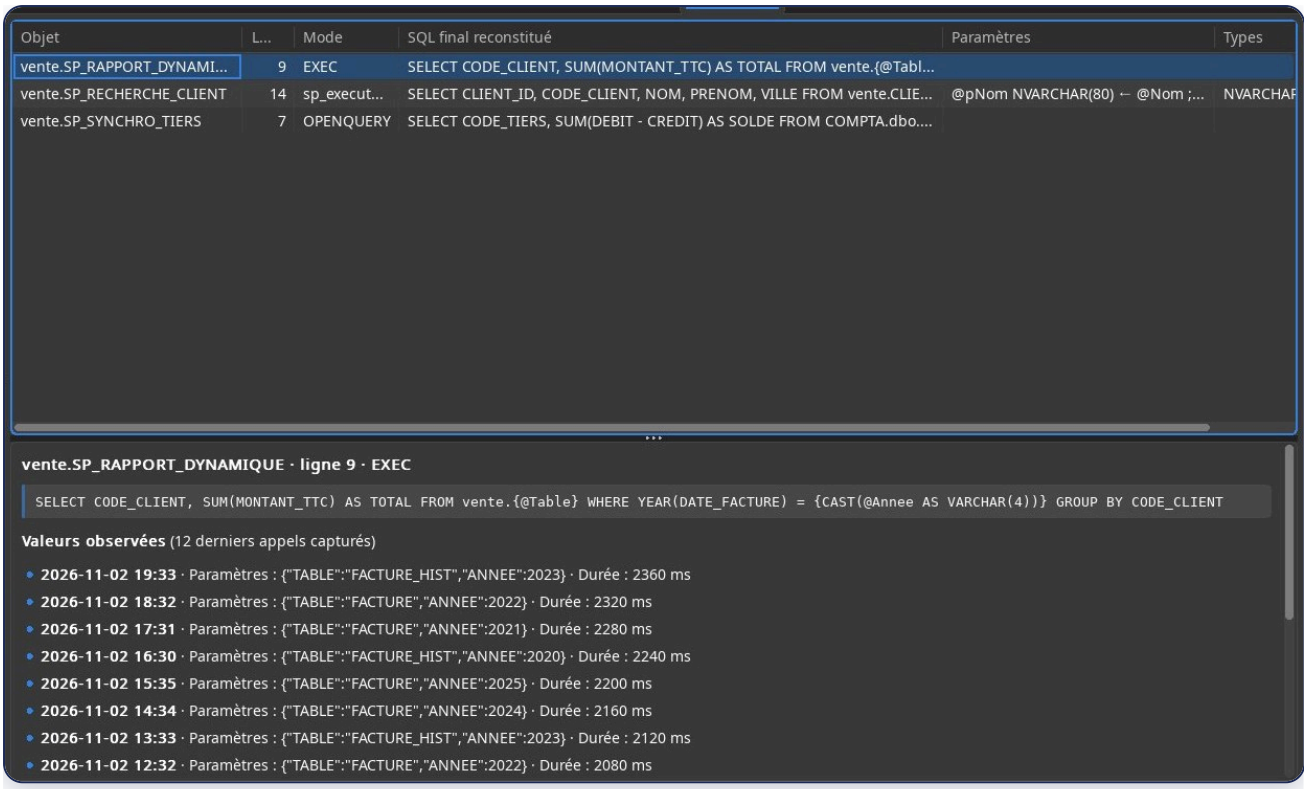


Figure 17 — L'onglet Dynamique : SP_RAPPORT_DYNAMIQUE construit le nom de sa table ({@Table}) ; les derniers appels capturés montrent les valeurs FACTURE et FACTURE_HIST.

À retenir

Les valeurs observées éclairent les noms construits : ici `vente.{@Table}` vaut `FACTURE` ou `FACTURE_HIST`. Le graphe, lui, ne peut pas relier un nom construit : aucune dépendance n'est créée pour `{@Table}`. C'est à vous de décider s'il faut rendre ce code statique.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 14 · LES PERFORMANCES

Trouver ce qui coûte vraiment

Les statistiques d'exécution et les plans en cache, les règles de performance, les index manquants transformés en script, et l'onglet Perfs.



14.1 D'où viennent les mesures
14.2 Les mesures par objet
14.3 Les règles de performance
14.4 Des index manquants au script
14.5 L'onglet Perfs

Dans ce chapitre, vous apprendrez à...

- savoir d'où viennent les mesures
- raisonner en coût × fréquence
- lire les six règles de performance
- comprendre le reclassement des constats
- transformer un index manquant en CREATE INDEX
- produire le script des corrections

14.1 D'où viennent les mesures

Avec l'option *Statistiques d'exécution et plans en cache*, l'extraction lit trois familles de vues de gestion dynamiques (droit `VIEW SERVER STATE`) :

Source SQL Server	Ce qu'Atlas en retient
`sys.dm_exec_procedure_stats`	par procédure : exécutions, temps CPU, durée, lectures et écritures logiques, dernière exécution, **plan XML**
`sys.dm_exec_query_stats` (200 requêtes les plus coûteuses)	par instruction : texte, lignes renvoyées (moyenne et maximum), mémoire accordée
`sys.dm_db_missing_index_details` et voisines	index suggérés par l'optimiseur : colonnes d'égalité, d'inégalité, incluses, gain estimé, nombre de demandes

Dans chaque plan, Atlas relève le **coût estimé** et les **opérateurs physiques** (Clustered Index Scan, Hash Match, Key Lookup...).

Des statistiques depuis le dernier redémarrage

Ces vues sont remises à zéro au redémarrage de SQL Server et ne gardent que les plans encore en cache. Analysez de préférence après une période d'activité représentative (fin de mois, clôture...).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

14.2 Les mesures par objet

EXEMPLE Les objets qui coûtent le plus (coût × exécutions) — duckdb

```duckdb
select o.sch || '.' || o.nom as objet, s.executions, round(s.cout, 1) as cout,
       round(s.duree_ms / s.executions, 1) as ms_exec, s.operateurs
from stats_requetes s join objets o on o.id = s.objet_id
order by s.cout * s.executions desc limit 6
```

▶ CE QUE PRODUIT ATLAS

objet	executions	cout	ms_exec	operateurs
vente.SP_CLIENT_FICHE	210000	31.8	0	Clustered Index Scan
vente.SP_STAT_VENTES	4212	145.2	3	Clustered Index Scan
vente.SP_CA_CLIENT	4211	145.2	3	Clustered Index Scan
vente.SP_CA_MENSUEL	4210	145.2	3	Clustered Index Scan
dw.SP_CHARGER_FAIT_VENTE	365	1520.3	262.5	Hash Match
dbo.SP_BALANCE	900	410.6	9	Clustered Index Scan

`vente.SP_CLIENT_FICHE` a un coût unitaire modeste mais 210 000 exécutions : c'est elle qui pèse le plus sur le serveur. C'est pourquoi Atlas raisonne toujours en **coût × fréquence**.

14.3 Les règles de performance

Règle	Déclenchée quand…	Niveau
`TABLE_SCAN`	le plan contient un Table Scan ou un Index Scan et le coût dépasse 20	critique au-delà d'un coût de 500, sinon important
`JOINTURE_COUTEUSE`	le plan contient un Hash Match et le coût dépasse 100	important
`INDEX_MANQUANT`	SQL Server suggère un index sur une table	critique si gain estimé ≥ 80 %, sinon important
`JOINTURE_SANS_INDEX`	une colonne de jointure d'une table de plus de 10 000 lignes n'est en tête d'aucun index	opportunité
`CURSEUR_INUTILE`	un constat CURSEUR de l'analyseur, pondéré par les exécutions	critique si exécutions × coût > 10 000
`FILTRAGE_TARDIF`	un constat FONCTION_FILTRE, LIKE_DEBUT ou NOT_IN, pondéré par les exécutions	critique si exécutions × coût > 10 000

Pourquoi reclasser les constats de l'analyseur

Un curseur dans une procédure lancée une fois par mois est un détail ; le même curseur exécuté 23 000 fois par jour est un problème. Le moteur de performances reprend les constats de l'analyseur et les rend **critiques** quand les statistiques montrent qu'ils coûtent vraiment.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

14.4 Des index manquants au script

EXEMPLE Les index suggérés par SQL Server — duckdb

```
1  select table_nom, egalite, inclus, round(impact) as impact, demandes from index_manquants order by
   impact desc limit 5
```

▶ CE QUE PRODUIT ATLAS

table	egalite	inclus	gain	demandes
LOG_TRAITEMENT			95	540
FACTURE	[CODE_CLIENT]	[MONTANT_HT], [MONTANT_TTC]	87	12800
ECRITURE	[CODE_TIERS]	[DEBIT], [CREDIT]	71	900
COMMANDE	[CLIENT_CODE]	[STATUT], [REMISE]	64	3100
CONTACT	[CODCLI]		58	210000

Pour chacun, Atlas écrit le `CREATE INDEX` correspondant, nommé selon une convention lisible (`IX_table_colonnes`) :

CORRECTIF Le correctif d'un constat INDEX_MANQUANT — diagnostics.correctif

```
1  CREATE NONCLUSTERED INDEX IX_COMMANDE_CLIENT_CODE ON ERP.vente.COMMANDE ([CLIENT_CODE]) INCLUDE
   ([STATUT], [REMISE]);
```

Attention

Les suggestions de SQL Server sont locales à une requête : elles se recouvrent souvent. Regroupez-les (un index peut en servir plusieurs), et pesez le coût en écriture avant de créer un index sur une table très mise à jour.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

14.5 L'onglet Perfs

69 requêtes / objets mesurés (statistiques SQL Server, plans en cache) · 27 recommandations — Script des corrections

Requête / objet	Exécu...	Coût ...	Lignes	Temp...	CPU (...	Mémoi...	Lectures...	Opérateurs du plan
stock.SP_INVENTAIRE	365	980.1	5200000	640	512	1024	117612	Clustered Index Scan
dw.SP_CHARGER_FAIT_VENTE	365	1520.3	9800	262.5	210	512000	182436	Hash Match
vente.SP_EMISSION_FACTURE	365	812.4	2300	230	184	512000	97488	Hash Match
dbo.FN_TTC	76349	0.04	34	0.5	0.4	1024	4	Clustered Index Seek
vente.SP_CONTACT_MODIFIER	73458	0.06	17	0.5	0.4	1024	7	Clustered Index Seek
vente.SP_REGLEMENT_SUPPRI...	70567	0.08	40	0.5	0.4	1024	9	Clustered Index Seek
rh.SP_EMPLOYE_LIRE	67676	0.01	23	0.5	0.4	1024	1	Clustered Index Seek
vente.SP_SYNCHRO_TIERS	71321	0.01	38	0.4	0.3	1024	1	Clustered Index Seek
dbo.SP_PARAMETRE_LIRE	68430	0.03	21	0.4	0.3	1024	3	Clustered Index Seek
vente.SP_CONTACT_AJOUTER	65539	0.05	4	0.4	0.3	1024	6	Clustered Index Seek
vente.SP_REGLEMENT_MODIFI...	62648	0.07	27	0.4	0.4	1024	8	Clustered Index Seek
stock.SP_ENTREPOT_SUPPRIMER	59757	0.09	10	0.5	0.4	1024	10	Clustered Index Seek

Niveau	Règle	Objet	Constat	Optimisation proposée
critique	CURSEUR_INUTILE	vente.SP_EMISSION_FACT...	Curseur cur_cmd : traitement ligne à ligne — 365 exécutions	Réécrire en traitement ensembliste (UPDATE ...
critique	CURSEUR_INUTILE	stock.SP_INVENTAIRE	Curseur c_art : traitement ligne à ligne — 365 exécutions	Réécrire en traitement ensembliste (UPDATE ...
critique	FILTRAGE_TARDIF	vente.SP_CLIENT_FICHE	Fonction UPPER() sur une colonne filtrée : l'index ne peut pa...	Réécrire en intervalle (col >= @debut AND col <
critique	FILTRAGE_TARDIF	vente.SP_EMISSION_FACT...	Fonction YEAR() sur une colonne filtrée : l'index ne peut pas ...	Réécrire en intervalle (col >= @debut AND col <
critique	FILTRAGE_TARDIF	vente.SP_CLIENT_FICHE	NOT IN (SELECT ...) : piège des NULL et plan coûteux — 210...	Remplacer par NOT EXISTS (corrélé)
critique	INDEX_MANQUANT	dbo.LOG_TRAITEMENT	Index manquant sur dbo.LOG_TRAITEMENT (gain estimé 95...	Créer l'index proposé (vérifier les index existant
critique	INDEX_MANQUANT	vente.FACTURE	Index manquant sur vente.FACTURE (gain estimé 87.0 %, 12...	Créer l'index proposé (vérifier les index existant
critique	TABLE_SCAN	stock.SP_INVENTAIRE	Balayage (Clustered Index Scan) : coût 980.1, 365 exécution...	Index couvrant sur les colonnes filtrées ou joint
critique	TABLE_SCAN	dbo.SP_NETTOYAGE_LOG	Balayage (Table Scan) : coût 1240.7, 180 exécutions, CPU tot...	Index couvrant sur les colonnes filtrées ou joint
▲ important	CURSEUR_INUTILE	dbo.SP_CLOTURE_EXERCICE	Curseur c : traitement ligne à ligne — 23807 exécutions	Réécrire en traitement ensembliste (UPDATE ...
▲ important	FILTRAGE_TARDIF	vente.SP_PURGE_COMMA...	Fonction CONVERT() sur une colonne filtrée : l'index ne peut...	Réécrire en intervalle (col >= @debut AND col <

Figure 18 — L'onglet Perfs : en haut les mesures par requête et objet (exécutions, coût, temps, opérateurs), en bas les recommandations triées par gravité.

Script des corrections rassemble tous les correctifs (index, réécritures) dans un script commenté, ouvert dans l'éditeur pour relecture. L'assistant donne la même lecture, objet par objet :

QUESTION **Demander une optimisation** — Assistant

```
1  optimise SP_CA_MENSUEL
```

▶ RÉPONSE DE L'ASSISTANT

```
**Constats sur SP_CA_MENSUEL** :

- ▲ Balayage (Clustered Index Scan) : coût 145.2, 4210 exécutions, CPU total 10.0 s → *Index couvrant sur
les colonnes filtrées ou jointes ; réécrire les prédicats non SARGables*
- ◇ Jointure CLIENT.CODE_CLIENT = FACTURE.CODE_CLIENT : FACTURE.CODE_CLIENT n'est en tête d'aucun index (5
objets) (l.5) → *CREATE INDEX IX_FACTURE_CODE_CLIENT ON FACTURE (CODE_CLIENT)*

Mesures : 4210 exécutions, coût 145.2, 3 ms et 17.4 k lectures par exécution ; opérateurs : Clustered Index
Scan.
```


© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 15 · LES DOUBLONS

Le copier-coller, mesuré et mutualisé

Les quatre genres de doublons, la similarité de Jaccard expliquée pas à pas, les indices de duplication et la génération du code mutualisé.



15.1 Quatre genres de doublons

15.2 La similarité de Jaccard, pas à pas

15.3 Indices et onglet Doublons

Dans ce chapitre, vous apprendrez à…

- distinguer doublons identiques, voisins, jointures et règles
- calculer une similarité de Jaccard
- comprendre le seuil de 72 %
- lire les indices de duplication
- parcourir les groupes de doublons
- générer une proposition de mutualisation

15.1 Quatre genres de doublons

Genre	Deux objets sont rapprochés quand…
identique	une de leurs instructions a la même **empreinte** : même texte normalisé (littéraux remplacés par `?`, variables par `@v`, espaces et casse uniformisés)
voisine	deux instructions SQL de plus de 80 caractères ont une **similarité** d'au moins 72 % (section 15.2)
jointure	la même jointure (mêmes tables, mêmes colonnes) apparaît dans au moins trois objets
règle	la même règle de gestion normalisée (un `CASE`, un filtre) apparaît dans au moins deux objets

Chaque groupe reçoit une **proposition** : une vue pour une jointure répétée, une fonction ou une colonne calculée pour une règle, une procédure commune (ou un paramètre supplémentaire) pour du code voisin.

15.2 La similarité de Jaccard, pas à pas

Pour comparer deux instructions, Atlas découpe leur texte normalisé en **trigrammes de mots** (suites de trois jetons consécutifs) et calcule l'**indice de Jaccard** : le nombre de trigrammes communs divisé par le nombre de trigrammes distincts des deux.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CALCUL **Deux requêtes qui ne diffèrent que par le tri** texte normalisé

```
A : SELECT NOM , VILLE , PAYS FROM CLIENT WHERE ACTIF = ? AND PAYS = ? ORDER BY NOM
B : SELECT NOM , VILLE , PAYS FROM CLIENT WHERE ACTIF = ? AND PAYS = ? ORDER BY VILLE
```

▶ CALCUL D'ATLAS

```
trigrammes de A : 17      trigrammes de B : 17
communs         : 16      (seuls « ORDER BY NOM » et « ORDER BY VILLE » diffèrent)
distincts       : 18
Jaccard = 16 / 18 = 0,889   →   ≥ 0,72 : instructions « voisines »
```

Ajouter une colonne au milieu de la liste change davantage de trigrammes : `SELECT NOM , VILLE FROM …` et `SELECT NOM , VILLE , CP FROM …` n'ont que 10 trigrammes communs sur 16 (0,625) et ne sont pas rapprochés. Le seuil de 0,72 vise le copier-coller légèrement retouché.

15.3 Indices et onglet Doublons

EXEMPLE **Les premiers groupes de la démonstration** duckdb

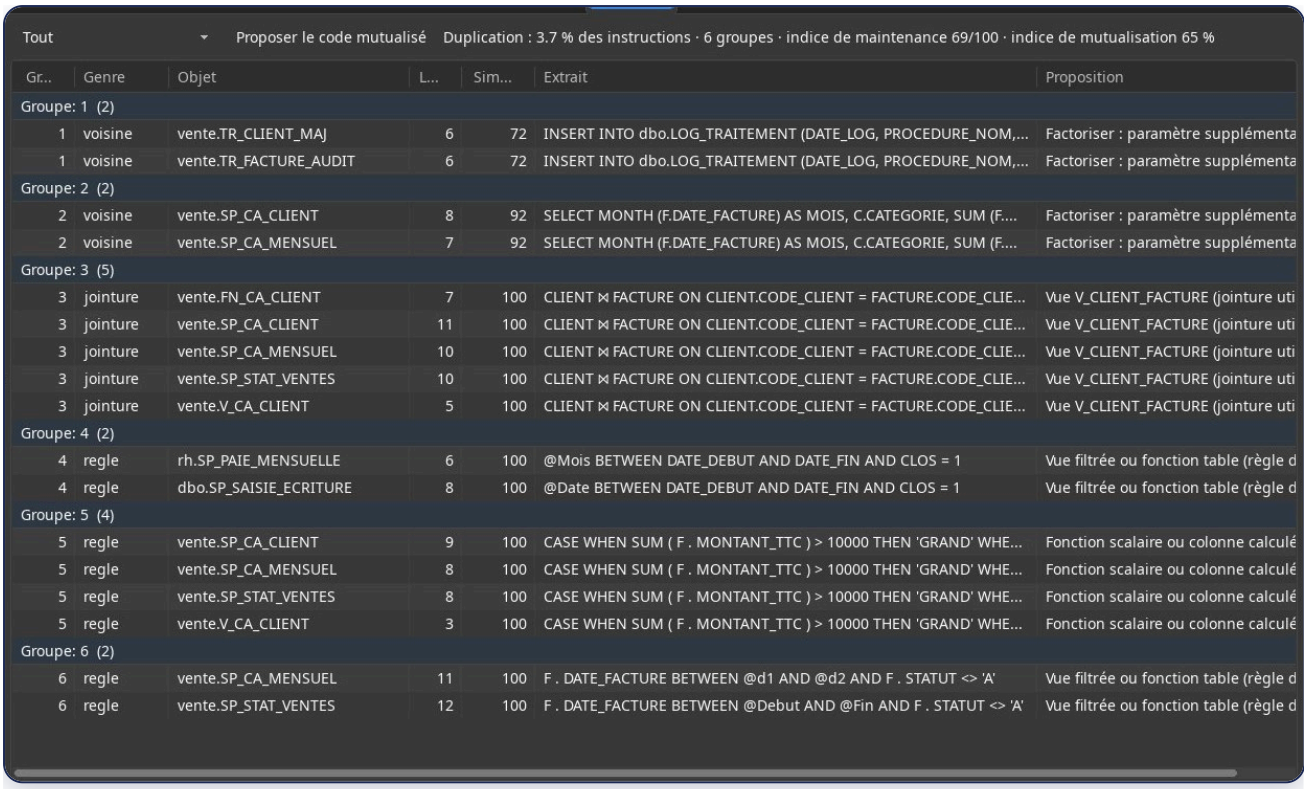
```
select groupe, u.genre, o.nom, round(similarite * 100) as sim from duplications u join objets o on
o.id = u.objet_id order by groupe, o.nom limit 10
```

▶ CE QUE PRODUIT ATLAS

groupe	genre	objet	sim
1	voisine	TR_CLIENT_MAJ	72
1	voisine	TR_FACTURE_AUDIT	72
2	voisine	SP_CA_CLIENT	92
2	voisine	SP_CA_MENSUEL	92
3	jointure	FN_CA_CLIENT	100
3	jointure	SP_CA_CLIENT	100
3	jointure	SP_CA_MENSUEL	100
3	jointure	SP_STAT_VENTES	100
3	jointure	V_CA_CLIENT	100
4	regle	SP_PAIE_MENSUELLE	100

Indice	Calcul	Démonstration
duplication	instructions SQL identiques ou voisines ÷ instructions SQL (SELECT, INSERT, UPDATE, DELETE, MERGE, WITH)	3,7 %
groupes	nombre de groupes de doublons	6
indice de maintenance	maintenabilité moyenne des modules (chapitre 16)	69/100
indice de mutualisation	100 − groupes ÷ occurrences : plus il est haut, plus chaque groupe rassemble d'objets	65 %

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr



Gr...	Genre	Objet	L...	Sim...	Extrait	Proposition
Groupe: 1 (2)						
1	voisine	vente.TR_CLIENT_MAJ	6	72	INSERT INTO dbo.LOG_TRAITEMENT (DATE_LOG, PROCEDURE_NOM,...	Factoriser : paramètre supplémenta
1	voisine	vente.TR_FACTURE_AUDIT	6	72	INSERT INTO dbo.LOG_TRAITEMENT (DATE_LOG, PROCEDURE_NOM,...	Factoriser : paramètre supplémenta
Groupe: 2 (2)						
2	voisine	vente.SP_CA_CLIENT	8	92	SELECT MONTH (F.DATE_FACTURE) AS MOIS, C.CATEGORIE, SUM (F....	Factoriser : paramètre supplémenta
2	voisine	vente.SP_CA_MENSUEL	7	92	SELECT MONTH (F.DATE_FACTURE) AS MOIS, C.CATEGORIE, SUM (F....	Factoriser : paramètre supplémenta
Groupe: 3 (5)						
3	jointure	vente.FN_CA_CLIENT	7	100	CLIENT ⋈ FACTURE ON CLIENT.CODE_CLIENT = FACTURE.CODE_CLIE...	Vue V_CLIENT_FACTURE (jointure uti
3	jointure	vente.SP_CA_CLIENT	11	100	CLIENT ⋈ FACTURE ON CLIENT.CODE_CLIENT = FACTURE.CODE_CLIE...	Vue V_CLIENT_FACTURE (jointure uti
3	jointure	vente.SP_CA_MENSUEL	10	100	CLIENT ⋈ FACTURE ON CLIENT.CODE_CLIENT = FACTURE.CODE_CLIE...	Vue V_CLIENT_FACTURE (jointure uti
3	jointure	vente.SP_STAT_VENTES	10	100	CLIENT ⋈ FACTURE ON CLIENT.CODE_CLIENT = FACTURE.CODE_CLIE...	Vue V_CLIENT_FACTURE (jointure uti
3	jointure	vente.V_CA_CLIENT	5	100	CLIENT ⋈ FACTURE ON CLIENT.CODE_CLIENT = FACTURE.CODE_CLIE...	Vue V_CLIENT_FACTURE (jointure uti
Groupe: 4 (2)						
4	regle	rh.SP_PAIE_MENSUELLE	6	100	@Mois BETWEEN DATE_DEBUT AND DATE_FIN AND CLOS = 1	Vue filtrée ou fonction table (règle d
4	regle	dbo.SP_SAISIE_ECRITURE	8	100	@Date BETWEEN DATE_DEBUT AND DATE_FIN AND CLOS = 1	Vue filtrée ou fonction table (règle d
Groupe: 5 (4)						
5	regle	vente.SP_CA_CLIENT	9	100	CASE WHEN SUM (F . MONTANT_TTC) > 10000 THEN 'GRAND' WHE...	Fonction scalaire ou colonne calculé
5	regle	vente.SP_CA_MENSUEL	8	100	CASE WHEN SUM (F . MONTANT_TTC) > 10000 THEN 'GRAND' WHE...	Fonction scalaire ou colonne calculé
5	regle	vente.SP_STAT_VENTES	8	100	CASE WHEN SUM (F . MONTANT_TTC) > 10000 THEN 'GRAND' WHE...	Fonction scalaire ou colonne calculé
5	regle	vente.V_CA_CLIENT	3	100	CASE WHEN SUM (F . MONTANT_TTC) > 10000 THEN 'GRAND' WHE...	Fonction scalaire ou colonne calculé
Groupe: 6 (2)						
6	regle	vente.SP_CA_MENSUEL	11	100	F . DATE_FACTURE BETWEEN @d1 AND @d2 AND F . STATUT <> 'A'	Vue filtrée ou fonction table (règle d
6	regle	vente.SP_STAT_VENTES	12	100	F . DATE_FACTURE BETWEEN @Debut AND @Fin AND F . STATUT <> 'A'	Vue filtrée ou fonction table (règle d

Figure 19 — L'onglet Doublons : chaque groupe avec ses objets, la ligne, la similarité, l'extrait et la proposition ; « Proposer le code mutualisé » génère un script.

Sélectionnez une ligne puis **Proposer le code mutualisé** : Atlas écrit, dans l'éditeur, la vue, la fonction ou la procédure commune qui remplacerait les copies, avec les appels à substituer. Pour fusionner deux procédures voisines, voyez aussi l'opération *Fusionner* du chapitre 18.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 16 · LE PATRIMOINE

La dette technique, chiffrée

Couplage, cohésion, complexité, dette, maintenabilité, réutilisation, score de risque et classes : comment Atlas évalue chaque objet du patrimoine.



16.1 Les métriques d'un objet
16.2 Les formules, sur un exemple
16.3 Les classes
16.4 L'onglet Patrimoine

Dans ce chapitre, vous apprendrez à...

- ✓ connaître chaque métrique d'un objet
- ✓ recalculer une maintenabilité à la main
- ✓ comprendre le score de risque
- ✓ lire les cinq classes
- ✓ ne pas confondre inutilisé et supprimable
- ✓ exploiter l'onglet Patrimoine

16.1 Les métriques d'un objet

Métrique	Définition
`lignes_code`, `nb_instructions`	taille du module
`complexite`	complexité cyclomatique (chapitre 6)
`entrant`	**couplage entrant** : nombre d'objets qui dépendent de lui
`sortant`	**couplage sortant** : nombre d'objets dont il dépend
`cohesion`	part des instructions qui portent sur sa table principale (1 = très cohésif)
`dette_min`	**dette technique** en minutes : 60 par constat critique, 20 par important, 5 par opportunité
`maintenabilite`	indice de 0 à 100 (section 16.2)
`reutilisation`	min(1, entrant ÷ 5) : 1 dès que cinq objets l'utilisent
`score`	**score de risque** de 0 à 100 (section 16.2)
`classe`	Critique, Sensible, Obsolète, Inutilisé, À refactoriser (section 16.3)

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

16.2 Les formules, sur un exemple

L'**indice de maintenabilité** reprend la formule classique d'Oman et Hagemeister, ramenée à 0-100 :

CALCUL **Maintenabilité et score de vente.SP_EMISSION_FACTURE** calcul

```
maintenabilité = (171 − 5,2 × ln(instructions × 12) − 0,23 × complexité − 16,2 × ln(lignes)) × 100 / 171
               = (171 − 5,2 × ln(37 × 12) − 0,23 × 7 − 16,2 × ln(64)) × 100 / 171
               = (171 − 31,70 − 1,61 − 67,37) × 100 / 171  =  41

score = min(100, complexité × 2 + (entrant + sortant) × 1,5 + dette ÷ 6 + 20 si « Critique »)
      = 7 × 2 + (1 + 11) × 1,5 + 200 ÷ 6 + 20  =  14 + 18 + 33,3 + 20  =  85
```

▶ LES MÊMES VALEURS DANS L'ENTREPÔT

objet	complexite	entrant	sortant	mi	dette	score	classe
vente.SP_EMISSION_FACTURE	7	1	11	41	205	86	Critique, À refactoriser
vente.SP_CLIENT_FICHE	1	0	4	68	195	61	Critique, Sensible, Obsolète, À refactoriser
stock.SP_INVENTAIRE	3	1	3	57	140	55	Critique, À refactoriser
vente.FACTURE	1	11	1		60	50	Critique
dbo.SP_LOG	1	29	1	75	0	47	
rh.EMPLOYE	1	8	0		60	44	Critique, Sensible

À retenir

Une maintenabilité sous 45 classe l'objet « À refactoriser ». Le score combine taille, couplage et dette : c'est lui qui trie les **objets à risque** de l'accueil et l'onglet Patrimoine.

16.3 Les classes

Un objet peut appartenir à plusieurs classes à la fois (« Critique, Sensible, Obsolète, À refactoriser »).

Classe	L'objet…
Critique	porte un constat critique, **ou** est appelé par un job SQL Agent, **ou** totalise plus de 100 000 exécutions
Sensible	contient des données personnelles, ou lit une table qui en contient
Obsolète	n'a pas été modifié depuis cinq ans, ou s'appelle `…OLD…` / `TMP…`, **et** personne ne l'utilise
Inutilisé	table, vue, procédure ou fonction que rien n'utilise, jamais exécutée, sans clé étrangère
À refactoriser	complexité > 12, ou dette ≥ 120 min, ou maintenabilité < 45, ou présent dans un groupe de doublons

« Inutilisé » ne veut pas dire « à supprimer »

Atlas ne voit ni les applications clientes, ni les rapports, ni les ETL externes. Avant d'archiver un objet inutilisé, capturez les appels quelques semaines (chapitre 19) et cherchez son nom dans les dépôts de code des applications.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

QUESTION **Ce que l'assistant dit des objets inutilisés** Assistant

```
1  objets inutilisés
```

▶ **RÉPONSE DE L'ASSISTANT**

```
**Objets inutilisés ou obsolètes** :

- ⚡ vente.TR_CLIENT_MAJ — Sensible, Obsolète, À refactoriser (modifié le 2016-11-08)
- ⚡ vente.TR_FACTURE_AUDIT — Obsolète, À refactoriser (modifié le 2020-07-21)
- λ vente.FN_CA_CLIENT — Sensible, Obsolète, À refactoriser (modifié le 2021-08-16)
- λ dbo.FN_REGEX_MATCH — Obsolète, Inutilisé (modifié le 2021-09-30)
- ƒ stock.SP_ARTICLE_AJOUTER — Obsolète (modifié le 2014-08-15)
- ƒ stock.SP_ARTICLE_LIRE — Obsolète (modifié le 2014-08-15)
- ƒ stock.SP_ARTICLE_LISTE — Obsolète (modifié le 2014-08-15)
…
```

16.4 L'onglet Patrimoine

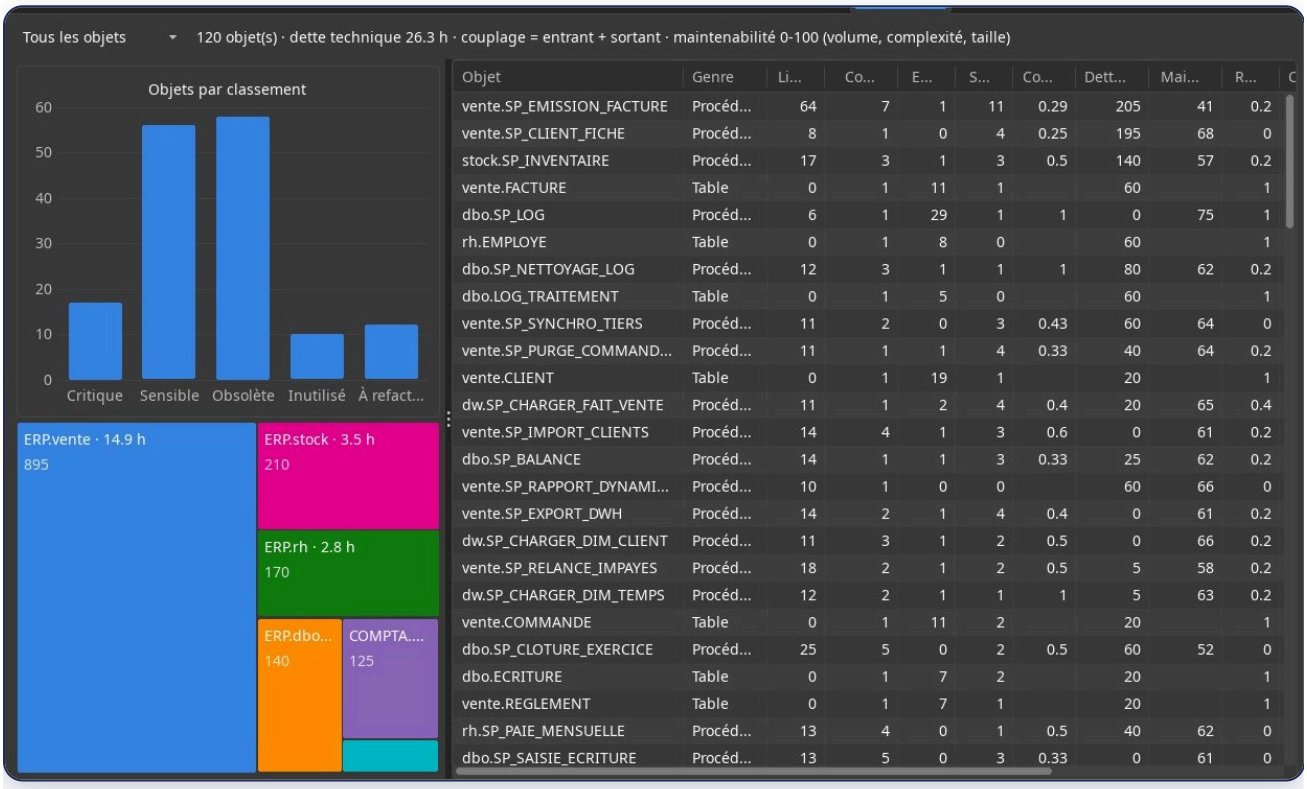


Figure 20 — L'onglet Patrimoine : objets par classe, dette par base (mosaïque), et le détail des métriques de chaque objet, trié par score.

- Le filtre choisit une classe (Critique, Sensible…) ; l'en-tête rappelle la dette totale (26,3 h pour la démonstration).
- La mosaïque montre la dette par base et par schéma : la surface est proportionnelle aux heures.
- Un double-clic sur une ligne rend l'objet courant ; `Code` ouvre son code.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 17 · L'ÉDITEUR ET LE DÉBOGUEUR

Comprendre un module ligne à ligne

L'éditeur T-SQL avec analyse en direct, le synoptique synchronisé, l'arbre syntaxique, l'exécution en lecture seule et le débogueur symbolique.



17.1 L'éditeur T-SQL
17.2 Le synoptique, l'arbre et l'analyse en direct
17.3 Exécuter une requête en lecture
17.4 Le débogueur symbolique

Dans ce chapitre, vous apprendrez à...

- ouvrir, formater et enrichir du code T-SQL
- utiliser extraits, complétion et définition (F12)
- lire le synoptique et l'arbre d'un module
- exécuter une requête en lecture seule
- déboguer pas à pas avec les paramètres capturés
- connaître les limites du débogueur symbolique

17.1 L'éditeur T-SQL

L'onglet **Code** est un éditeur T-SQL complet. On y arrive par `Code` (barre d'outils), par un double-clic dans l'explorateur, par un lien *Ouvrir le code* ou par `Nouveau` pour un script libre. Chaque document a son onglet, marqué de l'icône du genre de l'objet.

Commande	Effet
`Nouveau`	script vide (analysé comme le reste)
`Formater`	remet en forme le code (une clause par ligne, indentation)
`Extraits...`	insère un modèle : TRY...CATCH avec transaction, MERGE, CTE récursive, curseur → ensembliste, SQL dynamique paramétré, pagination, traitement par lots, index couvrant, procédure type
`Renommer...`	renomme une variable ou un nom dans le document
`Définition (F12)`	ouvre l'objet sous le curseur (table, procédure, fonction...)
`▶ Exécuter (lecture)`	exécute un SELECT sur la source de l'objet (section 17.3)
`Enregistrer .sql...`	enregistre le document dans un fichier

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE **Formater : avant et après** script

```
1  select c.CODE_CLIENT,c.NOM,sum(f.MONTANT_TTC) ca from vente.CLIENT c join vente.FACTURE f on
   f.CODE_CLIENT=c.CODE_CLIENT where f.STATUT<>'A' group by c.CODE_CLIENT,c.NOM order by ca desc
```

▶ APRÈS « FORMATER »

```
SELECT c.CODE_CLIENT, c.NOM, sum (f.MONTANT_TTC) ca
    FROM vente.CLIENT c
    JOIN vente.FACTURE f ON f.CODE_CLIENT = c.CODE_CLIENT
    WHERE f.STATUT <> 'A'
    GROUP BY c.CODE_CLIENT, c.NOM
    ORDER BY ca desc
```

La complétion propose les noms de l'entrepôt (tables, colonnes, procédures) et les variables du document ; les constats sont soulignés et détaillés au survol de la marge.

17.2 Le synoptique, l'arbre et l'analyse en direct

Le code est analysé **pendant la frappe** (une fraction de seconde après la dernière touche). Le panneau de droite offre six onglets :

Onglet	Contenu
Synoptique	le module découpé en blocs de phases (chapitre 6), reliés de haut en bas ; cliquer un bloc place le curseur sur sa première ligne, et le bloc courant suit le curseur
Arbre	l'arbre syntaxique : blocs, si/sinon, boucles, TRY/CATCH, avec ligne et phase de chaque nœud
Analyse	synthèse : lignes, instructions, complexité, synoptique, paramètres, variables, tables temporaires, curseurs, références par nature, SQL dynamique, constats
Dépendances	ce que l'objet utilise et qui l'utilise (depuis l'entrepôt)
Débogueur	l'exécution symbolique pas à pas (section 17.5)
Résultat	les lignes renvoyées par ▶ Exécuter (lecture)

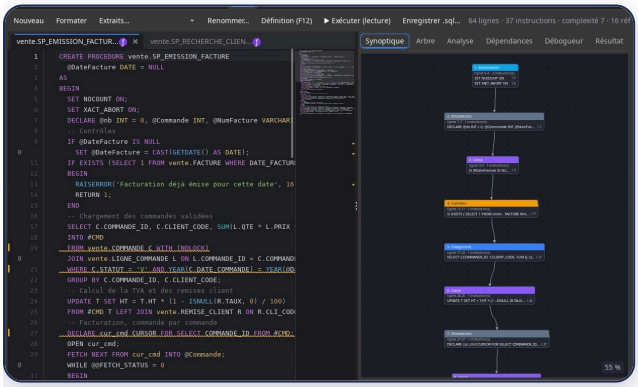


Figure 21 — Le synoptique de SP_EMISSION_FACTURE, à droite du code.

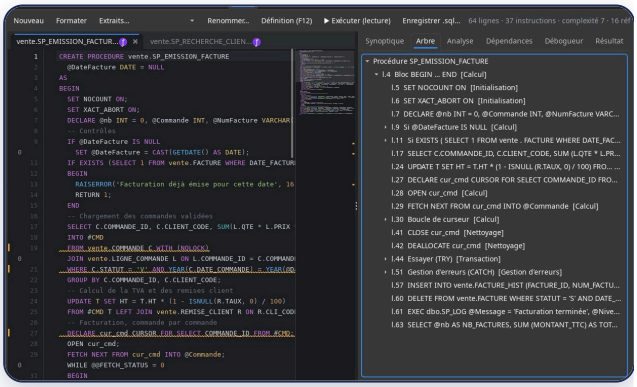


Figure 22 — L'arbre syntaxique du même module.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

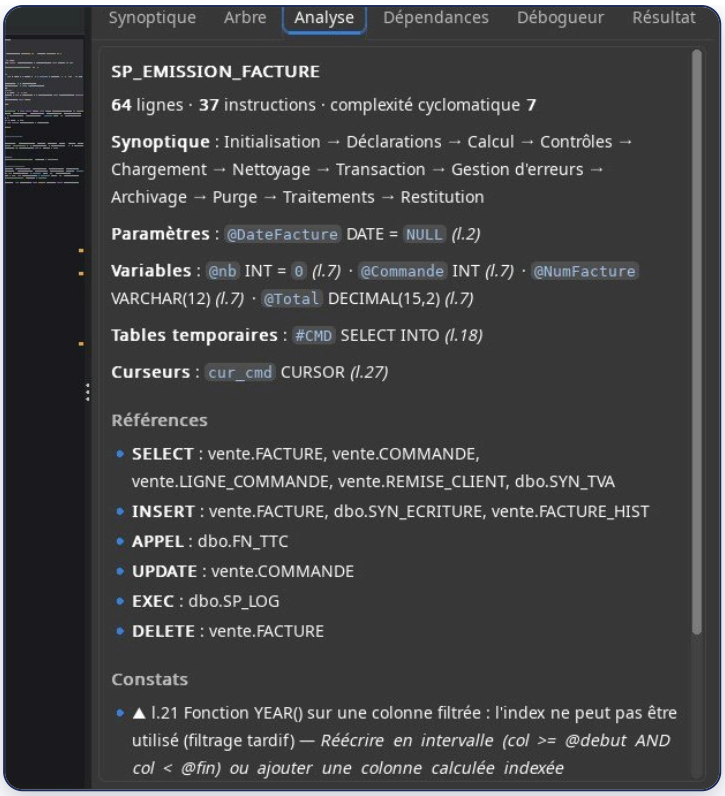


Figure 23 — L'onglet Analyse : la fiche technique du module en cours d'édition.

17.3 Exécuter une requête en lecture

`▶ Exécuter (lecture)` envoie la sélection (ou le document) à la source SQL Server de l'objet et affiche au plus 500 lignes dans l'onglet **Résultat**.

> **Sécurité**
>
> Seuls les textes commençant par `SELECT` ou `WITH` sont exécutés ; tout le reste est refusé avec le message « *Atlas ne modifie jamais les bases analysées* ». La source reste attachée en lecture seule.

17.4 Le débogueur symbolique

Le débogueur fait avancer le code **instruction par instruction**, sans l'exécuter sur le serveur : il suit les valeurs connues des paramètres et des variables, l'état des curseurs, les tables temporaires créées et les CTE. C'est un outil de **compréhension**, idéal pour une procédure de plusieurs centaines de lignes.

Commande	Touche	Effet
`▶ Démarrer`		prend les paramètres du dernier appel capturé (sinon leurs valeurs par défaut) et se place sur la première instruction
`Pas`	`F10`	instruction suivante
`Détaillé`	`F11`	si l'instruction est un `EXEC`, entre dans la procédure appelée (pile d'appels)
`→ Curseur`		avance jusqu'à la ligne du curseur de l'éditeur
`Continuer`	`F5`	avance jusqu'au prochain point d'arrêt (`F9` ou clic dans la marge)
`■`		arrête le débogage

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

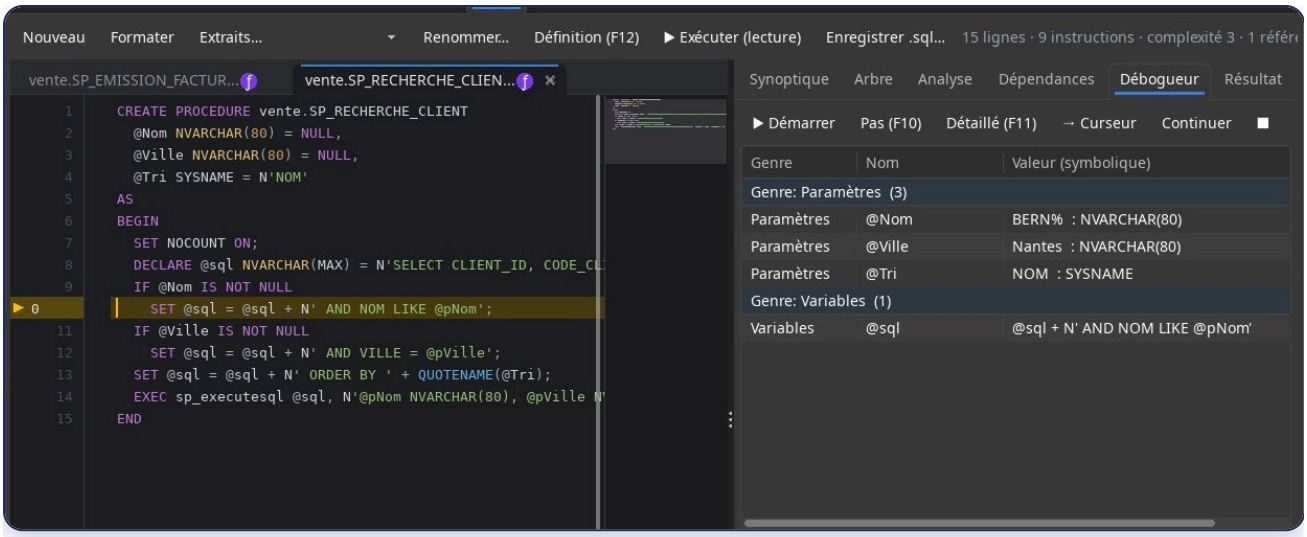


Figure 24 — Débogage de SP_RECHERCHE_CLIENT avec les paramètres d'un appel capturé : la ligne courante est surlignée, les paramètres et variables (avec leur valeur symbolique) à droite, l'historique des pas en bas.

Instruction	Effet suivi par le débogueur
`DECLARE`	la variable apparaît avec sa valeur par défaut ; un curseur passe à « déclaré » ; une variable table est créée vide
`SET @x = …`, `SELECT @x = …`	nouvelle valeur ; les paramètres connus sont substitués ((valeur)) ; « ← lu en base » si la valeur vient d'une table
`OPEN` / `FETCH` / `CLOSE` / `DEALLOCATE`	état du curseur ; `FETCH INTO` : « ← ligne suivante du curseur »
`CREATE TABLE #t`, `INSERT` / `UPDATE` / `DELETE`	table temporaire créée, puis marquée des écritures qu'elle reçoit
`WITH`	CTE déclarée (portée : l'instruction)

Ce que le débogueur ne fait pas

Il n'évalue pas les conditions : il parcourt les instructions **dans l'ordre**, les deux branches d'un `IF` comprises, et une boucle une seule fois. Les valeurs calculées par des requêtes restent symboliques. Pour une exécution réelle, utilisez le rejeu (chapitre 19).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 18 · LE REFACTORING

Transformer sans rien casser

Renommer, extraire, découper et fusionner : chaque opération calcule son impact sur tout le patrimoine, montre un aperçu ligne à ligne et produit un script à relire.



18.1 Le principe : l'impact d'abord

18.2 Renommer un objet ou une colonne

18.3 Extraire, découper, fusionner

Dans ce chapitre, vous apprendrez à...

- suivre les quatre étapes d'un refactoring
- renommer une colonne partout en un script
- lire l'aperçu avant / après
- extraire une sélection en procédure
- découper un module par phases
- fusionner deux procédures voisines

18.1 Le principe : l'impact d'abord

L'onglet **Refacto** prépare des transformations du patrimoine. Chaque opération suit les mêmes étapes :

1. choisir l'opération et ses paramètres ;
2. `Calculer l'impact` : tables, objets et lignes de code touchés, sur **tout** le patrimoine analysé, avec l'aperçu de chaque ligne concernée, avant et après ;
3. `Aperçu → script` : le script T-SQL complet, ouvert dans l'éditeur pour relecture.

Sécurité

Atlas **n'applique jamais** un refactoring. Il produit un script, transactionnel quand c'est possible, que vous relisez, testez sur une recette, puis exécutez.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

18.2 Renommer un objet ou une colonne

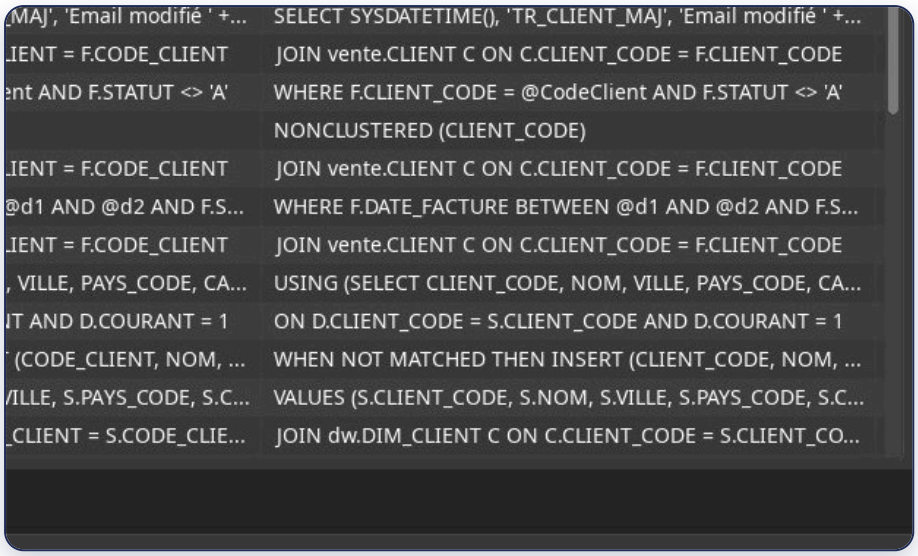


Figure 25 — Renommer CODE_CLIENT en CLIENT_CODE : 6 tables et 23 objets touchés, 42 lignes ; l'aperçu montre chaque ligne avant et après.

SCRIPT **Le script généré (extrait)** renommage_CODE_CLIENT.sql

```sql
-- Renommage CODE_CLIENT → CLIENT_CODE (généré par Atlas SQL : relire, tester, puis exécuter)
SET XACT_ABORT ON;
BEGIN TRANSACTION;

EXEC ERP.sys.sp_rename N'vente.CLIENT.CODE_CLIENT', N'CLIENT_CODE', 'COLUMN';
EXEC ERP.sys.sp_rename N'vente.FACTURE.CODE_CLIENT', N'CLIENT_CODE', 'COLUMN';
-- … une ligne par table

-- SRV-ERP › ERP : vente.SP_CA_CLIENT
USE [ERP];
GO
CREATE OR ALTER PROCEDURE vente.SP_CA_CLIENT
  -- … le code complet, CODE_CLIENT remplacé par CLIENT_CODE (mot entier, casse respectée)
GO
COMMIT TRANSACTION;
```

Le remplacement se fait **mot entier** (`CODE_CLIENT` ne touche pas `CODE_CLIENT_OLD`), et chaque module est ré-écrit en `CREATE OR ALTER` .

18.3 Extraire, découper, fusionner

Opération	Ce qu'elle produit
Extraire la sélection en procédure	les instructions sélectionnées dans l'éditeur deviennent une procédure `dbo.SP_EXTRAITE_…` ; les variables utilisées mais déclarées ailleurs deviennent ses paramètres (avec leur type) ; l'appel `EXEC` à substituer est donné
Découper par phases	une procédure par bloc de phase du synoptique (`…_1_CONTROLES` , `…_2_CHARGEMENT` …), plus une procédure « chef d'orchestre » qui les appelle dans l'ordre
Fusionner (doublons)	pour l'objet courant et son plus proche voisin (chapitre 15) : une procédure `…_FUSION` avec un paramètre `@Mode` qui choisit la variante ; les deux anciennes deviennent des enveloppes

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

SCRIPT **Découper par phases : la structure produite** SP_EMISSION_FACTURE_decoupe.sql

```sql
-- Découpage de SP_EMISSION_FACTURE en 9 phases (Atlas SQL)
-- Les variables partagées entre phases sont à passer en paramètres OUTPUT ou dans une table temporaire.
CREATE OR ALTER PROCEDURE vente.SP_EMISSION_FACTURE_1_INITIALISATION @DateFacture DATE AS ...
CREATE OR ALTER PROCEDURE vente.SP_EMISSION_FACTURE_2_CONTROLES @DateFacture DATE AS ...
CREATE OR ALTER PROCEDURE vente.SP_EMISSION_FACTURE_3_CHARGEMENT @DateFacture DATE AS ...
-- …
-- Chef d'orchestre
CREATE OR ALTER PROCEDURE vente.SP_EMISSION_FACTURE @DateFacture DATE
AS
BEGIN
    SET NOCOUNT ON;
    EXEC vente.SP_EMISSION_FACTURE_1_INITIALISATION @DateFacture;
    EXEC vente.SP_EMISSION_FACTURE_2_CONTROLES @DateFacture;
    -- …
END;
```

Attention

Découper une procédure qui partage des variables ou une transaction entre ses phases demande un travail manuel : le script le rappelle en tête. Il fournit la structure ; la logique de passage des données reste à écrire.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 19 · L'INTERCEPTION

Les appels réels, capturés et rejoués

Capturer les appels des applications par Extended Events sans les modifier, les importer, les simuler dans le débogueur, les rejouer dans une transaction annulée.

19.1 Voir les appels sans toucher aux applications
19.2 Installer la capture
19.3 Importer et lire les appels
19.4 Simuler, rejouer, exporter



Dans ce chapitre, vous apprendrez à...

- comprendre la capture par Extended Events
- installer et arrêter la session de capture
- importer les appels et leurs paramètres
- simuler un appel avec des paramètres modifiés
- rejouer un appel en toute connaissance de cause
- exporter les appels en CSV

19.1 Voir les appels sans toucher aux applications

Le code des bases ne dit pas tout : quelles procédures les applications appellent-elles vraiment, avec quels paramètres, à quelle fréquence ? Atlas s'appuie sur les **Extended Events** de SQL Server : une session de capture côté serveur enregistre les appels terminés dans des fichiers `.xel`, sans modifier ni ralentir sensiblement les applications.



19.2 Installer la capture

`Script de capture (XE)...` ouvre dans l'éditeur le script à exécuter sur chaque serveur observé (droit `ALTER ANY EVENT SESSION`) :

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

EXEMPLE La session de capture d'Atlas — capture_xe.sql

```sql
IF EXISTS (SELECT 1 FROM sys.server_event_sessions WHERE name = N'AtlasCapture')
    DROP EVENT SESSION [AtlasCapture] ON SERVER;
GO
CREATE EVENT SESSION [AtlasCapture] ON SERVER
ADD EVENT sqlserver.rpc_completed (
    ACTION (sqlserver.client_app_name, sqlserver.username, sqlserver.database_name, sqlserver.client_hostname)
    WHERE sqlserver.database_name <> N'master' AND sqlserver.database_name <> N'msdb' AND duration > 0),
ADD EVENT sqlserver.sql_batch_completed (
    ACTION (sqlserver.client_app_name, sqlserver.username, sqlserver.database_name, sqlserver.client_hostname)
    WHERE sqlserver.database_name <> N'master' AND sqlserver.database_name <> N'msdb')
ADD TARGET package0.event_file (SET filename = N'AtlasCapture.xel', max_file_size = (200), max_rollover_files = (5))
WITH (MAX_MEMORY = 8 MB, EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS, MAX_DISPATCH_LATENCY = 5 SECONDS, STARTUP_STATE = OFF);
GO
ALTER EVENT SESSION [AtlasCapture] ON SERVER STATE = START;
```

Les fichiers tournent sur cinq fois 200 Mo. Pour arrêter : `ALTER EVENT SESSION [AtlasCapture] ON SERVER STATE = STOP;`

19.3 Importer et lire les appels

Analyse › Importer les appels capturés (ou Importer les captures) place un travail *captures* par source : il lit les fichiers avec `sys.fn_xe_file_target_read_file`, extrait le texte de l'appel et ses **paramètres** (convertis en JSON), la durée, le nombre de lignes, l'application et l'utilisateur.

EXEMPLE Les derniers appels capturés — duckdb

```sql
select strftime(quand, '%Y-%m-%d %H:%M') as quand, objet, parametres, duree_ms, lignes, application
from captures order by quand desc limit 5
```

▶ CE QUE PRODUIT ATLAS

quand	objet	parametres	ms	lignes	application
2026-11-03 23:41	vente.SP_RECHERCHE_CLIENT	{"NOM":"BERN%","VILLE":"Nantes","TRI":"NOM"}	35	313	ERP Client lourd
2026-11-03 22:34	vente.SP_RECHERCHE_CLIENT	{"NOM":"LE%","VILLE":"Paris","TRI":"CODE_CLIENT"}	18	282	ERP Client lourd
2026-11-03 21:27	vente.SP_RECHERCHE_CLIENT	{"NOM":"MART%","TRI":"VILLE"}	91	251	ERP Client lourd
2026-11-03 20:20	vente.SP_RECHERCHE_CLIENT	{"NOM":"DUR%","VILLE":"Lyon","TRI":"NOM"}	74	220	ERP Client lourd
2026-11-03 19:13	vente.SP_RECHERCHE_CLIENT	{"TRI":"DATE_CREATION"}	57	189	ERP Client lourd

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

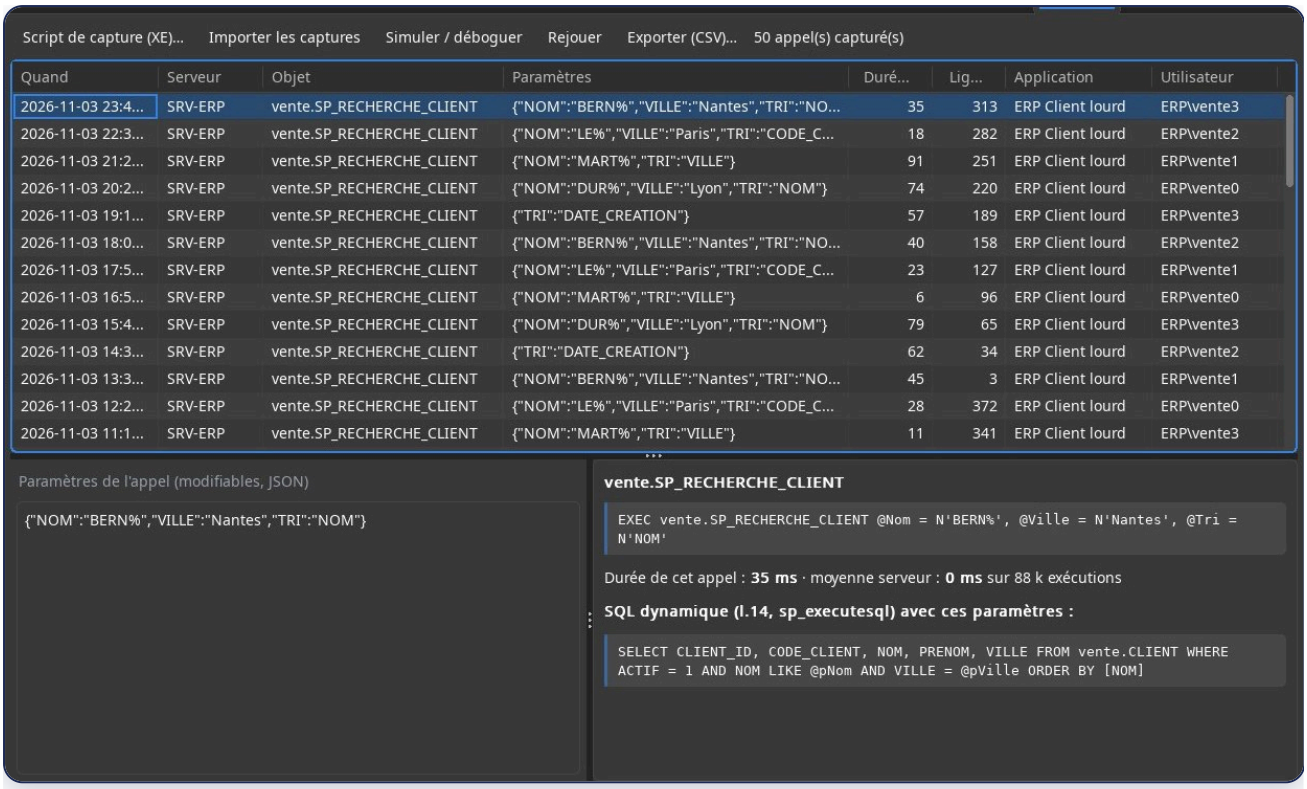


Figure 26 — L'onglet Captures : la liste des appels, les paramètres modifiables (JSON) et, à droite, l'appel reconstitué et le SQL dynamique avec ses valeurs.

19.4 Simuler, rejouer, exporter

Bouton	Effet
Simuler / déboguer	ouvre le code de la procédure et lance le débogueur avec les paramètres de l'appel — **éventuellement modifiés** dans la zone JSON (chapitre 17)
Rejouer	exécute l'appel sur le serveur dans une transaction **annulée**, après confirmation (ci-dessous)
Exporter (CSV)...	enregistre les appels affichés dans un fichier CSV

REJEU **Ce qu'Atlas envoie pour un rejeu** travail « rejeu »

```
SET NOCOUNT ON; BEGIN TRANSACTION;
EXEC vente.SP_RECHERCHE_CLIENT @Nom = N'BERN%', @Ville = N'Nantes', @Tri = N'NOM';
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
```

Les limites du rejeu

Le `ROLLBACK` annule les écritures en base, mais pas les effets **hors transaction** : envoi de courriels (Database Mail), appels à des serveurs liés, valeurs de séquences et d'identité consommées, fichiers écrits, jobs démarrés. Atlas demande donc toujours confirmation, et la démonstration simule le rejeu. Rejouez de préférence sur une copie de recette.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 20 · L'ASSISTANT

Interroger le patrimoine en français

Le moteur de règles intégré, les familles de questions, les actions sur l'objet courant, et le branchement facultatif d'un modèle local ou distant.

20.1 Poser une question en français
20.2 Brancher un modèle de langage



Dans ce chapitre, vous apprendrez à...

- poser des questions sur les relations et les thèmes
- compter, résumer, expliquer un objet
- obtenir l'impact d'un objet ou d'une colonne
- documenter un objet en un clic
- brancher un modèle GGUF ou un serveur compatible OpenAI
- savoir ce qui est envoyé, et à qui

20.1 Poser une question en français

L'onglet **Assistant** répond aux questions sur le patrimoine. Par défaut, il n'utilise **aucun modèle d'IA** : un moteur de règles reconnaît la question et interroge l'entrepôt. Les réponses sont donc exactes et immédiates.

Famille de questions	Exemples
relations	*qui écrit dans FACTURE ?* · *qui lit CLIENT ?* · *qui appelle SP_LOG ?* · qui supprime, met à jour, insère
thèmes	*quelles tables contiennent des données personnelles ?* · *objets inutilisés* · *liens inter-serveurs* · *SQL dynamique* · *types métier*
comptages	*combien de procédures ?* · *combien de colonnes ?* · *combien d'objets ?* (synthèse)
impact	*impact de CLIENT* (objet) · *impact de CLIENT_CODE* (colonne)
sur un objet	*résume* · *explique* · *anomalies de* · *optimise* · *refactorise* · *documente* + nom de l'objet

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

QUESTION Qui écrit dans FACTURE ? Assistant

```
1  qui écrit dans FACTURE ?
```

▶ **RÉPONSE DE L'ASSISTANT**

vente.FACTURE (Table) — 2 objet(s) :

- DELETE · ƒ SRV-ERP.ERP.vente.SP_EMISSION_FACTURE · 365 exécutions
- INSERT · ƒ SRV-ERP.ERP.vente.SP_EMISSION_FACTURE · 365 exécutions

QUESTION Combien de procédures ? Assistant

```
1  combien de procédures ?
```

▶ **RÉPONSE DE L'ASSISTANT**

66 procédure(s) :

- 🖥 SRV-COMPTA › COMPTA : **3**
- 🖥 SRV-DWH › DWH : **3**
- 🖥 SRV-ERP › ERP : **60**

QUESTION Explique une procédure Assistant

```
1  explique SP_EMISSION_FACTURE
```

▶ **RÉPONSE DE L'ASSISTANT**

Fonctionnement de vente.SP_EMISSION_FACTURE

Paramètres : @DateFacture (DATE, défaut NULL).

1. **Initialisation** — SET NOCOUNT ON ; SET XACT_ABORT ON
2. **Déclarations** — DECLARE @nb INT = 0, @Commande INT, @NumFacture VARCHAR (12), @Total …
3. **Calcul** — Si @DateFacture IS NULL
4. **Contrôles** — Si EXISTS (SELECT 1 FROM vente . FACTURE WHERE DATE_FACTURE = @DateF…
5. **Chargement** — SELECT C.COMMANDE_ID, C.CLIENT_CODE, SUM (L.QTE * L.PRIX * (1 - ISNUL…
6. **Calcul** — UPDATE T SET HT = T.HT * (1 - ISNULL (R.TAUX, 0) / 100) FROM #CMD T L…
7. **Déclarations** — DECLARE cur_cmd CURSOR FOR SELECT COMMANDE_ID FROM #CMD
8. **Calcul** — OPEN cur_cmd ; FETCH NEXT FROM cur_cmd INTO @Commande ; Boucle de curseur
9. **Nettoyage** — CLOSE cur_cmd ; DEALLOCATE cur_cmd
10. **Transaction** — Essayer (TRY)
11. **Gestion d'erreurs** — Gestion d'erreurs (CATCH)
12. **Archivage** — INSERT INTO vente.FACTURE_HIST (FACTURE_ID, NUM_FACTURE, CODE_CLIENT,…
13. **Purge** — DELETE FROM vente.FACTURE WHERE STATUT = 'S' AND DATE_FACTURE < DATEA…
14. **Traitements** — EXEC dbo.SP_LOG @Message = 'Facturation terminée', @Niveau = 'INFO'
15. **Restitution** — SELECT @nb AS NB_FACTURES, SUM (MONTANT_TTC) AS TOTAL_TTC FROM vente.…

vente.SP_EMISSION_FACTURE — Procédure de SRV-ERP › ERP. Code de 64 lignes, complexité 7, maintenabilité 41/100. Lit : SYN_TVA, COMMANDE, FACTURE, LIGNE_COMMANDE, REMISE_CLIENT, TVA. Écrit : FACTURE (delete), ECRITURE (insert), SYN_ECRITURE (insert), FACTURE (insert), FACTURE_HIST (insert), COMMANDE (update). Appelle : FN_TTC, SP_LOG. Utilisé par 1 objet(s) : ERP - Facturation nocturne. Classement : **Critique, À refactoriser**.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

QUESTION **Que faire de SP_CLIENT_FICHE ?** Assistant

```
refactorise SP_CLIENT_FICHE
```

▶ RÉPONSE DE L'ASSISTANT

```
**Refactorisation de SP_CLIENT_FICHE** :

- Dette estimée 195 min, due surtout à :
    - ⬤ FILTRAGE_TARDIF (×2) → Remplacer par NOT EXISTS (corrélé)
    - ▲ NOT_IN → Remplacer par NOT EXISTS (corrélé)
    - ▲ FONCTION_FILTRE → Réécrire en intervalle (col >= @debut AND col < @fin) ou ajouter une colonne
calculée indexée
    - ▲ TABLE_SCAN → Index couvrant sur les colonnes filtrées ou jointes ; réécrire les prédicats non
SARGables
    - ◇ SELECT_ETOILE (×3) → Lister les colonnes utiles (moins d'E/S, plans couvrants possibles, résistance
aux évolutions)
```

Astuce

Sans nom d'objet dans la question, l'assistant prend l'**objet courant**. Les boutons Résumer Expliquer Anomalies Optimiser Refactoriser Documenter posent ces questions pour lui. Documenter produit une fiche complète, colonnes comprises, à copier dans un wiki.

20.2 Brancher un modèle de langage

Réglages… choisit le moteur :

Moteur	Fonctionnement	Réglages
regles (défaut)	moteur intégré, sans IA ; exact, instantané, hors ligne	aucun
local	un modèle GGUF chargé sur le poste par le paquet `llm` de Lexis+	le fichier `.gguf`
http	un serveur compatible OpenAI (Ollama, llama.cpp, LM Studio, service en ligne)	URL `…/v1/chat/completions`, nom du modèle, variable d'environnement de la clé

RÉGLAGES **Réglages pour un serveur Ollama local** Assistant › Réglages…

```
moteur : http
URL    : http://localhost:11434/v1/chat/completions
modèle : qwen2.5-coder
clé    : (vide) — ou le NOM d'une variable d'environnement, ex. ATLAS_IA_CLE
```

Avec un modèle, la question est envoyée avec les **faits** extraits de l'entrepôt (fiche de l'objet, dépendances, constats) et une consigne : *« répondre en français à partir des faits fournis, sans inventer d'objets ni de chiffres »*.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Vos données restent chez vous

Avec les moteurs **regles** et **local**, rien ne quitte le poste. Avec **http**, seuls les faits utiles à la question (noms d'objets, extraits de code, métriques) sont envoyés au serveur choisi — jamais les données des tables. La clé d'API n'est jamais écrite dans un fichier : Atlas lit la variable d'environnement que vous indiquez.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

CHAPITRE 21 · ALLER PLUS LOIN

Sous le capot, et après

L'architecture d'Atlas SQL, ses performances, ses limites connues et la feuille de route du MVP à la version Enterprise.

21.1 L'architecture
21.2 Performances et volumétrie
21.3 Limites connues
21.4 De l'outil au service



Dans ce chapitre, vous apprendrez à...

- ✓ situer chaque fichier .prg dans l'architecture
- ✓ estimer les performances sur un gros patrimoine
- ✓ connaître les limites à garder en tête
- ✓ voir la feuille de route

21.1 L'architecture

Atlas suit une organisation MVC : l'**entrepôt** est le modèle, la **file d'attente** le contrôleur, les **moteurs** les services, et les **vues** ne font que lire l'entrepôt.

Fichier	Rôle
`principal.prg`	fenêtre, barre d'outils, panneau de gauche, navigation, recherche, fiche
`vues.prg`	accueil, carte, dépendances, dictionnaire, types, flux, doublons, dynamique, perfs, patrimoine, constats
`editeur.prg`	éditeur, synoptique, analyse en direct, débogueur, refactoring
`interception.prg`, `assistant.prg`	captures ; assistant
`agents.prg`	file d'attente, dépendances entre travaux, agents, panneau de suivi
`sources.prg`	connexions, secrets, extraction T-SQL, normalisation, planification
`sqlanalyse.prg`	analyseur T-SQL : jetons, instructions, arbre, phases, références, constats
`dependances.prg`	résolution, relations, impact, flux
`analyses.prg`	dictionnaire, types, duplications, performances, patrimoine
`entrepot.prg`	schéma DuckDB, accès, réglages, utilitaires
`menus.prg`, `demo.prg`	menus et exports ; démonstration

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Le fichier `ARCHITECTURE.md` (menu *Aide* › *Architecture*) détaille le modèle de données, les services et le plan de livraison.

21.2 Performances et volumétrie

- Découpage en jetons par DuckDB : environ 40 000 jetons en 0,3 s.
- Insertions par lots de 400 lignes (JSON déroulé par DuckDB).
- Démonstration complète : 37 travaux, environ 2 s avec 4 à 6 agents.
- Les agents ont chacun leur session DuckDB : le parallélisme vient des requêtes SQL, qui portent l'essentiel du travail.
- Pour un gros patrimoine (des milliers de modules), augmentez le réglage `LOT` (modules par travail d'analyse) et le nombre d'agents.

21.3 Limites connues

Domaine	Limite
Débogueur	symbolique : n'exécute pas le code, ne choisit pas les branches (chapitre 17)
Rejeu	transaction annulée : les effets hors transaction ne sont pas annulés (chapitre 19)
Analyseur	tolérant, sans grammaire T-SQL complète ; XML DML et CLR gardés comme texte
Impact	limité au code analysé : applications, rapports et ETL externes ne sont pas vus
Chiffrement strict	peut être refusé par l'extension mssql ; passer par ODBC
Interface	pas d'ancrage libre des panneaux, pas de repli du code, pas de sélection multiple dans la carte

21.4 De l'outil au service

Étape	Contenu	État
MVP	inventaire, entrepôt, file et agents, analyse T-SQL, dépendances, explorateur, recherche, fiche, carte	livré
V1	dictionnaire, types métier, flux, SQL dynamique, performances, constats, exports	livré
V2	éditeur avancé, synoptique, débogueur symbolique, refactoring, doublons, patrimoine	livré
V3	interception, simulation, rejeu, assistant, espaces de travail, fenêtres détachables	livré
Enterprise	analyse incrémentale, entrepôt partagé, historique des campagnes, SSIS/SSRS, lignage colonne à colonne, intégration continue	à venir

Pour conclure

Ajoutez votre premier vrai serveur (chapitre 3) avec une limite d'un agent, lancez l'analyse, puis posez à l'assistant la question qui vous a amené à Atlas. Gardez le guide ouvert sur l'annexe A pendant la lecture des constats.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

ANNEXE A · RÉFÉRENCE DES CONSTATS

Tous les constats, en un tableau

Les vingt-sept règles d'Atlas SQL : module, niveau, signification et correction proposée.

A.1 Les vingt-sept règles



Dans ce chapitre, vous apprendrez à...

- retrouver une règle par son nom
- connaître le module qui la produit
- savoir quoi corriger

A.1 Les vingt-sept règles

Module	Règle	Niveau	Signification	Correction
analyse	`SELECT_ETOILE`	opportunité	SELECT * : toutes les colonnes sont lues	lister les colonnes utiles
analyse	`SANS_WHERE`	critique	UPDATE ou DELETE sans WHERE	vérifier l'intention ; filtre explicite
analyse	`FONCTION_FILTRE`	important	fonction sur une colonne filtrée	intervalle, colonne calculée indexée
analyse	`LIKE_DEBUT`	important	LIKE '%...' : balayage	plein texte, préfixe connu
analyse	`NOT_IN`	important	NOT IN (SELECT ...) : piège des NULL	NOT EXISTS corrélé
analyse	`NOLOCK`	important	lectures sales	READ_COMMITTED_SNAPSHOT
analyse	`IDENTITY`	important	@@IDENTITY et déclencheurs	SCOPE_IDENTITY(), OUTPUT
analyse	`GOTO`	opportunité	flot de contrôle difficile	blocs IF / TRY
analyse	`CURSEUR`	important	traitement ligne à ligne	ensembliste ; LOCAL FAST_FORWARD
analyse	`INJECTION`	critique	SQL dynamique concaténé	sp_executesql paramétré, QUOTENAME
analyse	`TRANSACTION_SANS_TRY`	important	transaction hors TRY... CATCH	TRY...CATCH + XACT_ABORT

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Module	Règle	Niveau	Signification	Correction
analyse	`WAITFOR`	opportunité	attente active	SQL Agent, Service Broker
analyse	`ROWCOUNT`	opportunité	SET ROWCOUNT obsolète	TOP (n)
analyse	`BOUCLE`	opportunité	boucle WHILE	traitement ensembliste
dependances	`REF_INTROUVABLE`	important	référence sans objet correspondant	objet supprimé, base non analysée, nom dynamique
dictionnaire	`RGPD_SENSIBLE`	critique / important	table à données personnelles (très) élevée	registre, conservation, accès, pseudonymisation
types	`TYPE_INCOHERENT`	important	même donnée, types SQL différents	type utilisateur, alignement
types	`NOMMAGE_INCOHERENT`	opportunité	plusieurs noms pour une donnée	convention unique
duplications	`DUPLICATION`	important / opportunité	instructions dupliquées ou voisines	vue, fonction, procédure commune
performances	`TABLE_SCAN`	critique / important	balayage coûteux dans le plan	index couvrant, prédicats SARGables
performances	`JOINTURE_COUTEUSE`	important	Hash Match coûteux	index sur les colonnes de jointure
performances	`INDEX_MANQUANT`	critique / important	index suggéré par SQL Server	CREATE INDEX fourni
performances	`JOINTURE_SANS_INDEX`	opportunité	colonne de jointure non indexée	CREATE INDEX
performances	`CURSEUR_INUTILE`	critique / important	curseur fréquemment exécuté	réécriture ensembliste
performances	`FILTRAGE_TARDIF`	critique / important	filtre non SARGable fréquemment exécuté	réécrire le prédicat
patrimoine	`COMPLEXITE`	important / opportunité	complexité > 12 (> 20 : important)	découper par phases
patrimoine	`INUTILISE`	opportunité	objet sans utilisateur ni exécution	vérifier puis archiver

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

ANNEXE B · MODÈLE DE DONNÉES

Les 33 tables de l'entrepôt

Toutes les tables de atlas.duckdb et leurs colonnes, pour écrire vos propres requêtes et rapports.

B.1 Les tables et leurs colonnes



Dans ce chapitre, vous apprendrez à...

- ✓ trouver la table qui contient une information
- ✓ connaître le nom exact des colonnes
- ✓ joindre les tables entre elles

B.1 Les tables et leurs colonnes

Table	Colonnes
`ast`	objet_id, id, parent, genre, libelle, phase, ligne_debut, ligne_fin, ordre
`captures`	id, quand, serveur, base, objet, texte, parametres, duree_ms, lignes, application, utilisateur, origine
`colonnes`	objet_id, serveur, base, sch, table_nom, nom, ordre, type_sql, longueur, prec, echelle, nullable, identite, calculee, defaut, pk, fk_vers, distincts, nuls, exemples
`dependances`	de_id, vers_id, vers_nom, nature, ligne, interbase, interserveur, origine, frequence, volume_moy, volume_max, complexite
`diagnostics`	objet_id, module, regle, niveau, message, suggestion, correctif, ligne
`dico_colonnes`	objet_id, colonne, description, type_metier, sensibilite, rgpd, categorie_rgpd, cardinalite
`dico_tables`	objet_id, objet_metier, famille, nature, sensibilite, rgpd, description
`duplications`	groupe, genre, objet_id, num, ligne, empreinte, similarite, extrait, proposition
`dyn_variantes`	empreinte, objet_id, sql, premiere, derniere, occurrences, origine
`espaces`	nom, contenu, modifie
`ext_colonnes`	source_id, serveur, base, sch, table_nom, nom, ordre, type_sql, longueur, prec, echelle, nullable, identite, calculee, defaut, pk, fk_vers
`ext_dependances`	source_id, serveur, base, sch, nom, ref_serveur, ref_base, ref_sch, ref_nom, nature
`ext_index_manquants`	source_id, serveur, base, sch, table_nom, egalite, inegalite, inclus, impact, demandes

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Table	Colonnes
`ext_jobs`	source_id, serveur, job, etape, nom_etape, commande, base, heure_debut, duree_moy_s, frequence, actif
`ext_objets`	source_id, serveur, base, sch, nom, genre, parent_nom, proprietaire, cree, modifie, lignes, taille_ko, definition, cible, detail
`ext_profils`	source_id, serveur, base, sch, table_nom, colonne, distincts, nuls, exemples
`ext_stats`	source_id, serveur, base, sch, nom, texte, executions, cpu_ms, duree_ms, lectures, ecritures, lignes, lignes_max, memoire_ko, derniere, plan
`favoris`	cle, libelle, cree
`index_manquants`	objet_id, serveur, base, sch, table_nom, egalite, inegalite, inclus, impact, demandes
`instructions`	objet_id, num, ligne_debut, ligne_fin, genre, phase, profondeur, texte, normalise, empreinte, tables
`jointures`	objet_id, ligne, t1, c1, t2, c2
`journal`	quand, niveau, message
`meta`	cle, valeur
`metriques`	objet_id, lignes_code, nb_instructions, complexite, entrant, sortant, cohesion, dette_min, maintenabilite, reutilisation, classe, score
`objets`	id, cle, serveur, base, sch, nom, genre, parent_id, parent_nom, proprietaire, cree, modifie, lignes, taille_ko, definition, cible, detail, source_id
`refs_brutes`	objet_id, ref, p1, p2, p3, p4, nature, ligne, distant, sql
`regles`	objet_id, ligne, genre, texte, normalise, empreinte
`sources`	id, nom, genre, serveur, port, base, auth, utilisateur, options, actif, limite, dossier, cree, analysee
`sql_dynamique`	objet_id, ligne, mode, variable, sql_reconstitue, parametres, types, empreinte, valeurs
`stats_requetes`	objet_id, serveur, base, texte, executions, cpu_ms, duree_ms, lectures, ecritures, lignes, lignes_max, memoire_ko, derniere, plan, cout, operateurs
`travaux`	id, lot, genre, libelle, cible, source_id, priorite, etat, agent, cree, debut, fin, duree, progression, message, essais
`types_metier`	code, libelle, type_sql, motif, occurrences, utilisations, colonnes, variantes
`variables`	objet_id, nom, genre, type_sql, ligne, defaut

Les jointures usuelles

`objets.id` est la clé de tout : `colonnes.objet_id`, `dependances.de_id` / `vers_id`, `diagnostics.objet_id`, `metriques.objet_id`, `dico_tables.objet_id`, `instructions.objet_id`, `sql_dynamique.objet_id`.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

ANNEXE C · AIDE-MÉMOIRE

Les raccourcis et les gestes

Raccourcis clavier, gestes de souris et emplacement des principales commandes.

C.1 Raccourcis clavier · C.2 Gestes de souris · C.3 Où est… ?



Dans ce chapitre, vous apprendrez à…

- ✔ aller vite au clavier
- ✔ retrouver une commande

C.1 Raccourcis clavier

Touche	Action
F5	lancer l'analyse complète ; dans l'éditeur en débogage : continuer
Ctrl+F	rechercher
Ctrl+D	ajouter / retirer des favoris
Alt+← Alt+→	objet précédent / suivant
F12	aller à la définition (éditeur)
F9	point d'arrêt
F10 F11	pas à pas / pas détaillé
Ctrl+S	enregistrer le script
Entrée dans la recherche	ouvrir le premier résultat

C.2 Gestes de souris

Où	Geste	Effet
explorateur	double-clic sur un objet	ouvrir son code

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Où	Geste	Effet
explorateur	double-clic sur serveur / base / schéma	ouvrir sa cartographie
carte	molette · glisser · double-clic	zoom · déplacement · descendre d'un niveau
grilles	double-clic	rendre l'objet courant, ouvrir le détail
séparateurs	glisser les trois points	redimensionner
éditeur	clic dans la marge	point d'arrêt

C.3 Où est… ?

Je cherche…	C'est…
déclarer un serveur	+ Source ou Sources › Nouvelle source…
modifier ou supprimer une source	Sources › Modifier / Supprimer…, ou double-clic sur la source
relancer ce qui a échoué	↻ Relancer ou Analyse › Relancer les travaux en erreur
exporter	Fichier › Exporter… (CSV, Markdown)
changer de thème	Affichage › Thème sombre / clair
mettre la navigation sur un autre écran	⧉ (barre d'outils)
capturer les appels	onglet Captures › Script de capture (XE)…
brancher une IA	onglet Assistant › Réglages…

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

ANNEXE D · GLOSSAIRE

Les mots d'Atlas SQL

Les termes techniques employés dans le guide et dans l'application.

D.1 Glossaire



Dans ce chapitre, vous apprendrez à...

- retrouver le sens d'un terme

D.1 Glossaire

Terme	Définition
Agent	fil d'exécution qui prend et exécute les travaux de la file
AST	arbre syntaxique abstrait : la structure du code
Campagne	ensemble des travaux lancés par ▶ Analyser
Cohésion	part du code d'un objet qui porte sur sa table principale
Complexité cyclomatique	nombre de chemins indépendants à travers le code
Constat	remarque d'Atlas sur un objet (règle, niveau, suggestion)
Couplage	nombre de dépendances entrantes (qui m'utilise) et sortantes (ce que j'utilise)
CTE	expression de table commune (`WITH`)
Dette technique	temps estimé pour corriger les constats d'un objet
DuckDB	base de données analytique embarquée qui porte l'entrepôt
Empreinte	condensé du texte normalisé d'une instruction
Entrepôt	la base atlas.duckdb du projet
Extended Events (XE)	mécanisme de traces léger de SQL Server
Jaccard	indice de similarité : communs ÷ distincts
Jeton	plus petit élément d'un code : mot, chaîne, nombre, variable, opérateur
Maintenabilité	indice 0-100 de facilité de maintenance
OPENQUERY	requête envoyée telle quelle à un serveur lié

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Terme	Définition
Phase	rôle d'une instruction : contrôle, chargement, écriture…
RGPD	règlement général sur la protection des données
SARGable	prédicat qui permet d'utiliser un index
Serveur lié	serveur distant déclaré dans SQL Server (`sys.servers`)
Synonyme	alias d'un objet, éventuellement distant
Synoptique	suite des phases d'un module
Travail	unité de traitement de la file d'attente
Type métier	regroupement des colonnes qui désignent la même donnée

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

ANNEXE E · DÉPANNAGE

Questions fréquentes

Les messages que vous pourriez rencontrer, et ce qu'il faut faire.

E.1 Problèmes et solutions



Dans ce chapitre, vous apprendrez à...

- ✔ résoudre un problème de connexion
- ✔ comprendre un travail en erreur ou bloqué
- ✔ expliquer une vue vide

E.1 Problèmes et solutions

Symptôme	Cause et solution
Atlas indique « base du projet verrouillée ».	Une autre instance d'Atlas (ou un autre outil) a ouvert la même base. Fermez-la, ou ouvrez un autre projet.
« TLS negotiation failed » au test.	Cochez *Faire confiance au certificat* ; Tester essaie aussi les autres réglages (3.4). Si rien ne passe : chiffrement strict, passer par ODBC.
« extension DuckDB mssql non installée ».	Le poste n'a pas accès à Internet (ou passe par un proxy) au premier lancement. Installez l'extension une fois sur un poste connecté, ou utilisez ODBC.
Erreur 451, conflit de classement, à l'extraction.	Corrigé dans la version actuelle (`COLLATE DATABASE_DEFAULT`). Mettez Atlas à jour puis relancez.
Erreur « Deprecated lambda arrow » (DuckDB).	Corrigé : Atlas utilise la syntaxe `lambda x: …` des versions récentes de DuckDB.
Un travail est « bloqué ».	Une dépendance stricte a échoué : lisez son message, corrigez, puis ↻ Relancer.
L'onglet Perfs est vide.	Le compte n'a pas `VIEW SERVER STATE`, ou l'option statistiques est décochée, ou le serveur vient de redémarrer.
Une procédure est classée « Inutilisé » alors qu'elle sert.	Elle est appelée par une application : Atlas ne la voit pas. Capturez les appels (chapitre 19).

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr

Symptôme	Cause et solution
Une source n'apparaît pas dans l'explorateur.	Elle apparaît « à analyser (F5) » tant qu'aucun objet n'a été extrait ; lancez l'analyse.
Le mot de passe est redemandé à chaque lancement.	Cochez *Mémoriser sur ce poste* dans la fiche de la source.

© 2026 Jean-Marc QUÉRÉ, sonaliwan.fr · SIRET 13033319800013 · Atlas SQL, Lexis+ et wdlexis+ sont distribués sous licence CC BY-NC-SA 4.0 · usage commercial : autorisation de l'auteur · metalab (at) sonaliwan.fr